

A Work Project, presented as part of the requirements for the Award of a Master's degree
in Business Analytics from the Nova School of Business and Economics.

OPTIMIZING THE MATCH BETWEEN CONTACT MANAGERS AND CLIENTS FOR A
PORTUGUESE TELCO COMPANY

Maria Teresa Daffan

Runa Maria Kleppek

Marvin Schumann

Raquel Santos

Work project carried out under the supervision of:

Patrícia Xufre

Nuno André da Silva

16-12-2025

Abstract

This thesis investigates how predictive modelling and AI-based methods can reduce operational costs while preserving service quality in a Portuguese telecom contact centre. Chapter 1 develops a fragmented XGBoost-based framework that predicts call duration (TMC), first contact resolution (FTR) and technician visits (OT), recomposed into a cost-based routing rule. Chapter 2 designs and tests an LLM-augmented conversational IVR. Chapter 3 compares fragmented versus end-to-end cost models. Chapter 4 builds a GC-topic training recommender, and Chapter 5 evaluates reinforcement learning policies against a supervised greedy XGBoost baseline using NOS operational data and context.

Keywords

Machine Learning, Telecommunications, Contact Centers, Cost Prediction, Large Language Models, Conversational AI, Reinforcement Learning, Recommendation Systems, Upskilling, End-to-end Models, Action Masking, Simulator Validation

This work used infrastructure and resources funded by Fundação para a Ciência e a Tecnologia (UID/ECO/00124/2013, UID/ECO/00124/2019 and Social Sciences DataLab, Project 22209), POR Lisboa (LISBOA-01-0145-FEDER-007722 and Social Sciences DataLab, Project 22209) and POR Norte (Social Sciences DataLab, Project 22209).

Table of Contents

1. Group Part – PBL Project	13
1.1 Introduction	13
1.2 NOS Business Context	14
1.3 Theoretical and Literature Background	15
1.3.1 Predictive Modeling in Call Center Optimization	16
1.3.2 Cost Optimization and Performance Metrics in Telecom	17
1.3.3 Machine Learning Models for Predictive Tasks	19
1.4 Data Overview and Preparation	22
1.4.1 Data Sources and Structure	23
1.4.2 Exploratory Data Analysis	24
1.4.2.1 Call Data Analysis	24
1.4.2.1 GC Data Analysis	26
1.4.2.3 Client Data Analysis	29
1.4.3 Data Cleaning and Imputation	31
1.4.4 Feature Engineering	32
1.4.5 Target Variables	32
1.5 Modeling Approach and Development	33
1.5.1 Operational Context and Real-Time Input Pipeline	33
1.5.2 Preprocessing and Model Setup	34

1.5.3 Model Development.....	36
1.5.4 Evaluation Metrics	38
1.5.5 Cost Construction Procedure.....	39
1.6 Results and Discussion	40
1.6.1 Feature Importance.....	42
1.7 Plan for Testing.....	44
1.7.1 Testing Objectives and Setup	44
1.7.2 Offline and Online Testing Approaches	44
1.7.3 KPI Definition and Evaluation Strategy.....	46
1.7.4 Retraining and Monitoring Plan	46
1.8 Limitations and Further Research	47
1.8.1 Bias and Fairness.....	49
1.9 Main Insights and Implications for NOS	50
1.8 Conclusion.....	51
2. Automating call intake with LLM-augmented conversational IVR - Runa Kleppek.....	52
2.1 Current IVR System at NOS	52
2.2 Problem Statement and Research Objectives	53
2.3 Literature Review	54
2.3.1 IVR Systems: Current State and Limitations	54
2.3.2 LLMs in Customer Service and Conversational AI	55

2.3.3 Opportunities and Risks of LLM-Augmented IVRs	56
2.4 Data and Methodology	57
2.4.1 Research Goal and Approach	57
2.4.2 Prototype Design	58
2.4.3. Conversational Model	60
2.4.3.1. Information Fields	60
2.4.2.2. Prompt Design	61
2.4.2.3. Human-in-the-Loop	62
2.4.4 Prototype Testing	63
2.4.4.1 Metrics	63
2.4.4.2. Data Analysis and Results	64
2.6 Discussion and Outlook	65
3. Comparing End-to-End and Fragmented Models for Cost Prediction in Telecommunications – Maria Teresa Daffan.....	67
3.1 Introduction	67
3.2 Literature Review	68
3.2.1 Conceptual Foundations of Fragmented and End-to-End ML Architectures	68
3.2.2 Evidence from Applied Domains	68
3.2.3 Balancing Interpretability and Performance	69
3.3 Methodology	70

3.3.1 Target Variable Definition	71
3.3.2 Preprocessing and Feature Selection	72
3.3.3 Models Development	73
3.3.4 Evaluation Metrics	75
3.4 Results and Discussion	76
3.4.1 Feature Importance Analysis	77
3.4.2 Interpretability trade off	78
3.5 Managerial Implications	79
3.5.1 Recommended Actions for NOS	79
3.6 Limitations and further research	80
3.7 Conclusion	81
4. Identifying Personalized Training Opportunities through Recommendation Systems - Raquel Santos	82
4.1 Introduction	82
4.2. Literature Review	83
4.2.1 Fundamentals of Recommender Systems	83
4.2.2 Evaluation Metrics	84
4.2.3 Recommendation Systems in Learning and Upskilling Contexts	85
4.2.4 Research Gap and Objectives	85
4.3. Methodology	85

4.3.1 Data Preparation and Feature Aggregation	86
4.3.2 Rating Construction and Normalization.....	87
4.3.3 Collaborative Filtering Framework	88
4.3.4 End-to-End Workflow Overview	88
4.3.5 Models Tested.....	89
4.3.5.1 User-Based Collaborative Filtering.....	89
4.3.5.2 Item-Based Collaborative Filtering	89
4.3.5.3 Matrix Factorization	89
4.3.6 Model Evaluation	90
4.3.6.1 Train–Test Split Strategies.....	90
4.3.6.1 Evaluation Metrics	90
4.3.7 Business-Weighted Prioritization	91
4.4 Results	92
4.4.1 Accuracy and Ranking Performance	92
4.4.2 Coverage, Sparsity, and Structural Improvements	93
4.4.3 Robustness Across Split Strategies.....	94
4.5 Discussion	94
4.6 Business Impact and Future Work.....	95
4.7 Conclusions	96

5. Optimizing Call Routing Decisions Using Reinforcement Learning – Marvin Schumann

.....	97
5.1 Introduction	97
5.2 Literature Review	97
5.2.1 RL Applications to Call Center Routing	97
5.2.2 Reinforcement Learning Foundations	98
5.2.3 Deep RL Algorithms: Policy-Based vs Value-Based Methods	98
5.2.4 Action Masking for Constrained RL	99
5.2.5 Research Gap and Contribution	99
5.3 Markov Decision Process Formulation	99
5.3.1 State Space	99
5.3.2 Action Space.....	100
5.3.3 Reward Function	100
5.3.4 Discount Factor and Transition Dynamics	100
5.4 Simulator Validation.....	101
5.5 Methodology	102
5.5.1 Environment and Training.....	102
5.5.2 Action Masking Implementation	102
5.5.3 Algorithm Comparison: PPO vs. DQN	102
5.5.4 Baseline Policies and Evaluation	103

5.6 Results	103
5.6.1 Action Masking Validation Across Algorithms	103
5.6.2 Policy Performance Comparison	104
5.6.3 Training Dynamics	106
5.7 Discussion	107
5.7.1 Why RL Underperforms Greedy XGBoost	107
5.7.2 Algorithm-Independent Findings	108
5.7.3 Impact of Large Action Space	109
5.7.4 Limitations	109
5.8 Managerial Implications	110
5.8.1 Recommendation for NOS	110
5.8.2 When to Use RL vs. Heuristics	110
5.8.3 Alternative Approaches for NOS	110
5.9 Future Work	111
5.10 Conclusion	112
6. References	113
7. Appendix	124
Deleted	140

Formatted: Normal

Table of Figures

Figure 1. Structure and Overlap of Datasets Structure.....	23
Figure 2. Distribution of Leg Duration	24
Figure 3. Distribution of First Call Resolution Rate and Technician Dispatch Rate	25
Figure 4. FTR and TMC Correlation Matrix.....	26
Figure 5. Objective vs self-reported resolution (FTR vs RPC).....	27
Figure 6. Number of GCs by High Performer Count	28
Figure 7. Distribution of Client Age and Antiquity	30
Figure 8. Incoming Call Process	33
Figure 9. Overview of Model Architecture	34
Figure . Overview of Modelling Approach	37
Figure . Number of Calls per Topic/Category	52
Figure . Comparison of current and LLM-augmented call intake process (Prototype)	58
Figure . Example call intake dialogue in the NOS Voice IVR prototype.....	59
Figure . Cost Savings Calculation	65
Figure . Fragmented vs. E2E Model Architectures	71
Figure . Original and Refined Labels Before and After log Transformation	72
Figure . Construction of the GC–Topic Matrix	86
Figure . Conversion of Average Metrics into Ratings Using Median-Centered σ -Bands	88
Figure . End-to-End Pipeline for Model Training, Prediction, and Prioritization.....	88
Figure . Action masking impact - from 6% to 104% efficiency.....	104
Figure . Routing policy performance comparison.....	105
Figure . RL training dynamics - early plateau indicates learning limit	106
Figure . Cost distribution by routing policy	107

Figure . Simulator fidelity - predicted vs. actual costs.....	111
---	-----

List of Tables

Table 1. Feature Engineering Rationale	32
Table 2. TMC Model Performance Comparison	41
Table 3. FTR Model Performance Comparison	41
Table 4. OT Model Performance Comparison	42
Table 5. Overview of Variables collected by the Prototype	60
Table 6. E2E Model Performance Comparison.....	76
Table 7. Overview of Recommender System Methods.....	84
Table 8. Dataset Characteristics (Shape, Coverage, Sparsity).....	92
Table 9. Model Performance Metrics	92

List of Abbreviations

AT: Atendimento Técnico - Technical Line

GC: Gestor de Cliente - Contact Manager / Agent

OT: Ordem de Trabalho - Technician Visits

PBL: Project-Based Learning

RPC: Resolução Primeira Chamada - First Call Resolution (self-reported FTR)

SAC: Serviço de Atendimento ao Cliente - Customer Service (general) Line

TMC: Tempo Médio de Chamada - Average Call Time

IF: Internet Fixe / Home WIFI

VF: Voice Fixe / Landline

TV: Television

AHT: Average Handling Time

CRM: Customer Relationship Management

FTR: First Call Resolution

IVR: Interactive Voice Response

KPI: Key Performance Indicator

AUC: Area Under the ROC Curve

DiD: Difference-in-Differences

MAP: Mean Average Precision

MAE: Mean Absolute Error

ML: Machine Learning

NDCG: Normalized Discounted Cumulative Gain

NN: Neural Network

PSI: Population Stability Index

RMSE: Root Mean Square Error

SHAP: SHapley Additive exPlanations

SMOTE: Synthetic Minority Over-sampling Technique

SUTVA: Stable Unit Treatment Value Assumption

XGBoost: Extreme Gradient Boosting

GenAI: Generative AI

LLMs: Large Language Models

AI: Artificial Intelligence

STT: Speech-to-text

TTS: Text-to-speech

HITL: Human in the loop

1. Group Part – PBL Project

1.1 Introduction

“For 64% of customers, service quality is the single most important factor that sets companies apart. Get it wrong and drive customers away. Get it right, and you can build lifelong loyalty” (Accenture 2025).

In today’s competitive environment, customer service has become a strategic differentiator that determines whether customers remain loyal or switch to a competitor. This is especially true in the telecommunications industry, where offerings are largely standardized across providers. As a result, service quality and the ability to resolve issues efficiently has become a key driver of customer satisfaction and retention. To meet such expectations, telecom companies are increasingly turning to data-driven technologies to improve how their customer service operates. NOS, one of Portugal’s largest telecom providers, is part of this shift and has invested heavily in advanced analytics and artificial intelligence to make its service operations more efficient. One notable initiative is the Project-Based Learning (PBL) collaboration with NOVA SBE, which developed a machine learning-based framework to improve the matching of incoming calls and Contact Managers (GC, i.e., Gestor de Cliente). Building on the insights generated by the project, this study investigates whether predictive modelling can reduce operational costs by improving call-routing performance in a real-world telecommunications setting.

The first chapter outlines the full scope of the PBL project and describes how a fragmented modelling approach was developed by estimating each component of the call-cost function separately and then integrating these elements into a unified framework. It proceeds through a structured literature review, a detailed methodological description, and an analysis of the model’s results, concluding with a discussion of their implications for NOS.

Following the insights gained from the PBL project, this thesis develops four additional research avenues that both broaden the focus to wider telecommunications operations and offer alternative ways of addressing the research question at hand. Firstly, Chapter 2 explores how large language models (LLMs) can be integrated into the initial part of a customer service call to capture intent, extract relevant information, and improve the quality of routing inputs. Furthermore, Chapter 3 offers a comparative analysis of modelling strategies for telecom operational optimization, examining the benefits and drawbacks of end-to-end versus modular designs. Chapter 4 examines the potential of Collaborative Filtering methods for detecting weaknesses of the GCs, predicting unseen performance, and ranking training priorities according to business impact. Finally, Chapter 5 presents a dynamic framework for continuous learning and decision optimization, enabling adaptive call routing strategies.

1.2 NOS Business Context

Every month, NOS processes around 300,000 customer service calls, divided between two main service lines: the technical line (AT, i.e., Atendimento Técnico) and the generalist line (SAC, i.e., Serviço Atendimento ao Cliente). Each call costs approximately €0.35 per minute, and technician visits (OTs, i.e., Ordem de Trabalho) add an additional €22 per case. Given this cost structure, even small improvements in routing efficiency can generate substantial cost savings. Currently, calls are routed through a rule-based Interactive Voice Response (IVR) system, which directs customers to AT or SAC and then assigns calls to available GCs based on simple business rules and availability. These rules are shaped largely by contractual agreements with external partners rather than by any form of optimization logic. This static approach ignores critical contextual information such as the customer's history, call characteristics, and GC performance patterns. As a result, many calls are not matched to the most suitable GC, leading to longer handling times, repeat calls, and unnecessary

Commented [MT1]: Add current KPIs briefly

Commented [PX2]: this is the first time ivr appears, you need to say what it is. Interactive Voice Response (IVR)

technician visits. There are three main Key Performance Indicators (KPIs) that are currently tracked by NOS:

- First Call Resolution (FTR): Share of calls that are fully resolved during the first contact, without requiring a callback, transfer or follow up ticket
- Average Call Duration (TMC, i.e., Tempo Médio de Chamada): Average time in seconds a GC spends on a call from start to finish
- Technician Visits (OT): Number of on-site technicians being dispatched to a customer

The main challenge is that decision-making in the GC allocation process is currently not data-driven. Although NOS possesses extensive historical data, including customer attributes, call metadata, and the KPIs above, these are not yet used to make routing decisions. The objective of the PBL project is therefore to develop a predictive matching model that uses historical data to recommend the most suitable GC for each incoming call.

1.3 Theoretical and Literature Background

Inbound call centers serve as the primary link between customers and service providers in many service industries, including telecom (Gans et al. 2003). In these centers, incoming calls are handled by any qualified agent within a group, meaning that performance depends on the collective capabilities of the team rather than on individual employees (NAQC 2009).

Call centers use integrated technologies to manage interactions and to collect data that can later be used to measure performance. IVR systems collect basic caller information, such as the type of service requested. The system can recognize the client from the phone number and, consequently, agents can access the customer's account details in real time. Customer Relationship Management (CRM) tools record client information, such as past interactions and account history, to support faster, more personalized service. Workforce management systems aggregate data from multiple

agents to monitor performance, forecast demand, and improve scheduling efficiency (NAQC 2009; Gans et al. 2003). These datasets enable predictive modeling and cost optimization at both the individual and organizational levels (NAQC 2009).

1.3.1 Predictive Modeling in Call Center Optimization

Modern call centers sit at the intersection of customer experience and operational efficiency, making them a central application area for predictive modeling aimed at improving service quality while containing costs. Over the past decade, research has increasingly leveraged statistical and machine-learning approaches to anticipate KPIs such as call arrival rates, call durations, first-call resolution, abandonment likelihood, customer satisfaction, and overall cost efficiency (Nakkazi and Akay 2024). Traditional queueing-theoretic models, although analytically robust, often struggle to represent the stochastic, behavioral, and temporal complexities of contemporary service systems, particularly those with multichannel interactions and highly variable demand. Predictive models, by contrast, incorporate rich historical and contextual information to capture these dynamics more effectively and to support resource-allocation decisions in near real time.

A substantial body of work has thus emerged around call-arrival forecasting, a foundational element of workforce management. Early contributions such as Ibrahim et al (2012) highlighted the limitations of classical Poisson-based formulations when arrivals are strongly nonstationary and showed how mixed-effect and multivariate models can better accommodate intraday and interday dependencies. Subsequent studies extended this line of inquiry using more flexible learning algorithms. Janković (2021), for instance, demonstrated that ensemble tree-based models can significantly improve predictive accuracy over autoregressive benchmarks in hourly arrival forecasting. More recent work applies deep learning: Chacón et al. (2023) used recurrent neural networks to capture complex seasonal and intraday patterns, outperforming traditional time-series

models particularly in settings with pronounced nonlinear seasonality.

Beyond arrival prediction, machine-learning methods have been applied to service-level estimation, abandonment behavior, and cost-relevant performance metrics. Li, Wang, and Koole (2019) proposed a simulation-informed learning framework in which gradient-boosted models approximate the relationship between staffing decisions and resulting service levels, reducing the need for computationally intensive simulation during daily planning. Research also addresses behavioral phenomena that influence operational costs. For example, Castellanos et al. (2025) examined “silent abandonment” in digital support channels, using classification and expectation-maximization techniques to estimate unobserved dropout behavior and its consequences.

Collectively, these studies illustrate the evolution from traditional statistical models to contemporary ensemble and deep learning approaches, as well as a growing emphasis on interpretable, operationally meaningful metrics. This progression reflects a broader shift toward integrating predictive analytics directly into decision-support workflows, laying the foundation for cost-oriented optimization frameworks such as the one explored in this thesis.

1.3.2 Cost Optimization and Performance Metrics in Telecom

Cost efficiency in call centers is primarily determined by effective management of staff and capacity, which can represent 60-70% of total operating costs (Gans et al., 2003). Key performance and cost metrics include average call duration, First-Call Resolution, and the frequency of follow-up actions such as technician visits (ICMI n.d.; Gans et al. 2003; Holley and Layne, 2011).

FTR, also known as First-Call Resolution (FCR) or the “one and done” rate, shows the percentage of calls solved during the first contact, without transfers, escalations, or follow-ups (NAQC 2009; ICMI n.d.). The ICMI guide notes that FTR is a useful way to track performance over time, even though it is hard to standardize across companies (ICMI n.d.). According to NAQC (2009), FTR

Commented [RK3]: these were already explained in 1.2 so maybe reference here. Also I think it's fine to just write the abbreviations for FTR etc. since they have already been explained

can be measured for groups or individuals, reflecting both the center's overall efficiency and each agent's ability to solve issues without extra help or repeat calls. High FTR usually means lower costs, since unresolved problems lead to more callbacks and extra work. However, focusing only on FTR can be risky, since solving a call on the first try but taking too long may not be efficient (ICMI n.d.). For cost efficiency in telecom support, it is important to look at FTR and average call duration, as both affect costs and customer satisfaction.

Average Handle Time (AHT) measures call duration calculated as the sum of talk time and after-call work (NAQC 2009; ICMI n.d.). It reflects the total workload required to complete a customer interaction and, together with call volume, determines staffing and scheduling requirements (Gans et al. 2003). Literature identifies several factors that influence AHT, including call type, agent specialization, day of the week, time of day, process design, and system support (Gans et al. 2003; Plaza & Pawlik 2021; Stolletz 2003). Reducing AHT can improve efficiency. However, overly aggressive targets may compromise call quality and lower First Call Resolution, increasing repeat contacts (Schneider et al. 1998). Therefore, AHT should be considered alongside FTR and cost metrics to capture the trade-offs between efficiency, resolution, and service quality.

In telecom support, technician visits (OTs) represent a significant downstream cost component, as unresolved customer issues often require field service interventions (Stolletz 2003; Gans et al. 2003). Calculating the cost per OT typically involves aggregating labor, travel, and administrative overheads and dividing them by the number of OTs generated (Kaplan & Anderson, 2004). Integrating OT cost modeling with average call duration and FTR can reveal whether improvements in contact center performance can reduce the need for field intervention (Elkan 2001; Plaza & Pawlik 2021).

Cost per Call is a widely used financial metric in contact center management, calculated by dividing total operating costs by the number of calls handled within a specified period (ICMI n.d.). Although

Commented [RK4]: has been explained already so abbreviation is enough here

this metric provides a rapid assessment of efficiency, it requires contextual interpretation: in some cases, an increase in the cost per call may indicate process improvements.

In outsourced telecom environments, cost structures frequently differ substantially from those of in-house centers. External providers are typically compensated based on variable costs, such as per-minute or per-work-order rates, rather than comprehensive overhead costs. As a result, financial outcomes are primarily influenced by the management of these variable costs, since each additional minute or service action directly increases expenses (Gans et al. 2003; Stollertz 2003).

Activity-Based Costing and Time-Driven Activity-Based Costing both focus on linking costs to the activities that cause them (Kaplan & Anderson, 2004). These methods show how resource use leads to total spending and help connect operational metrics to cost outcomes. However, most studies examine duration, resolution, and follow-up separately. Newer research in cost-sensitive learning suggests combining these factors in models that predict costs based on expected outcomes (Elkan 2001). Recent work by Plaza and Pawlik (2021) shows that improving contact center systems with tools such as omnichannel communication, automation, and AI-assisted routing can change key metrics, including AHT, First Call Resolution, Cost per Contact, Abandon Rate, and Occupancy Rate. Their results show that updating technologies like IVR, CRM, and automated routing can improve cost efficiency. Despite advances in linking FTR, AHT, and technician visit rates to cost, the telecom support field continues to lack a unified, data-driven framework that integrates these key metrics into a single cost measure.

1.3.3 Machine Learning Models for Predictive Tasks

This section provides an overview of the main machine-learning approaches relevant to predictive modeling in call-center operations. It outlines four major algorithms that can be used to predict the three main KPIs that are currently tracked by NOS in their call center operations: FRT, TMC and

OT. Machine-learning models have shown major potential for addressing complex resource-allocation problems, such as in call centers. By learning from historical data, they can predict future performance and thereby enable intelligent, adaptive allocation that optimizes productivity, reduces costs, and improves service quality (Nayak 2025). Unlike traditional rule-based strategies, ML-based systems adapt dynamically to environmental changes and capture nonlinear relationships among multiple performance features. They therefore support organizations operating in data-rich environments by predicting key variables that can guide assignment decisions (Morariu et al. 2020). Depending on the task, regression models are applied to predict continuous variables, while classification models estimate binary outcomes.

Linear regression represents one of the foundational methods for predictive analytics and is widely used for modeling continuous outcomes. It assumes a linear relationship between a dependent variable and one or more independent variables, and it estimates coefficients that minimize the sum of squared residuals (Lu2021). This simplicity makes it interpretable and computationally efficient. Therefore, it often serves as a baseline model. In the context of call-center analytics, linear regression can estimate how features such as call topic or agent experience influence total call duration.

Logistic regression builds on the ideas of linear regression but adapts them for classification tasks, making it well-suited for predicting binary outcomes such as FTR or the probability of a technician visit. Instead of modeling a continuous output, it estimates the log-odds of an event occurring as a linear function of the predictors and transforms this into probabilities bounded between 0 and 1 (Sperandei 2014; UCLA OARC 2023). Logistic regression remains widely used in operational research because it provides interpretable coefficients that can be directly expressed as odds ratios. That means they quantify the marginal effect of each predictor (Norton & Dowd 2018). However,

Commented [MT5]: The thing is that at this point in the thesis we haven't yet said that these are the three target variables, I'd keep it more general maybe?

Commented [RK5R2]: they were kind of explained in chapter 1.3.2 but more in general telco terms not specific dor NOS. I think it makes sense to at least broadly include the target variables and not just generally write about the models but we can discuss that later

Commented [MT6]: Wrong citation style

Commented [RM6R2]: fixed

it assumes a linear relationship between predictors and log-odds and can perform poorly when nonlinear effects or class imbalance are present (Chen et al. 2024).

Random Forest is an ensemble learning method that combines many decision trees to improve predictive accuracy and robustness. A decision tree is a flowchart-like model that repeatedly splits the data using simple if-then rules until it reaches a prediction at a terminal “leaf.” Random Forest builds a large number of such trees on random subsets of observations and features, then aggregates their predictions. This randomness reduces correlation between trees, lowers variance, and improves generalization (Breiman 2001). Random Forest can capture nonlinear relationships and complex interactions without explicit specification and works well with mixed data types. It is relatively resistant to overfitting and can provide variable-importance measures to highlight influential predictors (Louppe 2014). However, because it averages over many trees, it is less interpretable than simpler models such as linear or logistic regression. In predictive call-center analytics, it can leverage call, agent, and customer attributes to estimate outcomes such as TMC, FTR, or OT.

Extreme Gradient Boosting (XGBoost) represents an advanced implementation of gradient-boosted decision trees and is very powerful prediction algorithm. It builds trees sequentially, where each new tree corrects the residual errors of the previous ones, thereby minimizing a differentiable loss function via gradient descent (Chen & Guestrin 2016). XGBoost introduces algorithmic improvements such as regularization, efficient handling of missing values, and optimized computing that enhance both accuracy and scalability (Brownlee 2021). It supports both regression (continuous outcomes like TMC) and classification (binary outcomes like FTR or OT). Research shows that gradient-boosted tree ensembles consistently outperform simpler tree-based methods for tabular prediction tasks, although this comes at the cost of interpretability.

Because Random Forest combines many decision trees, individual decision paths are hard to interpret, so post-hoc explanation tools such as SHAP values are often used to improve transparency (Ergün 2023).

Neural networks are a group of machine-learning models that can learn complicated, nonlinear patterns in data. They are built from several layers of connected nodes, where each node takes an input, multiplies it by weights learned during training, adds a bias term, and transforms the result through a nonlinear activation function. By stacking multiple layers of these nodes, neural networks enable models to learn hierarchical representations of the data and capture high-dimensional interactions (Goodfellow, Bengio & Courville 2016; Zhang et al. 2023). During training, network parameters are optimized to minimize a loss function using gradient-based methods such as the Adam optimizer, which adapts learning rates per parameter to improve convergence stability and speed (Kingma & Ba 2017). To enhance generalization and prevent overfitting, regularization techniques are applied. Examples are dropout, which randomly deactivates a subset of neurons during training, and batch normalization, which normalizes activations within each mini-batch (Srivastava et al. 2014). Additional strategies, such as early stopping and learning-rate scheduling, further improve robustness on unseen data. Despite their flexibility and often strong predictive performance, neural networks are typically regarded as “black-box” models, and understanding the internal reasons for their predictions remains challenging (Molnar 2025).

1.4 Data Overview and Preparation

The data available for training covers all GCs, clients and calls active between January 2024 and October 2024. The modeling dataset is created by merging relevant columns from three master tables: Masterdata GCs, Masterdata Calls and Masterdata Clients. Figure 1 summarizes the structure of these tables and the key variables that link them. The preparation of the modeling

Commented [RK7]: I slightly changed the text in 1.4 and 1.4.1 and included the graphic to make it easier to understand

dataset was a foundational step, involving a multi-stage pipeline that integrates the three sources and prepares them for modeling. This pipeline was designed to solve two primary challenges: first, preserving temporal integrity by correctly matching calls to historical client snapshots, and second, systematically handling data sparsity through imputation rules based on explicit business logic.

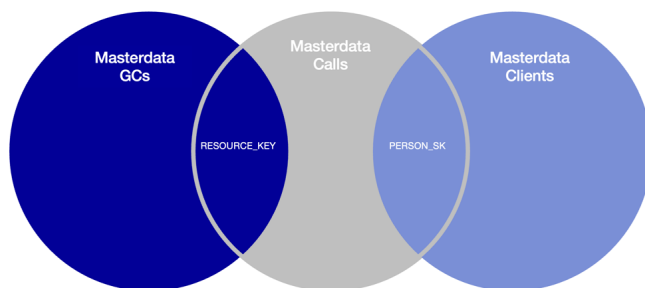


Figure 1. Structure and Overlap of Datasets Structure

1.4.1 Data Sources and Structure

The raw data was extracted from a PostgreSQL database comprising three primary entities: Firstly, Masterdata GCs contains information about GCs, including KPIs such as the number of calls handled and performance measures like FTR and TMC (both in total and per call category). Each GC has a unique RESOURCE_KEY, which also appears in the calls data. Secondly, Masterdata Calls contains information about calls at leg level, including timestamps and IVR-related information. It is the central table that links GCs and clients. Finally, Masterdata Clients contains client attributes, such as age and antiquity. Each client has a unique PERSON_SK, which also appears in the calls data. Therefore, the modeling dataset is built by merging Masterdata Calls with Masterdata GCs via RESOURCE_KEY and with Masterdata Clients via PERSON_SK, as illustrated in Figure 1. To prevent data leakage, a simple static merge between calls and clients was insufficient. Instead, a time-sensitive *pd.merge_asof* operation was employed to join the call and

client datasets. This method matches each call (based on its timestamp) to the most recent client snapshot on or before the call date, ensuring that all client features represent a valid historical state. Calls without a corresponding prior client record were subsequently dropped. Additionally, calls whose GC identifiers did not match any entry in the GC table were dropped, as these cases correspond to agents who are no longer active, resulting in the exclusion of nearly half of all calls. Remaining missing values were treated using a set of imputation rules that follow the business logic described later in this chapter.

Commented [MT8]: Added this sentence

1.4.2 Exploratory Data Analysis

1.4.2.1 Call Data Analysis

The Call dataset comprises 2.94 million call legs (i.e., interaction between one client and one GC not considering waiting times and handovers), with an average leg duration of 11.17 minutes and a notably high standard deviation of 10.11 minutes, indicating substantial variability in call handling times. Nevertheless, interactions are highly skewed toward short durations, with both call and leg distributions peaking near zero, indicating frequent brief or dropped contacts (Figure 2).

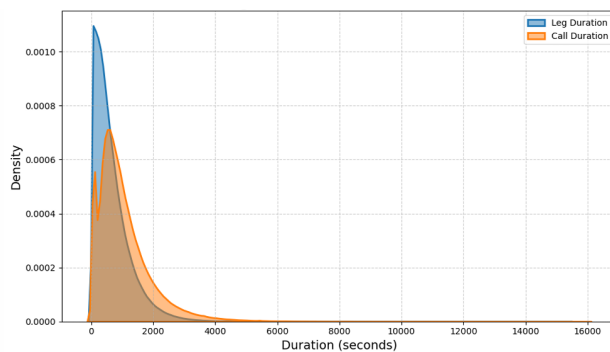


Figure 2. Distribution of Leg Duration

Additionally, Figure 3 shows that over 80% of calls are resolved on the first attempt, making FTR heavily skewed towards “resolved”, and fewer than 20% of calls require a technician dispatch, making OT skewed towards “no dispatch”.

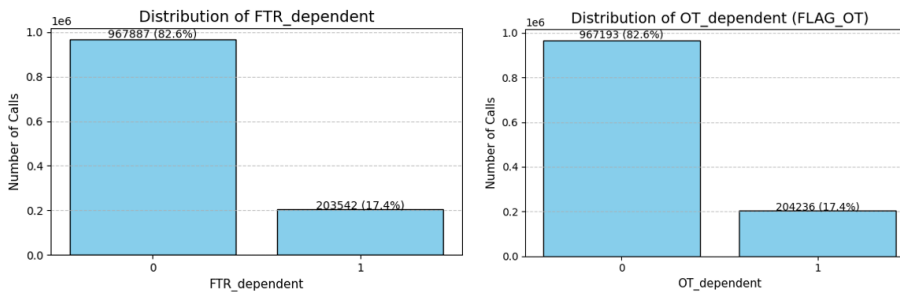


Figure 3. Distribution of First Call Resolution Rate and Technician Dispatch Rate

Furthermore, call volumes display a clear seasonal pattern, declining steadily from January to August, after which an unexpected drop in September suggests either operational changes or data-collection artefacts (Appendix 1.1). Finally, each call belongs to one of five categories, as per the output of the IVR: IF (*Internet Fixa, Broadband*), VF (*Voz Fixa, Landline*), TV (*Televisão, Television*), MOVEL (*Mobile*), or OTHER (*residual topics*). An analysis of the first-level typification codes shows that service-failure-related categories overwhelmingly dominate the reasons for customer contact across all these categories. This first-level typification is written manually by GCs and is therefore not standardized; a word-cloud and frequency analysis confirms that terms such as *serviço* and *falha* account for the majority of entries (Appendix 1.2). This manual specification is used primarily when the IVR returns the category *Other*, indicating that the client was unable to clearly identify the issue type and requiring the GC to provide a more specific description. (Appendix 1.3). Together, these patterns highlight a structurally imbalanced dataset, with short calls, strong temporal effects, and a dominant failure-related topic landscape.

1.4.2.1 GC Data Analysis

This section explores the main properties of the Masterdata GCs dataset. It contains one record per GC per day. Each row reports the number of calls handled and several performance metrics, both overall and broken down by IVR category. Given this structure, the dataset allows us to infer not only the performance profile of each individual GC, but also broader patterns in call characteristics and outcomes at an aggregate level.

Across the observation window, the dataset covers 653 GCs with an average of 3,145 calls and a standard deviation of 1,318 calls, indicating substantial heterogeneity in workload among GCs.

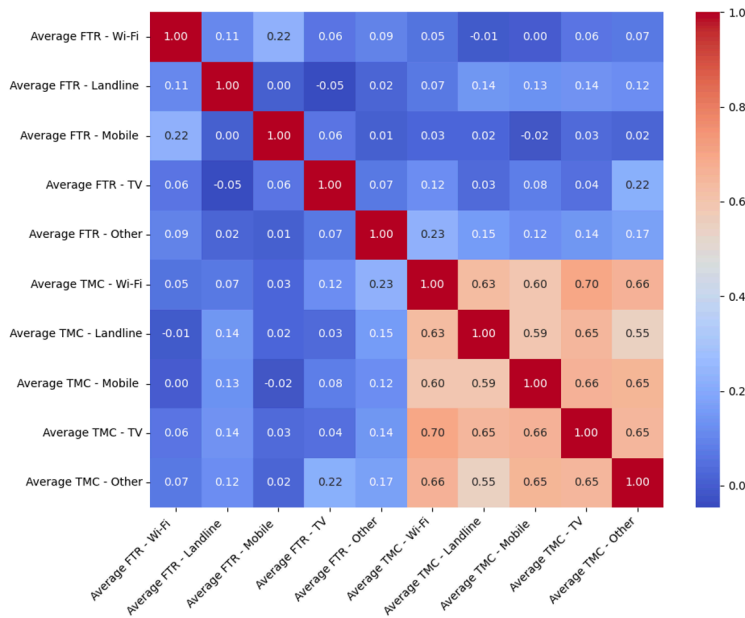


Figure 4. FTR and TMC Correlation Matrix

Figure 4 shows the correlation matrix between FTR and TMC across all categories. The diagonal blocks confirm that FTR and TMC of the same category are only weakly correlated, meaning that

GCs who resolve more calls on first contact are not necessarily the ones with shorter call times. In contrast, the correlations between TMC across different categories are consistently high: GCs who handle Wi-Fi calls quickly also tend to be fast in Landline, Mobile, TV and Other, while this cross-category pattern does not appear for FTR.

From a business perspective, this suggests that “speed” behaves like a general GC trait, whereas “quality of resolution” is more category specific. For routing and workforce management, it is therefore important to treat FTR and TMC as separate performance dimensions: GCs who are broadly efficient can be prioritized when queue pressure is high, while training and coaching efforts to increase FTR should be tailored by category instead of assuming that quality gains will automatically transfer across all types of calls.

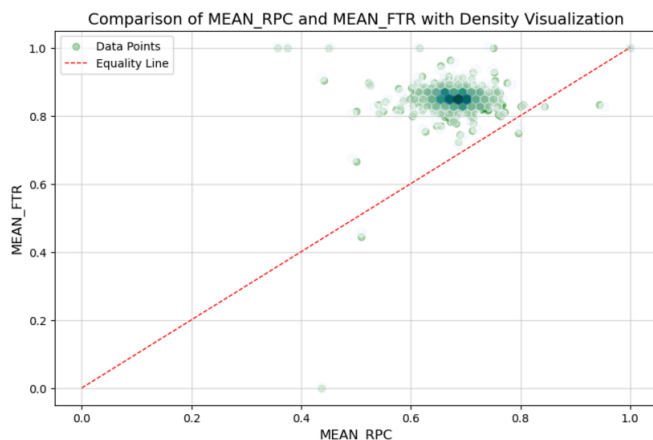


Figure 5. Objective vs self-reported resolution (FTR vs RPC)

Figure 5 plots the objective FTR (MEAN_FTR) on the y-axis against the self-reported resolution (MEAN_RPC) on the x-axis, where each point represents one GC and the red 45-degree line indicates perfect alignment between both measures. Most points lie above this equality line, which means the objective resolution rate is higher than the self-reported one and GCs tend to

underestimate their own performance. The cloud of points is concentrated around relatively high FTR values, while RPC (i.e. Resolução Primeira Chamada: used for self-reported FTR) is more dispersed, and the overall relationship between MEAN_FTR and MEAN_RPC is noisy and not perfectly aligned. This pattern suggests that RPC is not a reliable primary performance measure. Given this systematic gap, the analysis and subsequent models rely on MEAN_FTR as the main resolution metric.

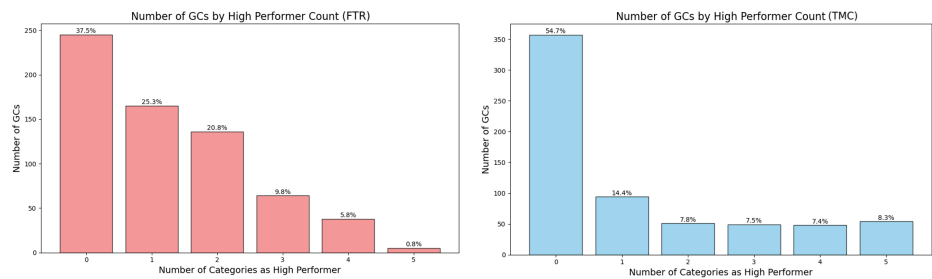


Figure 6. Number of GCs by High Performer Count

Figure 6 shows how many IVR categories each GC excels in. The left panel uses FTR and the right panel uses TMC. For each GC and category, the dataset contains an average FTR and an average TMC. High performers are defined per metric and per category using percentile cut offs: A GC is labelled a high performer in FTR for a category if their FTR lies above the 75th percentile of all GCs in that category since higher resolution rates are better. For TMC a GC is labelled a high performer if their average call duration lies below the 25th percentile, since shorter calls are better. For each GC, the number of categories in which they meet these thresholds is then counted, from zero to five, separately for FTR and TMC. The figure illustrates that high performance is usually concentrated in a small number of categories. In the FTR panel, around 62.5% of GCs are high performers in at least one category, but only a very small share reaches this level in all five. The TMC panel shows a similar pattern, with many GCs not classified as high performers in any

category and only a small group combining very short call durations across all categories. This suggests that most GCs have specific strengths instead of being uniformly excellent across the entire portfolio of topics. From a business perspective, routing and coaching strategies should therefore exploit category specific strengths, assigning GCs preferentially to the categories where they perform best rather than relying on a small group of supposed all-rounders for every complex or high priority call. Currently NOS does not operate with formally specialized GCs, so these patterns are not the result of predefined roles but emerge from the data itself. This makes targeted routing particularly attractive, since it offers a way to systematically use the existing, latent strengths in the workforce without changing contracts or organizational structures.

1.4.2.3 Client Data Analysis

This section examines the Masterdata Clients dataset, which contains client-level demographic and behavioral attributes captured as snapshots over time. The main attributes span demographic characteristics (CLIENT_AGE_YEARS and CLIENT_ANTIQUITY_YEARS), service portfolio size (NR_SAS and SERVICES_QTY), revenue metrics (ARPU_CALCULATED and ARPU_CLIENT), and call-related performance indicators including average call duration, resolution metrics, and incident frequency, all aggregated over rolling windows of 180 and 365 days by topic category. The observation window covers 614,100 unique clients with an average age of 55 years (standard deviation 14.6 years) and average antiquity of 11 years (standard deviation 6.9 years), indicating demographic concentration in middle-aged customers.

Figure 7 presents the distribution of client age and antiquity. Client age follows a somewhat uniform distribution between 40 and 80 years, though the data includes some unrealistic extreme values above 100 years and ages below 18 corresponding to family plans. Negative values indicate missing birth date information in the source system. Client antiquity shows a decreasing frequency

Commented [RK9]: @raquel does it make sense to hint at your chapter here, sth regarding GC specialization will be further explored in chapter...

Commented [RS9R2]: I think it makes more sense in the Bias and fairness part since my section is more focused on training than on specialization.

from 0 to 19 years but exhibits a pronounced spike at exactly 20 years, likely representing a default value applied during a historical system migration.

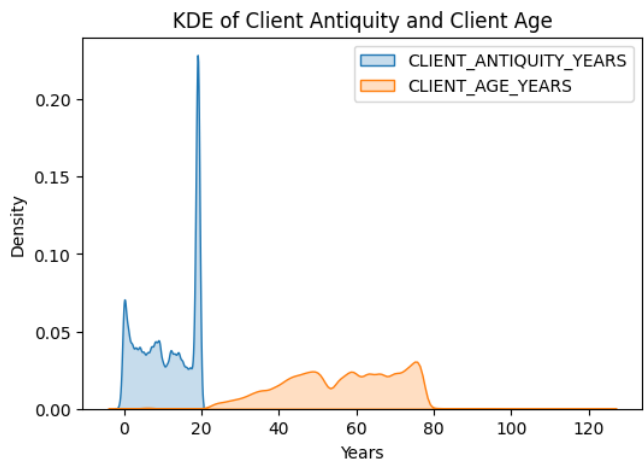


Figure 7. Distribution of Client Age and Antiquity

Furthermore, the majority of clients (92.1%) are consumer-segment customers, with only 7.9% classified as business clients. TV services achieve 100% penetration, while Home WiFi and Fixed Voice services are present in over 95% of accounts, reflecting NOS's bundling strategy. Mobile services show near-universal adoption at over 99%, while satellite services are adopted by only 12.4% compared to approximately 90% for cable and fiber optic connections. Among clients with set-top boxes, 56.4% have upgraded to the UltraHD model. Appendix 1.4 provides a more exhaustive breakdown of the service portfolio composition.

Analysis of KPIs across call categories reveals distinct patterns. Fixed Voice demonstrates the highest resolution efficiency with RPC and FTR rates approaching 0.80 and 0.82 respectively, suggesting straightforward issue resolution. In contrast, Home WiFi and TV categories exhibit lower resolution rates, indicating more complex troubleshooting requirements. Within the core

demographic of 40 to 80 years, KPIs remain remarkably stable, suggesting that client age does not significantly influence call outcomes within the main customer base (see Figure A.X in Appendix). Correlation analysis reveals limited direct relationships between client characteristics and call outcomes. However, expected structural correlations appear within the client data itself: ARPU_CLIENT and NR_SAS show moderate correlation (0.46), while ARPU_CALCULATED and NR_SAS correlate strongly (0.88), with business clients showing higher average revenue per user. The absence of strong correlations between demographics and call outcomes suggests that historical behavioral metric, such as previous call patterns, resolution history by category, and incident frequency, are more relevant predictors than static client attributes, as confirmed by feature importance analysis in Section 1.5.6. These patterns reveal a client base concentrated in middle age with long tenure, high service penetration driven by bundling strategies, and performance metrics that vary substantially by call category rather than demographics. The implications for modeling are clear: client history and call category represent more informative signals than demographic attributes, and any routing strategy must account for the distinct profiles across service types.

1.4.3 Data Cleaning and Imputation

Data cleaning was performed individually on each entity before the final merge, governed by an imputation strategy grounded in the data's semantic meaning. For the GC dataset, missing performance metrics were filled with 0. This choice was based on the business logic that a *NaN* value (e.g., for mean First-Time Resolution in a specific category) indicated "no activity" in that category, making 0 the correct semantic representation rather than an arbitrary imputation. A more granular, two-part rule was applied to the client data to differentiate between computed metrics and descriptive characteristics. Computed performance metrics, such as rolling averages and proportions, were also filled with 0 to represent a valid state of "no activity." Conversely,

Commented [MT10]: What are these columns?

descriptive characteristics (e.g., client tenure or average revenue per user) were imputed with -1. This created a distinct "Missing/Unknown" category, preventing the model from erroneously confusing a missing characteristic with a true, valid value of 0.

Furthermore, to preserve data integrity, a standard forward fill was deemed insufficient for client historical metrics. Instead, a conditional forward fill was implemented: data was only propagated forward if the client had zero calls in the corresponding period. If a client had call activity but a metric was still *NaN*, it was treated as a data quality issue and handled by the standard 0 or -1 imputation, thus preventing the pollution of valid data with stale, forwarded values. After the final merge, one last *dropna()* operation was performed to ensure no missing values remained.

1.4.4 Feature Engineering

To enrich the model's predictive power, features were engineered as described in Table 1.

Table 1. Feature Engineering Rationale

Feature	Description
Agent Experience	A feature for "Agent Active Days" was created as a proxy for operational tenure. The hypothesis is that GCs with more active days may exhibit different performance patterns due to accumulated experience.
Client Issue Concentration	A feature for the "Proportion of Subscribed Services with Issues" was engineered to normalize issue frequency against a client's total services. The hypothesis is that clients with a high proportion of problematic services represent a different behavioral profile.
Imputation Flagging	To ensure the model could distinguish imputed from real data, flags were created for key characteristics (e.g., for missing age or client tenure).

1.4.5 Target Variables

The following target variables were selected because they constitute the direct components of a call's cost function and are the three main KPIs tracked by NOS in their call center operations:

- 1. TMC: The continuous variable representing call duration in seconds. Calls exceeding 10,800 seconds (3 hours) were filtered as outliers based on stakeholder feedback

Commented [MT11]: I don't think we can talk about hypothesis when we never mention any hypothesis we are testing, I changed this up a little

2. FTR: A binary target derived from a pre-aggregated, source-system variable. It is 1 if this variable was greater than 0 (indicating resolution) and 0 otherwise
3. OT: A binary flag from the source data indicating if a technician visit was ordered

1.5 Modeling Approach and Development

The modelling framework works in the real-time decision environment of the NOS call center. Each routing decision follows a strict sequence, starting when a client calls and ending when the system assigns the call to a GC. This section explains the operational context, the data available to the models at decision time, and the steps for training, validating, and comparing the predictive models for GC-call pairing.

Commented [RK12]: do we need this paragraph?

1.5.1 Operational Context and Real-Time Input Pipeline

As illustrated in Figure 8, after a call is initiated, the IVR system prompts the client to specify the nature of the problem. The client's response assigns the contact to one of the five NOS issue categories identified in the Exploratory Data Analysis: IF, VF, TV, MOVEL, or OTHER. This categorization provides the initial structured information for subsequent routing processes.

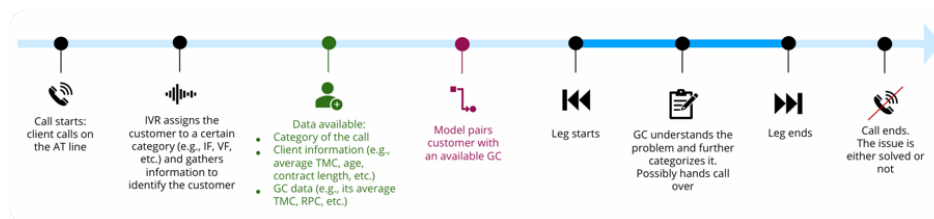


Figure 8. Incoming Call Process

Once the client is identified, either via the incoming phone number or client ID, the system accesses the corresponding client profile. Simultaneously, it retrieves the set of GCs currently available to receive a call. For each agent, the system accesses historical performance indicators, including

average call duration, objective and self-reported resolution rates, technician-dispatch frequencies, category-specific performance measures, and experience-related features. These variables quantitatively represent both general GC attributes and domain-specific strengths.

The IVR classification, client attributes, and GC-level information collectively comprise the complete set of variables observable at the time of a routing decision. This operational context defines the information environment in which a predictive model operates and clarifies the constraints influencing the design of the modelling framework described in subsequent sections.

1.5.2 Preprocessing and Model Setup

The modelling framework summarized in **Error! Reference source not found.** provides an overview of how the prepared data will be used during training. For each feasible GC-call pair, the input features are used to predict the three target variables: TMC, FTR, and OT. These predicted outcomes form the basis of the call-cost function, which in turn enables a ranking of all candidate GCs for a given call. The preprocessing steps described below ensure that the training data accurately reflects the information structure shown in Figure 8, allowing the models to learn the relationships necessary to perform this sequence of predictions and cost-based ranking.

Commented [RK13]: fix this

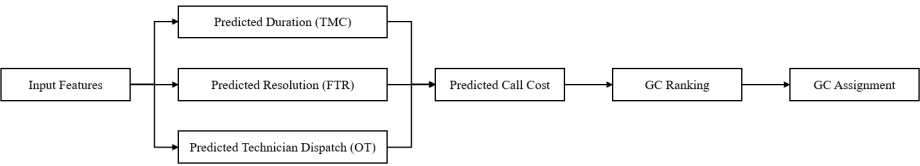


Figure 9. Overview of Model Architecture

The model development followed a systematic pipeline that mirrors standard practices of data preparation, training, and validation. First, we partitioned the data into a training set and a test set to enable unbiased performance evaluation. In our implementation, 20% of the complete dataset

was held out as a test set, with the remaining 80% used for model training. For the classification targets (FTR and OT), stratified sampling was used in this split to ensure the minority class (e.g. instances where a technician dispatch occurred or where a follow-up call was needed) was represented in both training and test sets in proportion to reality. This train/test hold-out approach allowed the models to be tuned on the training data while reserving the test set for final performance evaluation. Before training models, feature selection and preprocessing were performed with an emphasis on real-time deployability of the models. Feature selection was guided by the principle of real-time availability: we included only those predictor variables that would be known at the moment of the call, or at the start of the troubleshooting process. Any information that would be available only after the call's conclusion, such as resolution outcomes, was excluded to prevent look-ahead bias and to ensure the models can operate in live settings. Additionally, aggregate features, such as the average call duration per client or per GC, were provided for different time frames, i.e., covering 180 or 360 days, and we relied on the largest one because it contained the most complete and informative coverage. This resulted in a cleaned and reduced feature set capturing customer attributes, account history, and call context available up-front. Categorical variables were encoded using appropriate schemes, e.g., one-hot encoding for nominal categories, and numerical features were normalized. Specifically, continuous features were scaled using z-score standardization to have mean zero and unit variance.

An important challenge in our data was class imbalance in the two classification targets, as observed in the exploratory analysis described earlier. Training classifiers on imbalanced data can lead to biased predictions (e.g. always predicting the majority class). To address this, Synthetic Minority Over-sampling Technique (SMOTE) was applied on the training data for FTR and OT. SMOTE generates synthetic examples of the minority class to augment the training set, helping the classifier receive a more balanced learning signal (Chawla et al. 2002). After SMOTE

Commented [RK14]: here this is done correctly, in passive form so it should be consistent throughout the thesis

Commented [RK15]: we never write in we-form

Commented [RK16]: this chapter needs to be checked since it is written in we-form and we don't do that anywhere else

Commented [RK17]: maybe refer to EDA chapter here

Commented [MT17R2]: Do we have an EDA chapter?

oversampling, the proportion of positive vs. negative class examples in the training set for these targets was approximately 50/50, from original ratios of about 36% non-FTR (repeat calls) and 18% OT (dispatches) in the raw data. This resampling was done only on the training partition, the test set remained imbalanced to reflect real-world class frequencies for proper evaluation. By combining stratified splitting and SMOTE, the models learn effectively from rare outcomes without being overly optimistic on truly imbalanced test data.

Building on this, different models were trained for each of the three target variables. Although all three targets derive from the same call records, three distinct modeling streams were maintained. This separation was intentional: the three prediction tasks (duration, resolution, dispatch) have different output types (one regression, two classification) and potentially different relevant features or algorithms. Keeping them separate allowed the feature set and model type to be tailored to each specific prediction. Each model was trained on the designated training set, using internal cross-validation to tune hyperparameters when applicable, and the fitted models were then saved. Once trained, each model could generate predictions on new calls independently. This modular training approach aligns with the overall framework: it yields one model predicting call duration (*TMC*), one predicting whether a call is resolved (*FTR*), and one predicting whether the call will need a technician dispatch (*OT*). Conversely, Chapter 3 presents a unified modelling approach that bypasses the fragmented prediction structure entirely.

1.5.3 Model Development

In the multi-faceted nature of the problem, a modeling framework was developed that evaluates multiple families of predictive models for each of the three target variables (Figure 10). The objective was to balance simplicity, interpretability, and predictive performance by exploring a range of approaches, from straightforward rules to advanced machine-learning models.

Commented [RK18]: not sure if we need the full words here or if the abbreviations are enough. just need to be consistent

Commented [RK19]: could be nice to add a model overview table here like we had in the final PBL deliverable in addition to the text

Commented [MT19R2]: You mean like that in slide 37?

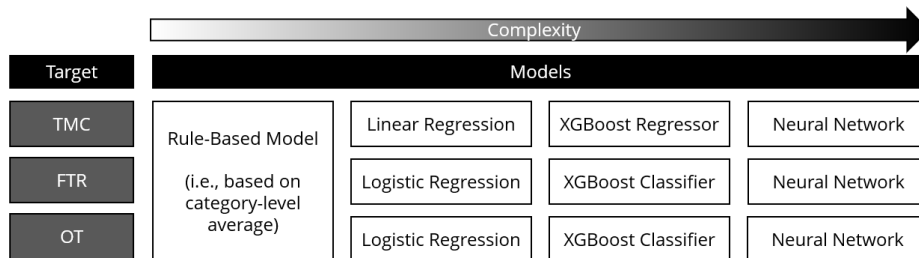


Figure 10. Overview of Modelling Approach

Rule-based baseline models were first implemented. These consisted of simple heuristics such as using the historical average of the target within each call category (e.g., the average duration for a given issue), providing a reference level of expected performance. Because NOS's internal routing rules were not available to the project team, this simplified baseline served as a practical approximation, not to reflect the current operational logic, but to establish a minimal benchmark against which more advanced models could be meaningfully evaluated. Furthermore, Classical linear models were then introduced, linear regression for call duration and logistic regression for the binary targets, to establish interpretable benchmarks and test for linear relationships.

Building on these baselines, a tree ensemble method, XGBoost, was incorporated to capture non-linear patterns and higher-order feature interactions. XGBoost is widely recognized for its efficiency and strong predictive accuracy in structured, tabular data problems (Chen and Guestrin 2016). Random Forest models were initially considered but ultimately excluded due to computational inefficiency, as preliminary tests indicated that training times were substantially higher than XGBoost without delivering improvements in accuracy. Finally, feed-forward multilayer perceptrons Neural Networks (NN) were trained to assess whether a more flexible function approximator could further improve predictive accuracy, particularly for call duration, where complex non-linear effects may be present. Accordingly, three separate NNs were trained,

Commented [RK20]: @Maria Teresa Daffan This sounds like they have a whole set of rules but in reality it is as good as random right? Should we just say here like in the beginning that the rules dont follow any optimization logic so the matching is basically random?

one for each target variable, each using a dense numeric input along with embedded categorical inputs. The merged representation is then passed through one to three fully connected layers, whose width, dropout rate, and learning rate are automatically determined through hyperparameter tuning. This multi-model exploration aligns with best practices in predictive analytics. For example, Moro, Cortez, and Rita (2014) conducted a systematic comparison of logistic regression, decision trees, neural networks, and support-vector machines to predict telemarketing call outcomes, demonstrating the value of evaluating a diverse set of algorithms on the same dataset. Similarly, each model family in our analysis provided distinct advantages, linear models for interpretability, XGBoost for capturing structured non-linearities, and neural networks for modeling complex relationships. Training all models on the same preprocessed dataset ensured a fair and meaningful comparison of their performance.

1.5.4 Evaluation Metrics

Throughout model development, a consistent set of performance metrics was used to evaluate and compare candidate algorithms, with criteria tailored to the nature of each prediction task. For the call-duration regression task (a continuous outcome), the primary metrics were the coefficient of determination (R^2) and the Root Mean Square Error (RMSE) on the test set. R^2 captures the proportion of variance in call duration explained by the model, while RMSE provides an interpretable measure of average error in minutes. Together, these metrics assess both explanatory power and predictive accuracy. For the binary classification tasks, FTR and OT, evaluation relied on standard classification metrics: Precision, Recall, F1-score, and ROC AUC (Area Under the ROC Curve). Precision and recall describe the trade-off between false alarms and missed positive cases. For instance, in the context of FTR, precision measures the share of predicted “resolved” calls that were resolved, whereas recall measures the share of resolved calls the model successfully

identified. The F1-score, the harmonic mean of precision and recall, summarizes this balance in a single number. ROC AUC, measuring the probability that a randomly chosen positive instance is ranked higher than a negative one, provides a threshold-independent indicator of overall discriminative ability. These metrics were computed on the held-out test data to enable fair comparison across model families, including logistic regression, XGBoost, and neural networks. Although these standard metrics were informative, the primary selection criterion for the final models was a bespoke measure aligned with the operational objective: Pairwise Accuracy. Because the system ultimately relies on ranking incoming calls by their expected cost, the key requirement is that the model correctly orders calls according to their true cost risk. Pairwise Accuracy is defined as the proportion of all call pairs for which the model correctly identifies which call is more expensive. Functionally, it serves as a measure of ranking consistency: if call A truly incurs higher cost than call B, the model is considered correct if its predicted cost for A also exceeds that for B. This metric is computed using 100k randomly selected call pairs, with random guessing serving as the 50% accuracy baseline. Prioritizing Pairwise Accuracy ensures that model evaluation reflects the decision context in which predictions are used, namely, the ranking of calls for resource allocation. When trade-offs emerged across metrics (e.g., a model with slightly lower RMSE but substantially higher pairwise accuracy), priority was given to ranking performance. In practice, the XGBoost models for all three targets consistently achieved the highest pairwise accuracy while also delivering strong results on R^2 , AUC, and F1-score. This combination of ranking effectiveness and overall predictive strength guided their selection as the final models. The relative ranking of calls is more important than precise duration estimates or other target-variable estimates.

1.5.5 Cost Construction Procedure

The final architecture fragmented by design: it consists of three independent models, i.e., the best performing for each component of cost. The model outputs are combined to compute the overall expected cost per call. Drawing on the above-mentioned conceptual framework and discussions with NOS industry experts, the cost structure is defined as:

$$Cost\ of\ a\ call = call_{cost} + tech_{cost} + repeat_{cost} \quad (1)$$

Building on this conceptual structure, the corresponding cost function is formalized as follows.

$$Cost\ of\ a\ call = (TMC \times 0.35) + ((1 - FTR) \times TMC_{pop} \times 0.35) + (22 \times P(OT)) \quad (2)$$

These cost parameters were provided directly by NOS. Call-handling costs are computed as the predicted TMC multiplied by €0.35 per minute. The expected technician cost equals the dispatch probability ($P(OT)$) times the fixed €22 fee. The expected repeat-call cost is given by $(1 - FTR)$ multiplied by the duration of the follow-up call and the same €0.35 per-minute rate. However, the duration of a potential follow-up call (TMC_{pop}) is unknown at the moment of the interaction, hence, the client's average call duration is used as a proxy.

1.6 Results and Discussion

Across all three target variables, the rule-based baseline performs substantially worse than any machine-learning model, confirming that data-driven approaches are necessary for NOS.

As shown in Table 2, all ML models provide clear improvements compared to the baseline: Linear Regression and the Neural Network reach R^2 values near 0.10 and pairwise accuracy around 61%, while XGBoost performs best with the highest R^2 (0.12), lowest RMSE (686s), and strongest ranking behavior (62.36%). Although call duration remains inherently difficult to predict, these results show that ML models, particularly XGBoost, offer meaningful gains in accuracy and GC ranking quality.

Commented [PX21]: TMC_pop?

Commented [PX21R2]: x 0.35) - this bracket is not necessary

Commented [PX21R3]: a final bracket is missing in the equation

Commented [PX22]: just say that these values were provided by NOS

Commented [MT23]: Would be nice to have all the results in a table

Commented [RS23R2]: Agree

Table 2. TMC Model Performance Comparison

Prediction	Model	R ²	RMSE (s)	Pairwise Accuracy
TMC	Rule-Based	−0.09	763	41.04%
	Linear Regression	0.11	692	61.86%
	XGBoost Regressor	0.12	686	62.36%
	Neural Network	0.09	697	61.00%

Furthermore, Table 3 shows that all three ML models substantially outperform the rule-based baseline across every metric in the prediction of FTR. Logistic Regression and XGBoost achieve the strongest overall balance, with identical F1 (0.80), precision (0.72), and recall scores and AUC values above 0.70, indicating that both models capture the key patterns driving first-contact resolution. The neural network attains the highest recall, suggesting strong sensitivity to cases that fail to resolve, but low pairwise accuracy (<50%) indicates less stable discrimination across scenarios. Overall, XGBoost performs marginally better in pairwise accuracy, emerging as the most reliable and well-calibrated models for FTR prediction.

Table 3. FTR Model Performance Comparison

Prediction	Model	F1	Precision	Recall	AUC	Pairwise Accuracy
FTR	Rule-Based	0.72	0.64	0.82	0.56	40.68%
	Logistic Regression	0.80	0.72	0.90	0.71	71.56%
	XGBoost Classifier	0.80	0.72	0.90	0.72	71.65%
	Neural Network	0.79	0.67	0.95	0.68	31.68%

A similar pattern appears in the prediction of technician dispatch (Table 4). The rule-based method performs weakly (F1 = 0.25, precision = 0.26, pairwise accuracy = 45%), while Logistic Regression and XGBoost substantially improve discrimination, achieving F1 scores between 0.36–0.38, AUC values around 0.63–0.65, and pairwise accuracy near 65%. The Neural Network again exhibits high recall but very low precision and poor ranking performance, making it unsuitable for this task

Table 4. OT Model Performance Comparison

Prediction	Model	F1	Precision	Recall	AUC	Pairwise Accuracy
OT	Rule-Based	0.25	0.26	0.24	0.62	45.33%
	Logistic Regression	0.36	0.24	0.79	0.64	63.94%
	XGBoost Classifier	0.38	0.25	0.73	0.65	65.18%
	Neural Network	0.34	0.21	0.91	0.63	17.91%

Taken together, the results across all three prediction tasks show that ML-based routing consistently outperforms rule-based decision rules, with XGBoost providing the most accurate, stable, and operationally meaningful improvements. Hence, due to superior performance and efficient scalability, XGBoost was selected as the final model for all three targets: TMC, FTR, and OT.

1.6.1 Feature Importance

Feature importance was assessed using SHAP (SHapley Additive exPlanations) values to determine the factors influencing final model predictions. SHAP tests how much each feature changes the prediction by comparing the model's output with and without that feature, across many combinations, and then averaging those effects. Absolute SHAP values reflect the magnitude of influence irrespective of direction. The corresponding SHAP bar charts illustrate the ranking of the most influential features for each model and target variable.

For the TMC target, the selected XGBoost model is most influenced by variables related to average call duration. The two most important features, avg_duration (average call duration for the topic) and gc_MEAN_TMC (average call duration for the GC), have nearly equivalent influence. The client's average duration over the past year, client_FTR_CALCULATED_MEAN_365_TOTAL, is also highly significant. Category-specific duration averages, such as gc_MEAN_TMC_[topic] for topics like TV or OTHER, are among the top contributing features. Feature-engineered variables, including hour_of_the_day, also contribute substantially, with hour_of_the_day ranking sixth in

Commented [MT24]: We have space left, should we also insert the visuals?? Consider removing subchapters and just adding graphs

importance. In contrast, the neural network and Linear Regression models place greater weight on the proportion of a topic in previous calls (PROP_ variables) and on variables related to OTs and incidents. Nevertheless, mean durations per category consistently appear among the top ten features in these models, indicating that duration history is structurally significant across all models.

For FTR predictions, feature-engineered variables strongly influence results across all models. The most influential variable across all models is the time elapsed since the client's last contact, which is substantially more important than other features. For XGBoost, this variable is `days_since_prev_call_for_client`; for Logistic Regression, it is `client_called_in_last_7_days`; and for the Neural Network, it is `IS_FIRST_CALL`. Each of these engineered features captures recency or first call effects. The second-most important variable across all models is `client_FTR_CALCULATED_MEAN_365_TOTAL`, which captures the client's long-term behavior over the past year. The remaining top features are primarily PREV_ or PROP_ engineered variables, which describe either the topic of the previous call or the proportion of a topic in historical calls. These variables effectively summarize customer history and strongly influence the probability of first-time resolution.

For the OT target, both XGBoost and Logistic Regression models are primarily influenced by `avg_duration`. `Hour_of_the_day` is also a significant variable across all models. In XGBoost, client portfolio characteristics, including the number of services and customer antiquity in years, are among the most influential features. Further details on the SHAP analysis for XGBoost and the other models are provided in Appendices 1.5–1.8.

The IVR input variables, which could be seen as valuable pre-call information, does not have a high feature importance for any of the models. This leaves NOS with room for improvement in their IVR system which will be explored in more detail in Chapter 2.

Commented [RK25]: I added this sentence here to hint at my sub-chapter

1.7 Plan for Testing

1.7.1 Testing Objectives and Setup

The primary objective of a testing plan for the predictive pairing system is to establish whether the model-based routing approach delivers measurable improvements in operational cost compared to the existing random allocation process. Although this thesis does not include deployment at NOS, this section outlines a theoretically sound testing framework that could be adopted for validating the model in a real-world environment. Such a framework must address three core requirements: it should confirm that the model generalizes well beyond the training period, it should quantify the operational impact in terms of cost reduction, and it should identify any degradation in performance over time that would necessitate intervention or retraining.

From a methodological perspective, testing must account for the fact that predictive models for operational systems face distinct challenges compared to static prediction tasks. The call center environment is dynamic: customer behavior evolves, new service offerings are introduced, GC skill profiles shift as staff turnover occurs, and seasonal patterns recur. All of these factors can influence the relationship between input features and outcomes, meaning that a model trained on historical data may not maintain its performance indefinitely. Consequently, a comprehensive testing approach requires both offline validation to assess baseline generalization and online evaluation strategies to measure real-world performance under deployment conditions.

1.7.2 Offline and Online Testing Approaches

Offline testing refers to evaluation conducted on historical data before any live deployment. While the standard train-test split provides an initial benchmark, a more rigorous offline validation strategy would employ temporal holdout testing, also known as walk-forward validation. Rather than randomly sampling the test set, temporal holdout respects the chronological ordering of the

data: the model is trained on observations from a historical window and tested on subsequent future observations (Cerqueira et al. 2019). This mimics the production setting where the model will predict "tomorrow" based on patterns learned from "yesterday."

In addition to standard accuracy metrics such as RMSE for call duration and AUC for binary targets, offline validation should incorporate ranking-based evaluation metrics. Since the ultimate goal is to rank GCs by expected cost, the model's ability to correctly order GCs is more critical than the absolute accuracy of individual predictions. Pairwise accuracy measures the proportion of GC pairs for which the model correctly identifies which GC is less expensive. Metrics from the information retrieval literature such as Normalized Discounted Cumulative Gain (NDCG) can also be applied, evaluating how well the model orders the top-k GCs (Evidently AI 2025).

Online testing evaluates the model's performance in a live operational setting, typically through controlled experiments such as A/B tests. In a deployment scenario at NOS, the treatment group would use the predictive pairing algorithm while the control group would maintain the existing rule-based routing system. However, standard A/B testing introduces significant challenges due to interference bias: GCs are a shared resource, so if a treatment group gains access to "optimal" GCs via smart pairing, those GCs become unavailable to the control group, artificially degrading its performance. This violates the Stable Unit Treatment Value Assumption (SUTVA), a core requirement of A/B testing (Ni et al. 2025). Alternative experimental designs can mitigate interference bias. Switchback testing alternates the entire call center between the treatment algorithm and the control algorithm over time rather than splitting calls into separate groups (Bojinov 2025). This eliminates interference at any given time point but requires attention to carryover effects. Another robust alternative is Difference-in-Differences (DiD) analysis, which compares changes in outcomes over time between treatment and control groups, accounting for pre-existing differences (Tchetgen et al. 2023).

1.7.3 KPI Definition and Evaluation Strategy

Defining clear Key Performance Indicators (KPIs) is critical for assessing deployment success. The primary KPI is total cost reduction, measured as the average cost per call under predictive pairing compared to random allocation as the scientific baseline. While NOS currently employs a rule-based routing system, random allocation provides a conservative lower bound for performance evaluation and enables unbiased comparison across different modeling approaches. This can be decomposed into the three cost components identified in equation (1). Monitoring each component separately allows for diagnostic insights into performance drivers. Additionally, comparisons to rule-based heuristics such as greedy cost-minimization policies provide further context for assessing the value of machine learning approaches.

Secondary KPIs include individual model performance metrics: RMSE for TMC predictions, AUC and F1-score for FTR and OT classifications, and pairwise accuracy for cost ranking. These metrics serve as early warning indicators of model degradation. If RMSE or AUC begins to decline on incoming data, it suggests that predictions are becoming less reliable even if cost has not yet visibly increased. Given the stochastic nature of call arrivals, hypothesis testing should be employed to determine whether observed cost reductions are statistically significant. Sample size calculations should be performed prior to testing to ensure sufficient statistical power to detect meaningful cost reductions with acceptable confidence and power levels.

1.7.4 Retraining and Monitoring Plan

Machine learning models deployed in operational environments are subject to performance degradation over time due to concept drift, which refers to changes in the data distribution or underlying relationships between features and outcomes (Bayram et al. 2022). In call center operations, this could manifest through shifts in customer behavior, GC performance profiles, or

technical infrastructure updates. To address concept drift, continuous monitoring operates on two levels: data-level monitoring tracks whether the distribution of input features is shifting using metrics such as Population Stability Index or Kullback-Leibler divergence, while performance-level monitoring tracks the model's actual prediction quality on incoming data through rolling-window calculations of RMSE, AUC, or F1-score (Bayram et al. 2022; Mallick et al. 2022).

When monitoring detects significant drift or performance degradation, the model should be retrained. Retraining strategies fall into two broad categories: scheduled retraining and triggered retraining (Mallick et al. 2022; Shankar and Parameswaran 2022). A hybrid approach is often optimal: baseline scheduled retraining, conducted at fixed intervals such as quarterly, ensures the model is refreshed periodically to capture gradual trends, while triggered retraining provides rapid response to abrupt shifts detected by monitoring systems. For the tree-based models (XGBoost) used in this thesis, full retraining on the combined dataset is generally more straightforward and reliable than incremental updates (Mallick et al. 2022). The retraining process should incorporate model validation and rollback mechanisms. After retraining, the updated model should be evaluated on a recent holdout set to confirm improved performance before deployment. If the new model performs worse, the system should automatically revert to the prior version and trigger an alert for manual investigation.

1.8 Limitations and Further Research

The modelling framework developed in this thesis faces several limitations linked to data availability, operational assumptions and deployment constraints. The dataset covers only ten months of activity, which restricts the model's ability to capture long-term behavior, such as annual seasonality, recurring peaks or unusual periods caused by holidays, product launches or large service incidents. Any future shift in call-center operations, including new products or changing

issue types, would introduce patterns not represented in the training data and would require model retraining. NOS's anonymization protocols also limited access to potentially informative variables, creating an unavoidable trade-off between privacy compliance and modelling richness. Several behavioral or demographic indicators, such as customer gender, detailed location or household characteristics, were unavailable, reducing the ability to capture finer patterns. Data completeness presented an additional challenge, since merging call, client and GC tables required discarding records with missing or unmatched identifiers. Many GC IDs in the call data were not present in the GC master file, and some clients existed without corresponding call entries, which reduced the size of the final dataset. Deployment constraints create further uncertainty, as the model must remain compatible with NOS's automatic call-distribution systems, which may require simplifications to ensure real-time operation. In addition, several routing rules and business constraints were not shared with the project team, meaning that certain operational requirements could not be reflected in the modelling pipeline.

Future work could expand the temporal scope of the data to incorporate multi-year trends, allowing the model to learn seasonality, customer lifecycle effects and evolving service contexts. Other research avenues include predicting the duration of the next call when issues remain unresolved, using sequence-aware models, or adopting multi-task learning frameworks that jointly estimate duration, escalation, churn risk and resolution probability. Further extensions could address fairness and bias auditing, integrate richer data sources such as device diagnostics or voice analytics, and explore real-time learning systems that update routing logic as behavior and conditions evolve. Together, these directions show that while the current model offers a strong basis for cost-informed call allocation, long-term value depends on monitoring, retraining and methodological innovation.

1.8.1 Bias and Fairness

Using predictive models for call routing can improve efficiency but also raises questions of bias and fairness, particularly in how GCs are evaluated and how work is distributed. Automated routing rules can create systematic advantages and disadvantages for certain GCs. Bias at the data generation stage is limited because calls were originally assigned at random, resulting in broadly even call volumes across GCs. However, two key fairness risks emerge once the data is used to drive model-based routing. First, performance estimates for GCs with limited historical data, such as new hires, are highly unstable. A GC who initially receives disproportionately complex calls may appear to underperform, while another exposed to easier cases may seem highly competent. These early distortions can mislead predictive models and skew subsequent routing decisions. Mitigation strategies include treating low-volume GCs separately, adjusting for case complexity, and updating performance estimates dynamically as more data becomes available.

Second, once deployed, the routing system can amplify initial performance differences. If calls are preferentially directed to high-performing GCs, they receive more practice and visibility, while lower-performing or newer GCs face reduced opportunities to learn. Over time, this dynamic can turn early randomness or onboarding effects into persistent disparities. Safeguards such as exposure quotas, call rotation limits, and controlled randomness in assignment can help counteract this. From a governance standpoint, fair routing also requires transparency about metric use and collaboration with NOS's business partners on workforce management policies.

While the initial random allocation offers a fair starting point, algorithmic routing introduces structural feedback loops that may entrench inequality without corrective mechanisms. Mitigating these effects requires not only careful routing design but also complementary systems that monitor GC trajectories and support development. This challenge motivates the personalized training

Commented [RS26]: I added the hint to my part here

framework proposed in Section 4, which aims to surface individualized skill gaps and counterbalance opportunity imbalances over time.

1.9 Main Insights and Implications for NOS

Implementing the predictive pairing model would have several direct and strategic implications for NOS. Most immediately, the system enables tangible cost reductions by lowering call durations, improving first-contact resolution, and reducing unnecessary technician visits. Given the volume of calls handled monthly, even small gains in these metrics can translate into significant operational savings and a measurable improvement in customer-service efficiency. Chapter 5 presents a Beyond cost benefits, the model would allow NOS to move from a rule-based allocation system to a data-driven decision framework, improving how calls are matched to the most suitable GCs. This can enhance service quality and streamline workloads across teams, while also providing clearer insights into GC performance and training needs. However, it also requires careful alignment with existing contracts and incentive structures, as routing decisions influence partner-company KPIs and compensation.

From an operational standpoint, deploying the model requires stable data pipelines that integrate with the current system, real-time feature availability, and scheduled retraining to maintain accuracy. As new products, service issues, or call patterns emerge, NOS would need to update the model to avoid performance degradation. This represents an opportunity to strengthen the company's analytics and AI capabilities but also necessitates IT investment and ongoing monitoring. Finally, adopting this system positions NOS as a leader in AI-driven call-center optimization, opening the door to broader applications such as adaptive call routing, personalized service strategies, and cross-channel customer-journey optimization. In this sense, the model is not only a cost-reduction tool but also a strategic enabler for innovation in customer operations.

Commented [MT27]: Should probably do like a simulation to see how much costs were reduced or refer to Marvin's part and say that he did it (probably better)

Commented [RK27R2]: agree, a link to Marvin's part would make sense here

1.8 Conclusion

This chapter developed a predictive framework to improve how NOS assigns incoming calls to its GCs. Using historical data on clients, calls and GC performance, the PBL project transformed a rule-based operational task into a structured modelling problem supported by a cost-based decision rule. The three target variables capture the main drivers of call-center performance: TMC reflects call duration and direct handling cost, FTR indicates whether issues are resolved on first contact, and OT identifies cases requiring technician visits. These outcomes summarize the balance between efficiency and service quality, which makes the project particularly relevant for NOS.

After extensive preparation and feature engineering, several models were tested for each target, with XGBoost emerging as the most effective. The resulting predictions were combined into a single cost estimate through an industry-informed formula, producing an interpretable ranking of routing choices for each incoming call. The chapter also outlined how this decision rule can be compared with current routing and how performance can be monitored over time.

From a business perspective, the findings show that predictive routing is a concrete and achievable step toward more data-informed customer service at NOS. The framework relies on data the company already collects, links model outputs directly to cost outcomes and supports gradual adoption through offline testing, pilot studies and later rollout. At the same time, predictive routing is only a starting point. It creates a foundation for further advances, including richer information at the beginning of each call, alternative modelling strategies, improved GC development and more dynamic decision rules. The following chapters build on these themes and explore additional opportunities for strengthening NOS's customer service.

2. Automating call intake with LLM-augmented conversational IVR - Runa Kleppek

2.1 Current IVR System at NOS

NOS currently uses a two-stage IVR system to capture customer call intent. In the first stage, the system differentiates between technical and non-technical support. In the second stage, customers are prompted to state the topic of their call, which is processed through voice recognition and mapped to one of five categories: Television, Broadband, Mobile, Landline, or Other. If the customer's spoken input does not clearly match a predefined keyword, it is automatically assigned to the “Other” category. As a result, **Error! Reference source not found.** shows that the “Other” group makes up a disproportionately high share of total calls, despite often including issues that could be more precisely classified.

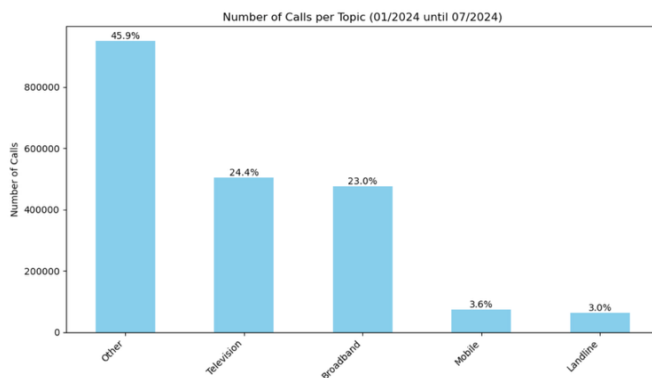


Figure 11. Number of Calls per Topic/Category

These high-level intent categories are used as input variables in the predictive models that support GC assignment (see Chapter 1.5). In the current system, only the recognized intent label and a small set of customer attributes, such as age or tenure, are available to the model and the GC before the call begins. However, analysis has shown that these inputs are of limited predictive value for key

Commented [PX29]: The Figure on the left is too small. It is not possible to clearly see the information provided. Do not forget to include a legend: "Figure 2 - ..."

operational outcomes, including FTR, TMC, and OT. In contrast, a more detailed “level 2” classification, assigned after the call ends, provides significantly better model performance. Because this more granular label is unavailable during the call routing process, it cannot contribute to real-time decision-making.

This limitation reflects a broader operational challenge in contact centers: inadequate information at the start of customer interaction reduces the effectiveness of both predictive models and human service. Studies show that richer pre-call data can significantly improve GC matching, increase resolution rates, and reduce downstream costs such as unnecessary OT dispatches (Behavioral Signals 2025). When customers are matched with the right GC using more precise contextual information, resolution times improve and call center efficiency increases (Gudipati 2025). In the NOS context, the current IVR therefore not only constrains model performance, as seen in chapter 1.5.6 that shows that the IVR variables have a very low feature importance for the matching models, but also leaves human GCs with limited situational awareness when they pick up the call.

2.2 Problem Statement and Research Objectives

Research indicates that providing GCs with richer pre-call information can increase operational efficiency and improve first-call resolution rates (Kelly 2025). At NOS, the current IVR gathers only very limited information before a call is handed over to a GC. This results in fewer input features for the matching models, which contributes to lower predictive accuracy, and leaves the human GC with minimal context when the conversation begins. The gap between the coarse pre-call intent labels and the more granular “level 2” classifications available only after the call creates an information bottleneck at the moment routing decisions are made. This also drives avoidable operational costs. The objective of this thesis is to design and test a prototype for an LLM-augmented IVR system that automates the initial portion of a customer service call. The prototype

aims to replace the static menu with a short, conversational intake handled by a generative Artificial Intelligence (GenAI) based “GC” that interacts with the customer in natural language. During this intake phase, the AI-GC collects structured, context-specific information such as customer identifiers, service type, problem description, and relevant technical details before the call is handed over to a human agent.

By leveraging a genAI model in a conversational IVR setting, the system is designed to gather as much relevant information as possible upfront, while keeping the interaction concise and easy to follow. The enriched information is then available both to the predictive matching algorithm and to the GC who receives the call, with the dual aim of improving routing quality and enabling faster problem-solving once the human agent joins the conversation. This chapter primarily quantifies the potential operational cost savings from automating the initial intake phase beyond just using the standard IVR, explicitly taking into account the cost of running the LLM-based system itself. In addition, it outlines how the enriched pre-call information generated by the prototype could, in principle, be fed into the cost-based GC matching framework developed in Chapter 1 as additional real-time features.

2.3 Literature Review

2.3.1 IVR Systems: Current State and Limitations

IVR systems have long been used to reduce service costs and automate basic routing. Most traditional IVRs rely on touch-tone menus or simple keyword recognition and follow static menu trees, for example “Press 1 for technical issues”. While still widely deployed, these systems often frustrate customers who find them rigid or confusing. Studies show that many businesses report IVR containment rates below 30%, meaning most calls are not resolved through automation and must be handled by a live agent (Teneo.ai 2023). Despite these limitations, the IVR market

Commented [NS31]: cost saving analysis requires the cost of running the LLM

Commented [NS32]: I would do it as 2.3.1.

continues to grow. A recent forecast estimates the global market will increase from around USD 6.4 billion in 2025 to nearly USD 11 billion by 2032 (Research and Markets 2025). This growth reflects ongoing investments in modernizing IVR systems, especially through conversational AI. Next-generation IVRs use speech recognition and natural language understanding to allow callers to speak naturally instead of navigating menus. Companies adopting these technologies have reported improved satisfaction and reduced agent transfers, with some showing fivefold increases in customer satisfaction scores (Agarwal, Kline, and Raabe 2023).

The contrast between static and AI-enhanced IVRs keeps growing. As customer expectations shift toward seamless, human-like interactions, older IVRs are increasingly seen as outdated. This creates a strong incentive for organizations like NOS to explore AI-driven alternatives that can offer more flexibility and better data collection during the early phases of a call.

2.3.2 LLMs in Customer Service and Conversational AI

The integration of LLMs into customer service operations represents a major shift in how companies manage interactions. LLMs are capable of understanding context, generating fluent replies, and handling tasks such as summarizing conversations, retrieving customer data, and answering questions. Industry surveys suggest growing adoption: 63% of executives expect to use genAI to support customer service agents by 2026 (Behavioral Signals 2025), and 94% of contact center leaders already report using AI tools to assist agents during live calls (Five9 2023). LLMs are being deployed in multiple ways. Some act as background assistants that summarize customer conversations, pre-fill case notes, or recommend next steps to human agents. Others function as virtual agents in chat or voice channels that interact directly with customers. These applications are particularly relevant in telecom environments, where many support calls follow predictable patterns and benefit from quick, structured data gathering. By taking over the repetitive, early parts

of a call, LLMs can collect key information that would otherwise have to be asked by a human GC, thereby freeing human agents to focus on complex diagnosis and resolution. LLMs are technically capable of handling many routine interactions end-to-end. However, research and industry practice show that they work best when they augment human support rather than replace it entirely (Agarwal, Kline, and Raabe 2023). In such hybrid models, AI handles simple inquiries or intake steps, while humans resolve more nuanced or sensitive issues. For a company like NOS, this setup aligns closely with the problem identified in Section 2.1 and the objective formulated in Section 2.2: an LLM-based assistant can gather richer pre-call information through natural dialogue, while final decision-making and problem resolution remain with human agents. In this context, LLM-augmented IVRs offer the potential to improve routing, reduce handling times, and support agents more effectively without sacrificing service quality (Gudipati 2025). The prototype developed in this thesis adopts exactly this hybrid approach, using an LLM as a conversational intake layer that enhances the information available at the moment of call assignment rather than replacing the GC entirely.

2.3.3 Opportunities and Risks of LLM-Augmented IVRs

Integrating LLMs into IVRs creates opportunities to gather better data before a human agent gets involved. When information like device type, location, or prior issue history is collected upfront, routing becomes more accurate and service faster. Companies using conversational IVRs have reported up to 30% reductions in average handling time (Sprinklr 2025). LLMs also make the interaction more natural. Callers can explain their problem in their own words, and the system can interpret and respond flexibly. One telecom study found that an LLM-based IVR could correctly classify 89% of inquiries (Rojas-Galeano 2025).

However, LLMs come with challenges. They can generate false or misleading responses, also called hallucinations, which risk undermining trust (Teneo.ai 2023b). They also rely on fast processing, and any delay in speech-to-text (STT), LLM generation, or text-to-speech (TTS) can create lags that degrade the user experience (Menon et al. 2025). Security and compliance are additional concerns, especially when customer data passes through external AI systems. Attacks like prompt injection, where users manipulate the AI's behavior with special inputs, are another emerging risk (Teneo.ai 2023b).

Customer preferences must also be considered. A Gartner survey found that 64% of customers prefer not to interact with AI for service (Gartner 2024). Systems without a clear path to a human agent may frustrate users. For this reason, best practices suggest that AI-IVR designs always include a handoff option and make it easy for customers to switch to a person.

2.4 Data and Methodology

2.4.1 Research Goal and Approach

Building on the research objective defined in Section 2.2, this section details the design and testing of the prototype that automates the initial part of a customer service call, including the IVR interaction and the first intake questions typically asked by a GC. Instead of a static IVR menu, an LLM-based “AI-GC” answers the call, conducts a short conversation with the customer, and gathers useful information for routing and problem solving before handing over to a human GC.

Figure 12 illustrates the high-level process flow. On the left, the current process shows the IVR separating technical and general calls, detecting the call category, and then handing the call to a GC with limited context. On the right, the prototype replaces this with a conversational intake process in which the AI-GC collects structured information and forwards the call to a human GC including a complete handover summary. The handover happens when all information is collected

or if an early handover is triggered. The research approach is therefore twofold: first, to implement a functional prototype that reflects this envisioned process; and second, to test it in controlled

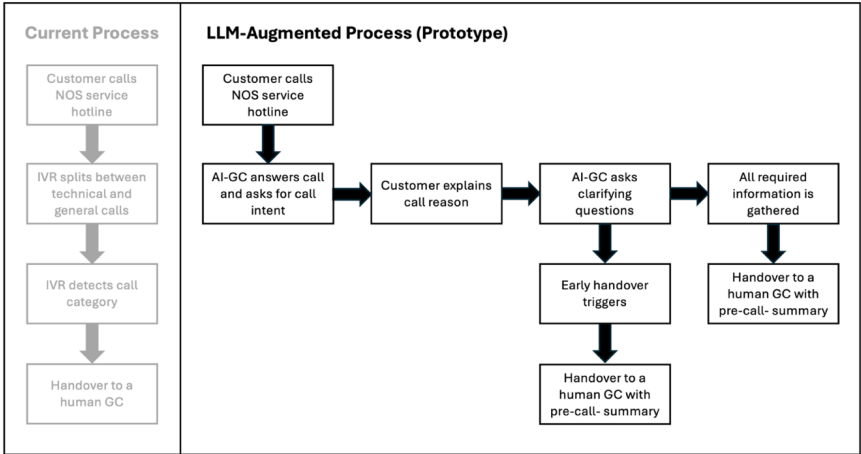


Figure 12. Comparison of current and LLM-augmented call intake process (Prototype) scenarios in order to estimate its impact on information quality and operational costs.

2.4.2 Prototype Design

The prototype is implemented as a local web application using Gradio, an open-source Python framework for interactive AI systems (Gradio. 2025). Gradio provides a browser-based interface that combines real-time microphone input, STT transcription, LLM responses, and TTS playback within a single environment. Figure 18 shows an example interaction in the NOS Voice IVR prototype.

Once the application is launched, the AI-GC greets the customer and collects their responses in a short back-and-forth dialogue. Everything the customer says is recorded, transcribed via STT, processed by the LLM, and converted back to speech. This loop continues until the required information fields are filled, after which the AI-GC announces handover to a human GC. The conversational engine of the prototype uses GPT-4o via Azure OpenAI. GPT-4o combines strong

Commented [NS33]: picture of the main steps of the prototype, so it helps following the text

language understanding with relatively low latency and cost compared to other GPT models. It is priced per input and output token. STT converts recorded customer audio into text and is billed per processed audio minute. TTS converts the model's replies into audio using Azure's "en-US-EmmaNeural" voice, chosen for its professional tone, and is billed per generated character. All usage-based rates are included in the later cost calculations (Azure AI Speech pricing, 2025). The prototype is a proof of concept and therefore comes with a few limitations. The tests are conducted in English language, while deployment at NOS would require a Portuguese system. The Gradio interface supports single-call sessions on a local setup whereas a real call center deployment would need scalable infrastructure for thousands of concurrent calls. Moreover, the prototype is not yet integrated with CRM or ticketing systems, which would be necessary to write collected data directly into customer profiles and display it in the GC interface.

Commented [NS34]: relative to what?

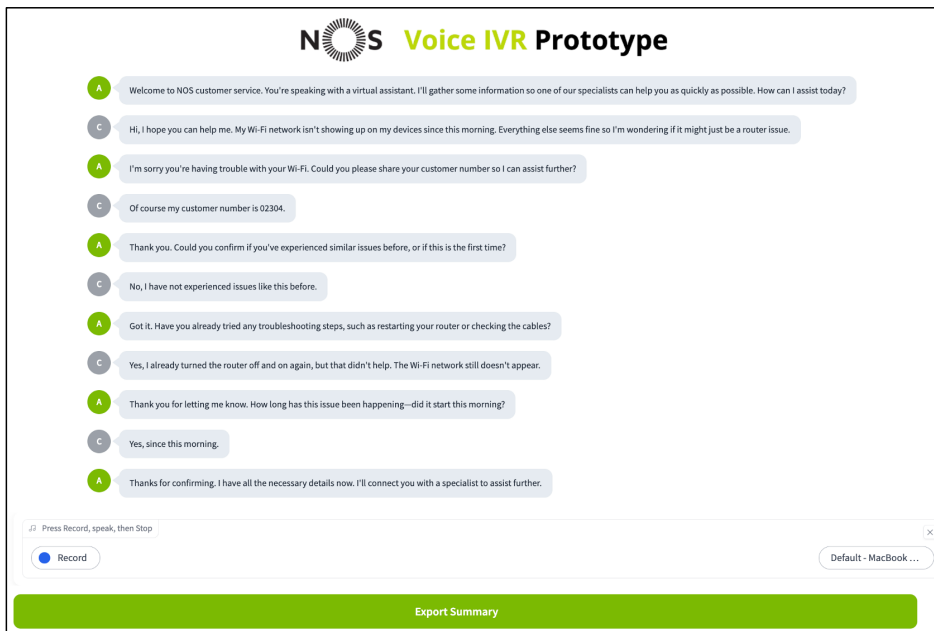


Figure 13. Example call intake dialogue in the NOS Voice IVR prototype

2.4.3. Conversational Model

2.4.3.1. Information Fields

During the intake phase, the AI-GC is tasked with filling a set of structured fields that are later passed on to the human GC as a compact handover package. These fields cover basic identifiers, the call intent, category and a summary. In addition, they include softer signals such as sentiment and perceived urgency, which are helpful both for routing and for the GC's preparation. An overview of all fields collected by the prototype is provided in Table 1.

Table 5. Overview of Variables collected by the Prototype

variable_name	description	Relevant for
customer_number	The customer's identification or account number	all
service_type	Television, Broadband, Mobile, Landline or Other	all
problem_description	Short description of the intent in 1–2 sentences	all
problem_duration	How long the issue has persisted	technical
device_type	The device involved (e.g., router, SIM card, TV box)	technical
previous_issue	Whether the customer experienced the issue before	technical
troubleshooting_done	Whether the customer already tried something to fix it	technical
sentiment_score	Detected emotional tone of the customer	all
intent_category	The AI-classified category of the call	all
summary_short	A one-sentence summary of the issue	all
summary_long	Concise overview including all provided information	all

Several of these fields are also relevant from the perspective of the cost framework developed in Chapter 1. For example, `problem_duration`, `previous_issue` and `troubleshooting_done` are likely to correlate with call duration (TMC), the probability of a repeat contact ($1 - \text{FTR}$), and the likelihood of a technician dispatch (OT). While this thesis does not retrain the matching model using these variables, it shows that an LLM-based intake can provide valuable information available at the start of the call, that could be used to improve the call routing model described in Chapter 1.

The distinction between Technical and General calls is very important because these calls will be treated differently. Technical calls, for example when a customer reports Wi-Fi problems, have high

automation potential because they follow standard troubleshooting patterns. For these calls, the AI-GC aims to collect detailed technical information such as the affected service, the device involved and how long the problem has persisted. This richer intake information allows a GC to skip basic questions and move directly towards diagnosis and solution after the handover.

General calls, such as billing questions are usually shorter and less structured. For these calls the system still collects a customer number and the main intent but it does not insist on technical details like device type or problem duration.

2.4.2.2. Prompt Design

The AI-GC's behavior is governed by a system prompt that defines its role, tone and objectives. It is instructed to act as a NOS customer service assistant, speak in a warm, concise and professional way, and collect missing information step by step. For technical calls, the prompt specifies that the intake should only be concluded once all technical fields are filled, unless the customer becomes clearly frustrated. Sentiment is also handled via the system prompt. After each turn, a separate extraction step asks the model to label the customer's emotional tone as positive, neutral or negative. This sentiment class is written into the structured fields and used in the conversation logic: if sentiment is neutral or positive, the AI-GC continues with short follow-up questions until all required fields are complete; if sentiment remains negative across several turns, the system triggers a handover to a human GC.

The prompt further distinguishes between technical and non-technical situations when deciding whether an apology is appropriate. In clear service failures such as connectivity problems, the assistant is expected to open with a brief apology before proceeding with clarifying questions.

The prompt was iteratively refined from rigid, option-based phrasing to more natural, open questions while preserving structured data capture. Tests with sample dialogues were used to reduce

Commented [NS35]: separate LLM? or the same with the same system prompt?

Commented [NS36]: is it a score or a class?

repetition, avoid overly formal language and ensure that all fields were collected. This underlines how sensitive LLM-based systems are to prompt wording and how careful design is needed to balance guidance with natural conversation.

2.4.2.3. Human-in-the-Loop

In the prototype, the human-in-the-loop (HITL) setup means that the AI-GC handles the initial intake, but a human GC takes over once the AI reaches predefined limits. These rules are encoded in the prompt. When a handover is triggered, the system stops the AI-GC and forwards a compact handover package with the structured variables and a summary, so the human GC can continue smoothly. For the customer, the AI-GC closes with a clear handover phrase. The prompt defines the following handover triggers:

- All required information collected: All fields in the variable table are filled, representing the typical case. The call is then handed over for problem-solving, with the handover package allowing the GC to proceed without reviewing the full transcript.
- Call duration too long: If the intake exceeds eight dialogue turns, the system assumes the process is taking too long and hands over to a human GC to avoid frustration. The threshold of eight turns was chosen to balance collecting sufficient information with keeping the intake phase short.
- Negative sentiment: Sentiment is inferred from the customer's language. If it remains negative for at least two turns or the customer explicitly expresses frustration, the AI-GC stops and escalates to a human GC.
- Unclear intent: If the AI-GC cannot reliably classify the issue, for example because the intent remains vague or mixes technical and billing indicators, it stops asking clarifying questions and hands the call over.

- Repeated STT failures: If the system receives silence or unintelligible audio twice in a row, despite asking the customer to repeat once, the call is forwarded to a human GC.
- Safety or compliance issues: Mentions of emergencies, threats, self-harm, sensitive payment data or other critical situations trigger an immediate handover.

2.4.4 Prototype Testing

The prototype testing was carried out by simulating 25 fictional calls through the Gradio interface. A human tester acted as the customer, while the AI-GC followed the previously designed prompt to collect the information needed for handover. The scenarios were based on real examples from the mastercalls dataset and enriched with additional details to ensure realistic testing conditions. They covered both technical **calls**, such as internet interruptions, mobile data problems and television errors, and general service requests such as billing updates, subscription changes and cancellations. Because the prototype uses local audio handling instead of end-to-end streaming, only active talk time was used when calculating durations. All other variables, including token usage, STT minutes, TTS characters and the number of dialogue turns, were recorded for the purpose of cost estimation and performance evaluation.

Commented [PX37]: why in bold?

2.4.4.1 Metrics

Each simulated call generates a detailed set of conversational and technical usage metrics that serve as the basis for evaluating both performance and cost. Conversational metrics include the time the AI-GC spent speaking, the customer's speaking time, the number of turns until the handover point and the proportion of required fields successfully collected for routing. Usage metrics cover the number of characters processed by STT and TTS, as well as all input and output tokens used by GPT-4o. Because Azure OpenAI and Azure Speech are billed per token, per character and per

audio minute (Microsoft Azure, 2025), these variables allow for an accurate calculation of the technical cost generated by each interaction.

To estimate operational savings, the avoided human GC time is compared with these technical costs. The cost per call is calculated using the human GC rate of 0.35 EUR per minute provided by NOS and the Azure pricing values defined in the assumptions. The resulting equation is shown below. Here, t_{min} denotes avoided human GC minutes per call, stt_{min} the minutes processed by STT, tss_{chars} the number of characters synthesized in TTS, $tokens_{input}$ the number of input tokens sent to GPT-4o, and $tokens_{output}$ the number of output tokens generated.

$$\begin{aligned} \text{Cost Savings} = & t_{min} * 0.35\text{€} - stt_{min} * 0.0143 \text{ €} - tss_{chars} * 0.000012861\text{€} \\ & - tokens_{input} * \text{€ } 0.00000214335\text{€} - tokens_{output} * \text{€ } 0.0000085734\text{€} \end{aligned}$$

In the data preprocessing a small latency factor of 0.4 seconds is added to approximate the short delay expected between a user's speech and the AI's response in a streamed production environment, without allowing prototype-specific processing delays to dominate the timing results.

2.4.4.2. Data Analysis and Results

After each scenario, the system generated two outputs. A structured JSON file documented all fields the AI-GC collected during the conversation, including customer identifiers, intent category, summaries and, in technical cases, diagnostic elements such as device type and problem duration. A separate Excel file contained the associated performance metrics, which enabled further quantitative analysis. These outputs were combined to assess the average time saved before handover to a human agent. The Azure usage metrics were then applied to the cost equation to calculate the net savings per call and the projected monthly effect.

The prototype completed the information-gathering phase reliably across the tested scenarios. In all 25 calls, the information written to the JSON files was correct. Quality issues arose when some

questions were not asked rather than from incorrect entries. Completion rates of the required fields were 82% for general service calls and 88% for technical calls. Technical calls naturally required more clarification steps and therefore involved more dialogue turns. At the same time, they offered higher automation potential because they follow structured troubleshooting patterns that the AI can collect consistently. This resulted in greater time savings. General service calls tended to be shorter and contained fewer actionable fields. When the measured time savings, which are based on the 25-call prototype experiment described in Section 2.4.4, are combined with Azure’s technical usage costs, the system achieves an estimated monthly operational saving of 73,930 EUR.

Cost Savings Calculation			
	General		Technical
Avg. GC Minutes Saved per Call		0.22	0.31
Avg. AI Cost per Call	€	0.02	€ 0.03
Avg. Cost Savings per call	€	0.20	€ 0.28
Monthly Calls		120,000	180,000
Total monthly Cost Savings	€	24,264	€ 49,666
			€ 73,930

Figure 14. Cost Savings Calculation

These savings arise from the reduction in human intake time, paired with relatively low per-call AI costs across STT, TTS and GPT-4o token usage. The detailed calculations and assumptions can be found in the appendix. These figures represent operational savings only. Any real deployment would require additional investment in integration, infrastructure and monitoring, which are not included in this estimate.

2.6 Discussion and Outlook

This chapter explored replacing the traditional IVR with an LLM-based call intake. Automating the initial stage allows the AI-GC to collect key information before handover, so human agents focus on complex problem-solving rather than routine data collection. The prototype indicates monthly operational savings of around 74,000 EUR, mainly from reduced human intake time after

Commented [NS38]: discussion and outlook?

Azure speech and LLM costs, consistent with reports that front-line AI agents can cut cost per call by up to 50% (Blackader et al. 2025). A pragmatic next step for NOS would be to combine the cost-based GC matching framework from Chapter 1 with an LLM-based intake: the matching model would still decide who handles the call, while the conversational IVR reduces intake time and generates additional variables that could be tested as features in the prediction models. Although integrating the prototype with NOS's live infrastructure and retraining the matching model lies beyond the scope of this work, the results suggest that a future system could use the new IVR as a richer input layer for the cost-based routing framework developed in the group part.

The prototype remains experimental and has several limitations. Latency is higher than would be acceptable in production because speech recognition, LLM inference, and speech synthesis are executed sequentially on a local machine rather than in a streamed architecture. It also operates on a small, fictional call mix instead of live traffic, so its performance is indicative rather than conclusive. Looking ahead, advances in agentic AI could extend automation from intake to full issue resolution (Shulzhenko 2025) by combining LLMs with retrieval-augmented generation over telecom knowledge bases and integrating with customer systems. Future extensions could use a multi-agent setup in which dedicated agents handle tasks such as intake, troubleshooting, routing, and quality control. Such coordinated agents could further reduce costs and improve consistency while keeping components more transparent and controllable. At the same time, human-in-the-loop oversight remains essential: some cases require empathy and judgment, and extensive automation raises concerns about customer dissatisfaction, as many still prefer human representatives and may distrust AI-only service (Zsomborgi 2025). Any expansion of AI in customer service should therefore be paired with human oversight, monitoring of service quality, and transparent communication so that operational gains do not come at the expense of trust or broader social outcomes.

3. Comparing End-to-End and Fragmented Models for Cost Prediction in Telecommunications – Maria Teresa Daffan

3.1 Introduction

Recent advances in machine learning have expanded the methodological choices available for operational modelling, forcing practitioners, especially in business and service analytics such as call-center operations, to balance architectural, performance, and interpretability trade-offs while ensuring that models integrate smoothly into decision processes and remain understandable to non-technical stakeholders. The literature contrasts modular prediction pipelines, valued for transparency and ease of auditing, with end-to-end architectures that optimize the full workflow jointly (e.g., Yan et al. 2022). Empirical studies show that end-to-end models can improve downstream performance, but often at the cost of reduced interpretability and greater engineering complexity (Chen et al. 2020). Conversely, in managerial and operational settings, trust and explainability remain essential for adoption (Assis et al. 2025). Together, these strands provide the foundation for assessing model design choices in operational environments. Against this backdrop, this thesis explores *whether a single end-to-end machine learning model that directly predicts call service costs can outperform a fragmented, multi-model pipeline that estimates cost components separately, while maintaining sufficient interpretability to support real-world decision-making in a call-center context.*

The following chapter synthesizes the relevant literature and positions this question within the broader debate over end-to-end versus fragmented pipelines in terms of both predictive performance and interpretability. Building on the problem framed in Chapter 1, the thesis develops an end-to-end model for call-cost prediction and benchmarks against the fragmented approach, ultimately providing actionable recommendations for operational management.

3.2 Literature Review

3.2.1 Conceptual Foundations of Fragmented and End-to-End ML Architectures

End-to-end (E2E) architectures train a single model from raw inputs to final outputs, removing intermediate components and aligning all parameters with one unified objective (Yan et al., 2022). Fragmented pipelines instead decompose workflows into discrete stages with explicit interfaces, supporting component-level testing, auditing, and staged development, and reflecting longstanding software-engineering practices (Cabrera et al. 2025). These modular designs historically dominated due to clear ownership boundaries, incremental deployability, and easier debugging, audit, and they continue to underpin many production systems (Cabrera et al. 2025). However, recent empirical work shows that eliminating artificial bottlenecks and jointly optimizing pipeline stages can yield superior task performance and generalization to new data, particularly when cross-stage interactions strongly influence the final objective (Chen et al. 2020). Task-specific studies in Natural Language Processing (NLP) show that well-designed joint models can outperform their fragmented counterparts (Yan et al. 2022). Beyond accuracy, integrated designs mitigate error propagation and reduce downstream uncertainty when upstream components are optimized with respect to the final objective (Feiner et al. 2023). Nonetheless, the practical advantages of modularity, e.g., observability and interpretability, remain significant, motivating hybrid strategies that combine E2E optimization with manageable engineering complexity (Shankar and Parameswaran 2022). Thus, while modular pipelines remain highly practical, growing theoretical and empirical evidence positions E2E approaches as a credible and often advantageous alternative.

3.2.2 Evidence from Applied Domains

Applied evidence indicates that E2E designs frequently improve task-level outcomes when models optimize the downstream objective directly. In information extraction, controlled comparisons with

matched encoders show that the best fully joint model surpasses the best pipelined alternative, while poorly structured joint designs can still underperform (Yan et al. 2022). In decision-centric analytics, decision-focused and predict-and-optimize formulations consistently outperform two-stage pipelines across several combinatorial tasks by aligning predictions with decision quality (Elmachtoub and Grigas 2022; Wilder et al. 2019; Vanderschueren et al. 2022). Healthcare studies echo this pattern: a full-stack MRI model jointly handling sampling, reconstruction, and segmentation outperforms modular baselines across multiple datasets (Wang et al. 2024). Similarly, differentiable tabular pipelines that learn preprocessing jointly with the model achieve superior accuracy on most evaluated datasets, reinforcing that upstream optimization against the final objective mitigates quality regressions (Li et al. 2023). Yet domain evidence also highlights where modularity remains competitive: in long-horizon energy forecasting, engineered features with probabilistic LightGBM and hierarchical reconciliation achieve the best results while maintaining transparency and maintainability (Rubattu et al. 2023); and in autonomous driving, modular systems excel with HD maps while E2E models dominate in mapless settings, motivating hybrid designs (Rosero et al., 2024). Overall, E2E or joint training tends to dominate when cross-stage interactions or decision quality determine outcomes, while modular or hybrid pipelines prevail where observability, traceability, and stable interfaces are decisive. For call-center operations, the key question is whether interdependencies are strong enough for joint architectures to outperform modular designs and whether the added complexity is acceptable in practice.

3.2.3 Balancing Interpretability and Performance

A further trade-off in choosing between integrated E2E and fragmented pipelines concerns accuracy versus interpretability. While integrated architectures can maximize predictive performance by optimizing jointly toward the final objective, their complexity limits transparency

and constrains explainability. Conversely, modular or inherently interpretable models offer clearer insight into intermediate processes but may sacrifice accuracy achieved by deeply integrated designs (Barredo-Arrieta et al. 2020). This tension is pronounced in domains where auditability, managerial comprehension, or regulatory oversight are prerequisites for deployment. Empirical evidence across business and operations research reflects this balance: ensemble models deliver superior credit-default predictions, yet explainable techniques are needed to restore managerial acceptance (Bussmann et al. 2021); while hybrid approaches combining strong models with post-hoc explanations similarly balance accuracy and transparency in credit scoring (Bücker et al. 2020). In process analytics, black-box models often yield the highest predictive performance, yet organizations prefer interpretable surrogates or local explanations for accountability (Mehdiyev et al. 2023). Forecasting research likewise shows that opaque models can achieve higher accuracy, while transparent models offer clearer decision rationale and lower latency (Assis et al. 2025), and post-hoc explanations may introduce instability or bias (Retzlaff et al. 2024). Overall, the interpretability-performance trade-off is context-dependent; Molnar et al. (2020) argue for embedding interpretability constraints directly into model selection. In call-center modelling, where predictive reliability and managerial transparency are both essential, this evidence supports joint or E2E designs augmented with interpretable components to ensure trust while retaining the performance benefits of unified learning.

3.3 Methodology

The methodological workflow begins by defining and computing the dependent variable, the result of iterative learning refinement. Once the data is ready, the study implements two modelling tracks showcased in Figure 15.

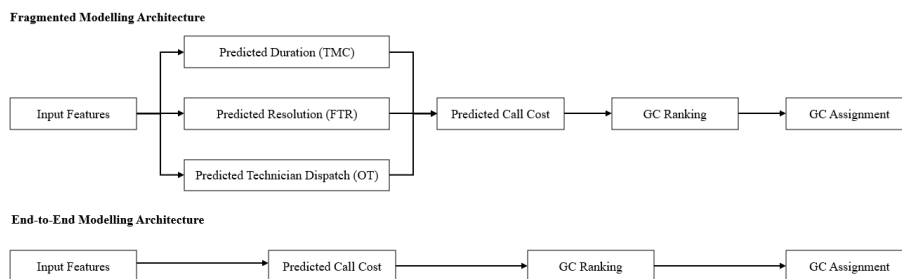


Figure 15. Fragmented vs. E2E Model Architectures

The fragmented pipeline consists of separate models predicting duration, probability of first-time resolution, and probability of technician dispatch. The methodology of these sub-models is described extensively in Chapter 1 of this study, and it will serve as the benchmark against the best end-to-end. Conversely, the end-to-end approach is implemented using three algorithms: Linear Regression, XGBoost, and a feed-forward Neural Network. Once the model is trained on past calls, for each incoming one, the system predicts the cost for all available managers, ranks them, and then assigns the incoming call to the first available one.

3.3.1 Target Variable Definition

The dependent variable of the study is the cost of each call, estimated using the industry-validated Equation (2~~Error! Reference source not found.~~ introduced in Chapter 1. Two methodological challenges emerge. First, the time of a possible follow up call is only observed ex post and is therefore unavailable at the time of the call cost prediction. Second, the large coefficient of $P(OT)$ signals its substantial influence on total cost, yet some technician visits are inherently necessary; thus, a higher expected cost should not automatically be treated as a negative outcome when the intervention is unavoidable. To mitigate these issues, an iterative label-refinement (ILR) procedure was applied before training the main cost models, with the aim of reducing noise in the call-level

Commented [PX39]: ???

Commented [MT39R2]: It's one of the terms in Equation 2 (Chapter 1), should I reinsert the equation? Or just be more specific like this?

cost labels and attenuating the distortions induced by non-observable cost components (Zhang et al. 2025). Firstly, for each call, the original label is computed under the baseline assumption that a potential follow-up call has the same duration as the current call. This approach follows Bastani (2018), who notes that an observable proxy may be used when the required operational variable is not available. Furthermore, multiple alternative cost formulations were computed by estimating the cost of the follow-up call under alternative assumptions (e.g., using group level means) and down-weighting the contribution of the technician dispatch. These were then summarized by their mean and their variance, which was interpreted as a label-uncertainty proxy. Moreover, a simple helper model was trained in an out-of-fold fashion to predict the cost values from stable features (i.e., call type), yielding predictions for each observation and a global estimate of model uncertainty. The final refined label is a convex combination of the original and model-based costs, so that calls with noisier original labels rely more on the model prediction. The refined label showed a highly right-skewed distribution, with most calls costing relatively little and a minority representing very high costs. This pattern is common in cost-modelling fields, such as healthcare or insurance. Teoh (2024) point out that skewness of this kind can severely distort regression estimates. To mitigate this issue, the study applies a natural log transformation to the cost variable (Figure 16).

Commented [NS40]: how many times it happened?

Commented [MT40R2]: All calls. this is an assumption in all of our thesis, because the duration of an eventual follow-up call is never available at the time of the first call (we would have to predict it but we don't have enough data about follow up calls to do so).

Commented [PX41]: the formulas appear mixed up with the text, making impossible to read

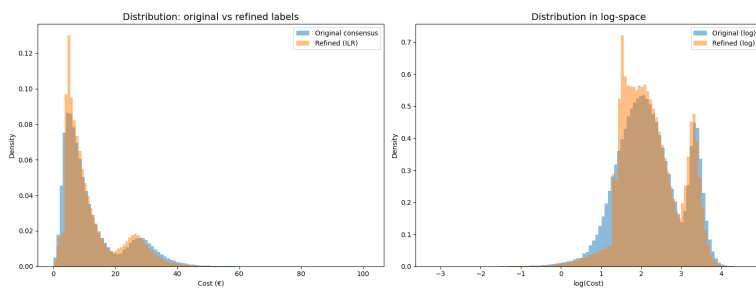


Figure 16. Original and Refined Labels Before and After log Transformation

3.3.2 Preprocessing and Feature Selection

The dataset used in this chapter is identical to that employed in Chapter 1; accordingly, the same preprocessing procedures were applied. Feature selection followed a similar rationale, with an emphasis on ensuring methodological consistency across model families. To guarantee comparability, all E2E models were trained on an identical feature set. First, variables unavailable at the time of prediction were excluded to avoid target leakage and maintain operational relevance (e.g., duration of the call). Secondly, several aggregate variables were available for two alternative time windows; both feature sets were tested, and the configuration yielding superior performance was retained for all subsequent models, namely the one covering the longest time frame, which is particularly beneficial for clients who have been less active recently. Finally, for XGBoost and neural networks, regularization techniques were employed to limit the influence of weak or noisy predictors, enhancing generalization without altering the underlying feature space, as detailed in the subsequent section.

3.3.3 Models Development

The first E2E model is a Linear Regression. As Molnar et al. (2020) notes, the transparency of linear equations makes them especially suitable as baseline models. The Linear Regression model was implemented using scikit-learn's `LinearRegression` estimator. The preprocessing and estimation steps are encapsulated in a Pipeline combining `StandardScaler` for feature standardization and Ordinary Least Squares (OLS) regression as the predictive model. Model training follows an 80/20 train-test split and no regularization is applied, consistent with the goal of using linear regression purely as a simple reference benchmark. The resulting coefficients also offer a preliminary indication of feature importance by reflecting the direction and relative strength of each predictor's association with the target, although these effects are restricted to linear relationships. To capture non-linear relationships and complex interaction effects beyond the

capacity of linear models, the second model employed is an XGBoost regressor, implemented using the XGBRegressor class from the xgboost Python library and trained using a squared-error loss. The dataset is again split into an 80/20 train–test split, with a further subdivision of the training set into a fixed validation set to enable early stopping based on validation RMSE. Model complexity is controlled through a combination of structural hyperparameters and explicit regularization terms (i.e., alpha and lambda). Hyperparameter optimization is performed using RandomizedSearchCV sampling from predefined ranges of hyperparameters. The final XGB model, selected as the best estimator from the random search, combines tree-based non-linearity, regularization, and data-efficient early stopping, offering a more flexible model. To evaluate the performance of a fully non-linear function approximator and benchmark it against tree-based methods, the study also implements a feed-forward neural network using the Keras API of TensorFlow. The model is designed as a multilayer perceptron with four hidden layers (128–128–64–32 units). All layers use the swish¹ activation function, which has been shown to improve optimization stability in regression settings (Dubey et al. 2021). The network incorporates several forms of regularization to mitigate overfitting. A linear activation is used in the final output layer to predict the log-transformed call cost. The training set is further split into a fixed validation subset, and two adaptive training controls are employed² to prevent overfitting and allow the network to converge smoothly towards a stable minimum. The NN provides a benchmark that complements the other models by capturing higher-order feature interactions and non-linear patterns, assessing whether significant performance gains arise from deeper architectures.

Commented [NS42]: already defined

Commented [PX43]: explain

Commented [PX44]: where?

¹ Swish computes $x \cdot \text{sigmoid}(x)$, allowing the network to learn flexible, non-linear transformations while avoiding the hard zeroing of negative values (Dubey et al. 2021)

² EarlyStopping monitors validation loss during training and stops the process once performance fails to improve for 15 consecutive epochs. ReduceLROnPlateau halves the learning rate when the validation loss plateaus for 5 epochs, enabling the optimizer to take smaller, more precise steps as it approaches a minimum.

3.3.4 Evaluation Metrics

Because call-level cost is highly right-skewed, all models are trained on the log-transformed target to stabilize variance and reduce the influence of extreme outliers. However, accuracy in log space is not directly interpretable for practitioners, so model performance is also evaluated on the original euro scale. As naïvely exponentiating predictions introduces retransformation bias, Duan's smearing estimator is applied to obtain unbiased euro-scale predictions. Accordingly, $R_{\log}^2$ and $\text{RMSE}_{\log}$ provide a statistically principled assessment of fit on the transformed target, while the smearing-corrected metrics quantify accuracy in operational terms that are understandable to managers. Given the practical objective of identifying low-cost calls, predictive accuracy alone is not sufficient; the ability to rank calls correctly is also crucial. Technical rank-correlation metrics, Spearman's ρ and Kendall's τ , measure the agreement between predicted and true orderings and remain valid under monotonic transformations such as log-scaling. Pairwise ranking accuracy complements these by offering a more intuitive interpretation: it reports the probability that the model correctly orders two randomly selected calls. To specifically evaluate how well models identify the cheapest calls, the bottom decile is treated as a positive class, and the Area Under the ROC Curve (AUC) is used to assess the model's reliability in surfacing true low-cost cases above others. Taken together, this set of metrics distinguishes between predictive accuracy (how close the predictions are to the true values) and ranking quality (how well the model orders calls by cost) and balances technical measures with managerially interpretable ones. Although models are primarily optimized with respect to accuracy metrics due to their direct operational meaning, ranking performance remains a fundamental component of the evaluation.

3.4 Results and Discussion

As showcased in Table 6, the end-to-end models outperform the fragmented benchmark across most metrics. XGBoost achieves the strongest results, with the highest R^2 (10.8%), the lowest RMSE on both the log scale and the euro scale (8.656 euros with Duan smearing), and the best global ranking behaviour as indicated by Spearman's ρ (0.324) and Kendall's τ (0.219). The neural network performs similarly but slightly worse, while linear regression underperforms both. These results are consistent with the characteristics of the algorithms: XGBoost is specifically designed to exploit structured, heterogeneous tabular data (Shwartz-Ziv and Armon 2021).

Table 6. E2E Model Performance Comparison

Metric	Model			
	Fragmented Model	Linear Regression	XGBoost	Neural Network
R^2	0.097	0.092	0.108	0.105
RMSE	—	0.696	0.689	0.691
R^2 (Duan smearing, euros)	—	0.080	0.092	0.089
RMSE (smearing, euros)	10.919	8.711	8.656	8.666
Spearman ρ	0.313	0.299	0.324	0.319
Kendall τ	0.210	0.201	0.219	0.215
AUC (bottom 10%)	0.646	0.703	0.714	0.712
Pairwise ranking accuracy	60.61%	59.53%	60.28%	60.24%

Commented [NS45]: is this in the same scale as the E2E model? RMSE of a log is the same as the original? the reader will try to compare directly with the 0.689 but it should look at 8.656

The only metric in which the fragmented model performs best is pairwise ranking accuracy (60.61% vs 60.28% from XGBoost). While this appears contradictory, the fragmented model is a deterministic transformation of engineered cost components (TMC, FTR, OT) and therefore preserves built-in monotonic relationships. When two calls differ only slightly in their underlying features, the formula is likely to maintain the correct ordering even if the absolute cost estimate is further from the true value. By contrast, XGBoost and the neural network adjust predictions non-

linearly to reduce global error, which can introduce small inversions in neighboring call pairs. As a result, the fragmented model makes fewer local inversions and achieves slightly higher pairwise accuracy. However, the E2E models recover the global ordering more effectively, as reflected in higher Spearman and Kendall values. Additionally, the XGBoost model performs best in identifying bottom-decile low-cost calls ($AUC = 0.714$), which determines prioritization and cost-saving potential. Notably, the XGBoost and NN have very similar performance metrics. Although neural networks are more flexible than gradient-boosted trees, in this dataset they end up learning very similar mappings because the underlying signal does not contain many deep or high-order non-linear interactions to exploit. A neural network would behave significantly differently only if the data contained richer latent structure, for example, hierarchical interactions, complex temporal dynamics, or continuous latent patterns, that tree models cannot easily capture. In their absence, both models converge to a similar performance ceiling imposed by the data, even though they learn different underlying functions, as evidenced by the SHAP analysis below.

3.4.1 Feature Importance Analysis

To interpret the models, post-hoc explanations were generated using SHAP values for the XGBoost and neural network models and standardized coefficients for the linear regression baseline, additional details are provided in Appendix C. The Neural Network produces attributions that most closely resemble the engineered cost logic (Figure C3), emphasizing smooth relationships between duration, first call resolution, technician dispatch. By contrast, XGBoost reveals a richer set of predictors (Figure C2): in addition to the core cost component of topic average duration, it assigns substantial importance to high-level behavioral and contextual variables such as hour of day, first-call indicators, and recent client activity. These capture second-order operational patterns that meaningfully influence observed costs but are not explicitly encoded in the cost formula. As a tree-

Commented [NS46]: it is not clearly stated, or in the figure (maybe it is there): are the most important variables the same in both models?

Commented [NS47]: missing, and would help to analyse the results

based model optimized for tabular data, XGBoost is designed to detect such nonlinear interactions and context-dependent effects. Although XGBoost and the neural network achieve comparable predictive performance, they do so through different functional representations; in moderately structured tabular data, many distinct functions can approximate the target equally well, which explains why the resulting SHAP patterns differ despite similar performance. Nonetheless, XGBoost attains a slightly higher performance, indicating that its inductive bias is marginally better suited to this prediction task.

3.4.2 Interpretability trade off

The comparison between the fragmented and end-to-end approaches ultimately reflects a clear interpretability-accuracy trade-off grounded in the empirical results. The fragmented pipeline delivers exactly what it is designed to provide: a cost prediction system whose structure mirrors the business logic of TMC, FTR, and OT. Because each component is modelled separately and recombined through the cost formula, the final prediction decomposes naturally into operational drivers. This modularity also supports scenario analysis, since financial parameters or component-level assumptions can be updated without retraining any model. However, this transparency comes at the cost of predictive performance. Fragmentation requires three independent models, each optimized for its own target rather than for total cost, which introduces error propagation and prevents global optimization of the business objective. This is reflected in lower accuracy in euros and weaker global ranking behavior. The small edge in pairwise accuracy stems from the deterministic structure of the cost formula, not from superior modelling. The E2E models, reverse these priorities. By learning a single mapping from features to final cost, XGBoost captures nonlinear operational patterns that the fragmented system does not. This yields lower RMSE and stronger ranking metrics, including the highest AUC for identifying bottom-decile low-cost calls.

The trade-off is a loss of structural decomposability: SHAP explanations offer insight into influential variables but cannot reproduce the separation into cost components.

3.5 Managerial Implications

From a managerial perspective, these differences translate into strategic choices regarding governance, operational clarity, and performance. If transparency and alignment with existing KPIs are top priorities, the fragmented system offers a clear advantage: its structure allows managers to communicate cost drivers to non-technical stakeholders and to perform what-if analyses when cost parameters are updated. Its modularity also reduces perceived model risk, since each component can be validated against historical behavior.

If predictive performance and efficiency are the priority, the E2E model is more compelling. Its higher accuracy enables more reliable identification of low-cost routing opportunities, improving resource allocation and supporting process optimization, while simplifying the IT pipeline and reducing maintenance overhead. Additionally, to approximate the operational impact of adopting the end-to-end model, a simulation was conducted replicating 2,000 incoming calls. For each call, a set of feasible GC–call assignments was generated, the expected handling cost of each assignment was predicted, and the lowest-cost option was selected. While the simulation is a simplified scenario, as the results rely entirely on model-generated predictions, assume full agent availability, and do not incorporate queueing dynamics, under these assumptions, the end-to-end approach yields an average reduction of €2.77 per call, corresponding to an indicative annual impact of approximately €13.8 million given NOS’s call volume.

3.5.1 Recommended Actions for NOS

Given NOS’s operational context, predictive performance should take priority over interpretability, as more accurate predictions and stronger ranking behavior directly reduce routing costs. Any

Commented [NS48]: what is the business impact if we use the E2E. Is the performance improvement resulting in much better results (€ speaking)? Can you simulate it with a scenario?

Commented [NS49]: something a about the EU AI Act and the importance of explaining the predictions?

decision to adopt a new model must, however, weigh these benefits against the effort required to integrate and maintain the system. Furthermore, the EU AI Act places growing emphasis on transparency, auditability, and the ability to provide “meaningful explanations” for algorithmic decisions in high-impact operational settings. This does not prevent the use of high-performing models but requires organization to document the logic of the system, monitor performance, and offer understandable justifications for individual outcomes. A pragmatic solution is to pair XGBoost with an explanatory layer (e.g., SHAP summaries) that communicates the main drivers of routing decisions for compliance and stakeholder reassurance.

As a first implementation step, NOS could deploy XGBoost in a shadow-mode pilot, run it alongside the current routing process, and gradually integrate monitoring, explainability dashboards, and retraining procedures before full rollout.

3.6 Limitations and further research

Despite providing a structured and empirically grounded comparison of fragmented and end-to-end cost prediction models, several limitations must be acknowledged. First, the analysis relies on historical call-center data whose labels contain inherent noise. The cost proxy is an engineered variable that approximates but does not perfectly represent “true” cost, and this affects the ceiling of model performance. Second, although the end-to-end approach optimizes for total cost directly, the fragmented pipeline optimizes its component models independently (TMC, FTR, OT). This asymmetry influences the comparison, as the fragmented system is not globally optimized for the final business objective. Third, model evaluation is conducted on past behavior and assumes stability in operational patterns. Changes in workforce composition, routing policies, product mixes, or external events may introduce distributional shift that degrades performance over time. Finally, the study does not evaluate how model recommendations would interact with behavioral

Commented [NS50]: very good!

responses: managers and clients might adjust their behavior once cost-based assignment is introduced, which could affect the realized savings.

Furthermore, several extensions could strengthen and broaden the findings. First, future work could experiment with learning-to-rank approaches that directly optimize assignment quality, allowing the model to prioritize ordering accuracy in a principled way. Secondly, a promising avenue is the development of hybrid architectures, where the fragmented model supports interpretability and scenario analysis while the end-to-end model handles high-accuracy prediction. Finally, future research should evaluate real-world deployment effects: A/B tests or pilot studies within NOS's routing system would reveal how model-driven assignments influence operational outcomes, manager workload, client experience, and total cost savings.

3.7 Conclusion

This thesis compares fragmented and end-to-end modelling strategies for predicting call-handling costs in a large telecommunications call center. The fragmented approach offers strong structural interpretability and reflects the business logic of call duration, resolution, and technician dispatch, yet its multi-stage architecture introduces error propagation and cannot optimize costs globally. In contrast, the end-to-end models, particularly XGBoost, achieve consistently higher predictive accuracy, stronger ranking performance, and superior identification of low-cost calls. The analysis shows that interpretability and accuracy are complementary rather than opposing aims: the fragmented model remains valuable for diagnostics and stakeholder communication, while the end-to-end model provides the most effective engine for cost prediction. Given NOS's current context, where routing rules are opaque and cost reduction is a priority, the end-to-end approach is the recommended choice for deployment.

4. Identifying Personalized Training Opportunities through Recommendation Systems - Raquel Santos

4.1 Introduction

In the group research project, call routing models were developed to optimize operational efficiency at NOS SGPS. However, this widened skill gaps: strong GCs handled more calls in their areas of expertise, while weaker GCs received fewer opportunities to improve. This dynamic illustrates how optimization systems can unintentionally reinforce disparities by amplifying historical performance patterns (Cowgill & Tucker, 2019; O’Brady, Doellgast, & Blatter, 2024). A purely egalitarian allocation would enhance fairness but undermine efficiency, necessitating a complementary system that prioritizes training based on individual needs. As automation reshapes work, personalized skill development becomes essential for workforce resilience (Li 2022).

The workforce context intensifies this challenge. NOS relies on outsourced GCs, a model associated with high turnover, lower investment in long-term skill development, and fragmented work structures that limit training depth (O’Brady, Doellgast & Blatter 2024). Attrition can reach 60% (Buesing et al. 2020). Outsourced GCs typically receive only basic onboarding and rely on online support resources thereafter, consistent with short-term operational priorities. Even with optimized routing, practical constraints persist: top GCs may be unavailable, call distribution must respect client-balancing rules, and overuse of high performers risks overload. Meanwhile, underexposed GCs may become demotivated, further increasing churn and reducing overall performance.

Information overload adds complexity. With 652 GCs handling 184 topics, manual prioritization of training is not feasible. Similar challenges arise in digital learning environments, where content abundance inhibits learner decision-making (Truss & Böhm, 2024). Recommender systems address

such issues by filtering information and highlighting relevant content (Ricci, Rokach & Shapira 2022). In this setting, they can identify high-impact development topics grounded in performance gaps and exposure history.

GC-topic performance data naturally forms a user–item matrix in which GCs are users, topics are items, and observed entries represent metrics such as average cost per call. Approximately 72% of this matrix is unobserved, mirroring the sparsity addressed in recommender systems (Koren, Bell & Volinsky 2009; Aggarwal 2023). Predicting missing entries enables inference of personalized training needs. Unlike traditional recommenders that identify items a user is likely to prefer, this system identifies topics where the GC is likely to underperform, creating a novel application of collaborative filtering (CF) and matrix factorization (MF) for HR development.

4.2. Literature Review

4.2.1 Fundamentals of Recommender Systems

Recommender systems help users navigate large datasets by ranking items according to predicted relevance. Their main methodological approaches can be summarized as follows:

Method	Key Idea	Advantages	Limitations	Best Use Case	Sources
Content-Based Filtering	Suggests new items by analyzing the characteristics of content the user has previously interacted with, using metadata and user preferences	Interpretable; no need for other users' data; robust to user sparsity.	Requires rich, reliable metadata; limited novelty (recommends “more of the same”).	Domains with well-defined item attributes; personalized content streams.	Ricci et al. (2022)
User-Based Collaborative Filtering	Finds similar users based on shared activity patterns and estimates preferences by	High personalization; intuitive and straightforward.	Poor scalability; sensitive to dynamic user bases; performance	Small-to-medium user bases; systems prioritizing interpretability.	Desrosiers & Karypis (2011)

	aggregating the behavior of those peer users.		degrades under sparsity.		
Item-Based Collaborative Filtering	Recommends items that share similarities with those the user previously engaged with, based on co-occurrence patterns.	Scalable; stable over time; highly explainable.	Less adaptive to rapid user changes; depends on item co-occurrence density.	Large-scale systems, HR analytics, and domains with consistent item relationships.	Sarwar et al. (2001); Ricci et al. (2022)
Matrix Factorization (MF)	Projects users and items into 2 latent matrices, to uncover hidden relationships and predict preferences in sparse datasets.	High accuracy; handles sparse matrices well; became state-of-the-art after the Netflix Prize.	Lower transparency; model interpretation is difficult; performance depends on tuning.	Sparse datasets; settings requiring latent skill/trait identification.	Koren et al. (2009); Aggarwal (2023)
Hybrid Models	Combine CF, MF, and/or content-based features to leverage complementary strengths.	Improved robustness and accuracy; balances interpretability and generalization.	More complex design; higher computational cost.	Production systems with diverse data and performance requirements.	Ricci et al. (2022)

Table 7. Overview of Recommender System Methods.

4.2.2 Evaluation Metrics

Two classes of evaluation metrics dominate. Predictive metrics (RMSE, MAE) assess pointwise accuracy, while ranking metrics (Precision@K, Recall@K, NDCG) evaluate the ordering of recommended items. For training systems, ranking accuracy is more valuable because correctly identifying weak topics matters more than precise numeric predictions. Rank-based measures such as Spearman’s ρ assess whether the model surfaces topics that genuinely require improvement (Herlocker et al. 2004). For recommender systems, GC-stratified (user-stratified) splits are recommended to ensure fair evaluation and avoid information leakage. This setup better simulates

real deployment scenarios, especially under sparse data, by preserving user independence and reducing cold-start bias (Liang et al. 2018).

4.2.3 Recommendation Systems in Learning and Upskilling Contexts

In education, recommenders personalize content to enhance engagement and mastery (Song, Shin, & Shin, 2022). In workforce development, tailored recommendations improve employee satisfaction and performance. As Li (2022) notes, over half of the global workforce requires upskilling due to technological change. Field experiments show targeted training improves call resolution and accuracy in customer service environments (Murthy et al. 2008). Yet, few systems exploit granular performance data to identify micro-skill gaps in operational roles, presenting an opportunity for CF-based HR analytics.

4.2.4 Research Gap and Objectives

Most training recommenders target motivated learners or top performers using course consumption data. Few systems infer performance weaknesses from operational data or focus on supporting underperforming employees in low-autonomy roles such as contact center agents.

Research Question: Can a recommender system identify individualized training priorities for underperforming GCs by predicting weak performance areas from sparse operational data, counterbalancing disparities amplified by performance-optimizing algorithms?

4.3. Methodology

This chapter describes the methodology for generating personalized training recommendations for GCs by predicting their performance on previously unobserved topics and prioritizing development actions. The approach integrates topic aggregation, normalized ratings, collaborative filtering models, evaluation procedures, and a business-oriented prioritization rule.

Commented [NS51]: already defined?

4.3.1 Data Preparation and Feature Aggregation

The analysis utilizes the dataset cleaned and pre-processed in section 1.4 (full_merged_df), which includes all calls handled between January and October 2024, operational metrics (TMC, OT, FTR), a three-level topic classification, and GC identifiers. Data cleaning procedures (Section 1.4.2) removed outliers and ensured consistency in operational variables. A cost metric, combining duration, efficiency, and quality, was calculated for each call, added to the dataset, and used as the primary performance indicator. The GC–topic matrix was constructed by grouping calls by GC and topic, using a concatenation of Level 1 and Level 2 classifications to define the initial topic structure (topic_full: level 1 | level 2). For each GC–topic pair, the total call count and average values for Cost, TMC, OT, and FTR were calculated. Two minimum thresholds were tested: one with a cutoff of three calls per pair (producing 642 GCs and 102 topics with 72.3% sparsity) and a stricter threshold of seven calls, later applied to the final matrix. This higher threshold improved the reliability of topic-level performance estimates and reduced model noise, though at the cost of excluding rarer topics. Most GCs do not address every topic. On average, each GC handled 28 topics, and each topic was seen by 178 GCs. This level of sparsity justifies the use of a matrix completion technique.

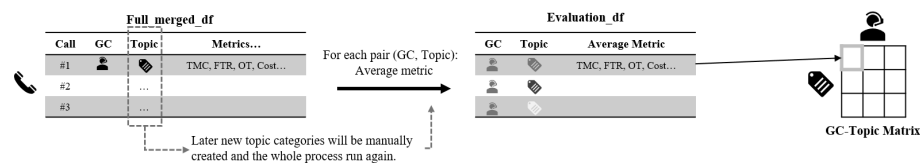


Figure 17. Construction of the GC–Topic Matrix

To address the unbalanced initial topic distribution, topics were later manually reorganized using all three classification levels: at each level, low-volume categories were merged into an 'Other'

group, conceptually related categories were combined, and certain high-volume topics were further subdivided up to Level 3 (the process may be found in the Appendix D). The effects of these adjustments on the dataset balance and model performance are provided in the Results section.

4.3.2 Rating Construction and Normalization

Operational metrics vary across topics, making raw values incomparable. To enable fair comparisons, each GC's performance per topic was converted into a 1–5 relative rating, with lower ratings indicating weaker performance. Operational metrics differ across topics, making raw values incomparable. Three rating methods were tested.

Quintile-based ratings enforced uniform distributions but exaggerated minor differences near the median. The second approach, mean-centered σ -band ratings, was distorted by right-skewed topic distributions, making underperformance hard to detect. The final deployed model, Median-centered σ -band normalization, used the median for robustness under skew and divided the $\pm 1\sigma$ region into three equal-width intervals ($\frac{2}{3}\sigma$), with values beyond $\pm 1\sigma$ assigned ratings 1 or 5 (Montgomery & Runger, 2018). Lower-is-better metrics applied directly; FTR and call volume were inverted. This produced stable, comparable 1–5 ratings across topics. A full comparison of methods is provided in Appendix D.

The final rating rules are:

- More than 1 σ below median $\rightarrow$ Rating 5 (best)
- $[\text{median} - 1\sigma, \text{median} - \frac{1}{3}\sigma[\rightarrow$ Rating 4
- $[\text{median} - \frac{1}{3}\sigma, \text{median} + \frac{1}{3}\sigma] \rightarrow$ Rating 3
- $] \text{median} + \frac{1}{3}\sigma, \text{median} + 1\sigma] \rightarrow$ Rating 2
- More than 1 σ above median $\rightarrow$ Rating 1 (worst)

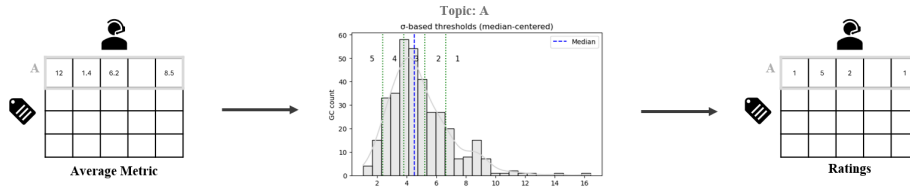


Figure 18. Conversion of Average Metrics into Ratings Using Median-Centered σ -Bands

For metrics where lower is better (Cost, TMC, OT), this was applied directly; for FTR and call volume, the scale was inverted. From this point onwards, the only performance metric considered will be the corresponding rating for **average cost** for a simple and baseline approach, since it already integrates and balances all important metrics, knowing that we could construct the same rating and consequent models using any metric (FTR, TMC, OT, number of calls...) for a more targeted outcome.

4.3.3 Collaborative Filtering Framework

CF was selected over content-based methods due to the absence of reliable topic-level descriptors and because CF leverages interaction patterns alone (Ricci, Rokach & Shapira 2015).

4.3.4 End-to-End Workflow Overview

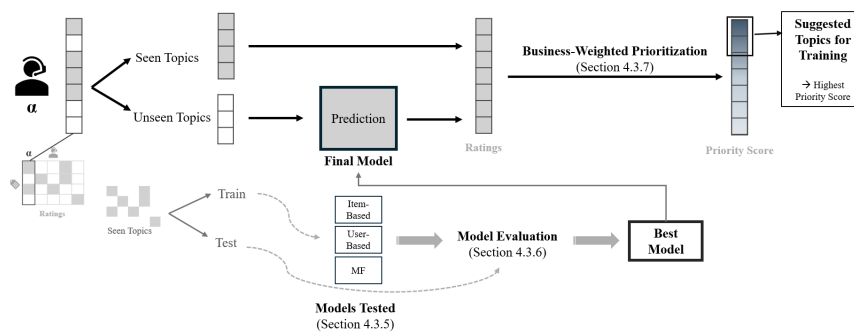


Figure 19. End-to-End Pipeline for Model Training, Prediction, and Prioritization

4.3.5 Models Tested

4.3.5.1 User-Based Collaborative Filtering

In this approach, each GC is described by a vector of their observed topic ratings. Cosine similarity is calculated between GCs to determine their behavioral proximity. For each missing GC–topic combination in the test set, a score is estimated by aggregating the ratings of the $k=10$ most similar GCs, weighted by their similarity scores. Zero is used to impute missing values during training, reflecting the absence of observed performance rather than failure. The choice of $k=10$ follows common practices in the literature (Desrosiers & Karypis, 2011; Ricci et al., 2022), though extensive tuning was not prioritized. The goal was to assess model trends, not to maximize predictive accuracy through hyperparameter optimization. Performance remained stable across alternative values of k ($5 \leq k \leq 20$). All similarity computations used pandas and scikit-learn libraries.

4.3.5.2 Item-Based Collaborative Filtering

Here, similarity is computed between topic vectors instead of user vectors. For a given GC, the system identifies the $k=10$ topics most similar (based on cosine similarity) to an unseen one, using the GC’s past ratings on those topics to generate a predicted score. As in the user-based model, a weighted average is computed using similarity as the weighting factor. This method assumes that topic-to-topic relationships are more stable over time than user-to-user patterns. Implementations relied on numpy, pandas, and scikit-learn’s built-in similarity tools.

4.3.5.3 Matrix Factorization

Matrix Factorization (MF) decomposed the GC–topic matrix into latent factor matrices that capture hidden interaction structures (Koren et al., 2009). A custom SGD implementation in numpy was used. Five configurations were tested: (10,10), (20,20), (40,40), (20,40), and (40,20) for (factors,

epochs). The final MF results used the (10,10) configuration, which achieved the best trade-off between accuracy and stability. Learning rate and regularization (0.01 and 0.02, respectively) followed standard defaults (Lo et al., 2018).

4.3.6 Model Evaluation

4.3.6.1 Train–Test Split Strategies

The primary evaluation used a GC-stratified random holdout, with each GC receiving ~20% of their topic pairs in the test set. This preserves user independence and avoids cold-start effects during evaluation, aligning with best practices in recommender systems for sparse, user-specific data, as discussed in the literature review. Additional split strategies were explored for robustness, though they are not central to the main results. These include topic-stratified split, ensuring that every topic appears in both training and testing, and rare-topic split, sending rare topics to the test set to simulate cold-start scenarios.

4.3.6.1 Evaluation Metrics

Given the objective of prioritizing training opportunities for each GC, the evaluation focused on both prediction accuracy and ranking quality.

Pointwise accuracy was assessed using mean absolute error (MAE) and root mean squared error (RMSE), with RMSE penalizing larger deviations more heavily. And as described in section 1.5.4, pairwise accuracy evaluates whether the model maintains the correct ordering of topics for each GC by comparing predictions and actual values for all topic pairs with differing ratings.

To assess how well models identify each GC’s low, mid, and high-performing topics, true and predicted ratings were grouped using the GC’s own 20th and 80th percentiles. Precision (of the topics the model labeled as “low”, how many are truly low?), recall (of the topics truly “low”, how

many did the model correctly label?), and F1-Score (the harmonic mean: $F1 = \frac{2PR}{P+R}$) were computed from per-GC confusion matrices and averaged across GCs.

NDCG@5 was used for ranking quality. NDCG_low measures the ability to rank weak topics (rating ≤ 2) near the top of the “improvement list”. In the same logic, NDCG_top captures the model’s ability to correctly rank strong topics (rating ≥ 4), complementing the analysis. NDCG is widely used to evaluate ordered recommendations, rewarding correct placement of high-relevance items at the top (Järvelin & Kekäläinen 2002; Manning et al. 2008). NDCG scores were computed using custom implementations based on true binary relevance (low-performing topics ≤ 2 , high-performing ≥ 4) and standard discounted cumulative gain formulas.

These metrics collectively assess accuracy, ranking effectiveness, and practical interpretability.

4.3.7 Business-Weighted Prioritization

Following the prediction of unseen GC–topic pairs, a business-weighted prioritization rule was implemented:

$$\text{Priority Score} = (C_{topic,r} - C_{topic,3}) \times f_{topic}$$

Here, r is the predicted or actual rating, C is the mean cost per rating level for that topic, and f_{topic} is its frequency. This penalizes low ratings on topics with high cost differences and high occurrence.

It stresses development opportunities with measurable business impact.

The score emphasizes improvement potential, with higher ratings corresponding to higher priority, as well as operational cost savings and impact through topic frequency.

The prioritization formula is applied to all ratings, seen and predicted, and ensures that training recommendations are personalized and aligned with business value.

4.4 Results

Table X summarizes model performance across three dataset configurations: the original taxonomy with thresholds of ≥ 3 and ≥ 7 calls, and the reaggregated taxonomy with ≥ 7 calls. Topic coverage and sparsity are included to contextualize differences in predictive behavior.

Dataset	# GCs	# Topics	% Coverage of original “level 1 level 2” topics	Sparsity
Original ≥ 3	652	102	56%	72.20%
Original ≥ 7	636	78	43%	76.20%
Reaggregated ≥ 7	636	76	80% (+ New level 3 topics)	65.80%

Table 8. Dataset Characteristics (Shape, Coverage, Sparsity)

Dataset	Model	MAE	RMSE	Pairwise	GC F1	NDCG_low	NDCG_top
Original ≥ 3	User	1.55	1.89	0.48	0.65	0.73	0.49
	Item	0.88	1.13	0.51	0.66	0.59	0.55
	MF	0.89	1.13	0.51	0.66	0.431	0.57
Original ≥ 7	User	1.66	2.02	0.46	0.77	0.64	0.42
	Item	0.87	1.11	0.55	0.78	0.59	0.55
	MF	0.89	1.13	0.56	0.78	0.43	0.62
Reaggregated ≥ 7	User	1.53	1.87	0.52	0.70	0.55	0.48
	Item	0.90	1.13	0.53	0.69	0.53	0.61
	MF	0.96	1.20	0.54	0.69	0.61	0.56

Table 9. Model Performance Metrics

4.4.1 Accuracy and Ranking Performance

Item-based CF achieves the lowest MAE and RMSE across all configurations (e.g., 0.879 / 1.126 under ≥ 3), but pointwise accuracy is not central in a training context. The key outcomes are category identification and ranking.

GC-level classification improves sharply when the minimum-call threshold increases from ≥ 3 to ≥ 7 (e.g., item-based F1 from 0.662 to 0.783), indicating that ratings constructed from very small sample sizes were unstable. After reaggregation, F1 stabilizes around 0.69, slightly lower than under the ≥ 7 original taxonomy due to smoother intra-topic variance, yet still substantially higher than the ≥ 3 baseline.

Pairwise accuracy shows modest increases when data quality improves. MF rises from 0.514 to 0.555 under the ≥ 7 threshold with original taxonomy and maintains 0.544 after reaggregation. However, pairwise accuracy is partially limited by the rating scale itself. Because true ratings are integers (1–5) and predictions are continuous, many GC–topic pairs share identical true ratings. These “ties” are ignored by the pairwise metric, which causes apparent compression toward 0.50. This mechanical constraint does not reflect poor model performance; it reflects the discrete nature of the rating scale. Metrics such as GC-level F1 and NDCG, therefore, provide more meaningful insight.

NDCG_low indicates different strengths across configurations. Under the original ≥ 3 taxonomy, user-based CF performs best (0.733). After reaggregation, MF becomes the strongest model (0.612), reflecting the value of coherent topic structure for latent-factor learning. MF also performs competitively on NDCG_top (0.555), suggesting balanced ability to identify strengths.

4.4.2 Coverage, Sparsity, and Structural Improvements

Raising the threshold to ≥ 7 reduces the number of topics from 102 to 78 while increasing sparsity to 76.2%, resulting in a cleaner but much narrower matrix. Reaggregation partially reverses this trade-off: it recovers 80% of the original “level 1 | level 2” topics, incorporates additional Level 3 subtopics, and reduces sparsity to 65.8%. This not only improves topic coverage but also enhances the semantic coherence of topic groupings. Structural diagnostics confirm these gains: median

item–item similarity increases from 0.13 to 0.43, and GC separability improves (silhouette 0.13 $\rightarrow$ 0.21; PCA explained variance 0.32 $\rightarrow$ 0.45). These structural improvements help explain why MF, in particular, benefits most from the reaggregated taxonomy.

4.4.3 Robustness Across Split Strategies

Models were also evaluated under random, topic-stratified, rare-topic, and leave-one-topic-out splits. Performance under GC-stratified, random, and topic-stratified splits was broadly consistent, with MF maintaining strong results (e.g., NDCG_low 0.603 under GC-stratified and 0.593 under topic-stratified). This indicates that the system generalizes well and does not depend on a single split strategy.

Rare-topic splits, which place low-frequency topics entirely in the test set, led to lower performance across all models (MF NDCG_low $\approx$ 0.49). This confirms that topic-level cold start remains a structural challenge. GC-level cold start is avoided by construction, but unseen-topic performance will require future enrichment through text features or metadata.

4.5 Discussion

The results show how data quality and topic structure jointly determine model behavior. The ≥ 7 threshold greatly improves classification accuracy (e.g., MF 0.775) by removing unreliable GC–topic estimates but also excludes over half of the original topics. In NOS’s context, where rare topics may carry substantial business cost, this reduction undermines practical applicability.

Reaggregation provides a more balanced solution. It restores almost all topics, decreases sparsity, and strengthens similarity structures, without reintroducing excessive noise. The smoother topic variance explains the slight reduction in GC-level F1 relative to the ≥ 7 original taxonomy, but this is offset by meaningful gains in coherence and coverage. In addition to rescuing overlooked small-volume topics, reaggregation also helps disaggregate broad, high-frequency topics that may

obscure meaningful internal variation, such as distinct sub-issues where GCs perform differently but are otherwise grouped under a single umbrella. This greater granularity improves both recommendation quality and the fairness of training prioritization.

The improved structure allows MF to become the strongest overall model under the final configuration, especially for ranking weak topics (NDCG_low 0.612).

Two limitations remain. First, two very large topics dominate the dataset even after reaggregation. Their size affects model balance, the reliability of latent factors, and the fairness of business prioritization, since these topics disproportionately influence cost-based ranking. Second, several semantically similar topics were historically classified separately. These inconsistencies limit item-item similarity and suggest further opportunities for data-driven refinement once text descriptions become available.

4.6 Business Impact and Future Work

A controlled A/B test should be conducted to quantify operational gains from MF-driven training recommendations. The evaluation should track handling time, cost per call, FTR, repeat interactions, and GC feedback. Because the prioritization rule incorporates topic frequency and cost differentials, its effect must be assessed jointly with the model predictions.

MF's balanced ranking of strengths (NDCG_top 0.555) also supports GC specialization. Identifying strong topics can assist team formation, improve routing alignment, and enhance motivation through clearer role differentiation, such as specialized teams.

Future taxonomy enhancements should focus on dividing the two oversized topics and merging categories with overlapping semantics. Embedding-based clustering or text-derived features would help mitigate topic-level cold start and improve item coherence. Ensemble approaches combining

MF with neighborhood models may offer additional robustness, since models show different strengths in accuracy and ranking capabilities.

Computational Complexity: User- and item-based CF trains quickly but becomes slower as the number of GCs or topics increases because they must repeatedly compute similarities. MF is slower to train but extremely fast at generating predictions, making it more scalable once deployed and well-suited for periodic offline retraining.

4.7 Conclusions

This study demonstrates that personalized, data-driven training recommendations for GCs are feasible despite high sparsity and operational complexity. Two methodological contributions, median-centered σ -band normalization and a reaggregated topic taxonomy, provide a coherent foundation for modelling GC–topic performance. Thresholding improves reliability but reduces topic coverage; reaggregation restores coverage while improving structure, enabling MF to achieve the strongest overall ranking performance (NDCG_{low} 0.612, F1 0.693).

The remaining topic imbalances and cold-start limitations highlight the importance of continued taxonomy refinement and the incorporation of text-based features in future iterations. While the prototype is methodologically sound, its practical relevance for NOS may be limited by organizational priorities. Nevertheless, it offers a transferable proof of concept for contexts where individualized skill development is more strategically aligned. As NOS expands the use of automated routing systems, the ability to infer unobserved weaknesses and strengths becomes increasingly relevant, especially to combat bias and fairness issues. The results indicate that MF, supported by a structured taxonomy and a business-weighted prioritization layer, offers a promising path for scalable, personalized development in the contact center environment.

5. Optimizing Call Routing Decisions Using Reinforcement Learning – Marvin Schumann

5.1 Introduction

Call center routing presents a complex sequential decision-making problem where each routing decision affects future system states through agent availability dynamics. Traditional rule-based approaches, while interpretable, may not fully exploit the sequential structure of the problem. Reinforcement Learning (RL) offers a principled framework for learning optimal policies through interaction with the environment, potentially discovering routing strategies that outperform hand-crafted heuristics.

This section investigates whether RL can learn effective routing policies for the NOS call center. The problem is formulated as a Markov Decision Process (MDP) with 653 agents (the full NOS workforce), and a Proximal Policy Optimization (PPO) agent is trained using action masking to handle availability constraints. A rigorous simulator validation methodology is implemented, and RL performance is compared against three baselines: Random, Rule-Based (NOS heuristic), and Greedy XGBoost (optimal one-step lookahead).

5.2 Literature Review

5.2.1 RL Applications to Call Center Routing

Recent work by Li and Karunaratne (2025) demonstrated that PPO can outperform Value Iteration in call routing tasks, motivating the algorithmic investigation pursued here. Wix Engineering emphasized the importance of simulator fidelity and staged implementation when deploying RL in production contact centers. Schulman et al. (2017) introduced PPO, highlighting its stability

advantages over earlier policy gradient methods, which are critical for constrained action spaces like those encountered in call center routing.

5.2.2 Reinforcement Learning Foundations

The theoretical foundations for applying RL to sequential decision-making problems are established in Sutton and Barto (2018), who provide a comprehensive treatment of Markov Decision Processes, value functions, and policy optimization methods. The MDP framework enables formal representation of problems where each action affects not only immediate rewards but also future system states. This is precisely the characteristic of call center routing where agent assignments impact future availability and workload distribution. Classical dynamic programming approaches like Value Iteration provide optimal solutions for small state spaces but become computationally intractable as dimensionality increases, motivating the development of function approximation methods.

5.2.3 Deep RL Algorithms: Policy-Based vs Value-Based Methods

Deep reinforcement learning methods combine RL with neural network function approximators to handle high-dimensional state and action spaces. Mnih et al. (2015) introduced Deep Q-Networks (DQN), demonstrating that value-based RL could achieve human-level performance in complex domains through experience replay and target networks. However, value-based methods face challenges in continuous or large discrete action spaces. Policy gradient methods, particularly Proximal Policy Optimization (Schulman et al. 2017), address these limitations through direct policy optimization with stability guarantees via clipped objective functions. The choice between value-based (DQN) and policy-based (PPO) methods depends on problem characteristics, action space structure, and constraint handling requirements.

5.2.4 Action Masking for Constrained RL

Action masking techniques have emerged as essential for handling hard constraints in RL applications. Traditional RL algorithms may waste significant training time exploring invalid actions, that is, actions that violate problem constraints such as selecting unavailable agents. Action masking restricts the policy's sampling distribution to valid actions only, eliminating invalid action penalties and accelerating convergence. While Stable-Baselines3 provides MaskablePPO with native masking integration (Raffin et al. 2021), generic action masking wrappers exist for other algorithms. However, the effectiveness of these wrappers varies across algorithm families, particularly between on-policy (PPO) and off-policy (DQN) methods, due to differences in how experience replay and target networks interact with dynamic action masks.

5.2.5 Research Gap and Contribution

However, several gaps remain: (1) Limited guidance on simulator validation before RL training, (2) No prior work applying action masking to call routing with 653 agents across multiple RL algorithms, and (3) Insufficient methodological frameworks for determining when RL is appropriate versus simpler alternatives. This section addresses these gaps through rigorous simulator validation, comparative algorithm analysis (PPO vs. DQN), and empirical evaluation against strong baselines.

5.3 Markov Decision Process Formulation

5.3.1 State Space

At each timestep t , the state encompasses call features, agent availability, and temporal context. Formally, the state is $s_t = [x_t, a_t, h_t]$ where: $x_t \in \mathbb{R}^{50}$ represents call feature vector including customer tier, call topic, queue wait time, and historical interactions; $a_t \in \{0,1\}^{653}$ is the binary

agent availability vector where $a_t[i] = 1$ if agent i is available; and $h_t \in \mathbb{R}^2$ captures temporal features including time-of-day and day-of-week. The total state dimension is $N_s = N_c + N_a + 2 = 50 + 653 + 2 = 705$.

5.3.2 Action Space

The action space is discrete: $A = \{0, 1, \dots, 653\}$ where each action corresponds to selecting an agent to handle the current call. At each timestep, only a subset of agents are available, defined by the validity constraint $A_{\text{valid}}(s_t) = \{i \in A : a_t[i] = 1\}$. The simulator checks two availability conditions: (1) agent i is within their scheduled shift hours, AND (2) agent i is not currently handling another call. In the current configuration, all agents are assigned to a single 8-hour shift (hours 0-8) matching the episode duration, so availability is effectively determined by call handling status. Agents become unavailable when assigned a call and return to availability after the predicted handling time (TMC) elapses. Selecting an unavailable agent results in an invalid action that incurs a 60-second penalty and does not route the call. Action masking (Section 5.5.3) restricts the policy to sample only from $A_{\text{valid}}(s_t)$, eliminating invalid actions entirely.

5.3.3 Reward Function

The reward function directly minimizes the cost per call: $R(s_t, a_t) = -C(s_t, a_t)$, where $C(s_t, a_t) = (\text{TMC}/60) \times 0.35 \times (1 + (1 - P(\text{FTR}))) + P(\text{OT}) \times 22.00$. The cost components reflect agent labor (€0.35/minute), repeat call probability $(1 - P(\text{FTR}))$, and technician dispatch costs (€22.00). The negative reward formulation converts cost minimization into reward maximization, aligning with standard RL conventions.

5.3.4 Discount Factor and Transition Dynamics

Commented [NS52]: only based on call or also on working time?

Commented [MS52R2]: Both. The code checks shift hours AND call status, but all agents work the same 8-hour shift covering the full episode, so availability is effectively determined by call handling only.

--> I also clarified this in the text just below your comment.

Thanks!

A discount factor $\gamma = 0.99$ is used, providing an effective horizon of approximately 100 timesteps (16 hours). The transition function $P(s_{t+1} | s_t, a_t)$ governs system evolution: call arrivals follow a time-dependent Poisson process, selected agents become unavailable for TMC minutes as predicted by XGBoost, and the environment progresses to the next call when agents return to the available pool.

5.4 Simulator Validation

Before training RL agents, the simulator's ability to predict actual costs must be validated. RL relies on accumulated reward feedback; if the simulator provides inaccurate cost estimates, agents will learn from false signals and converge to suboptimal policies. A rigorous validation protocol is implemented using an 80/20 train-test split (random_state=42). For each test call, XGBoost models are queried to predict TMC, P(FTR), and P(OT) for the historical agent, predicted cost is calculated, and results are compared to actual cost from ground truth outcomes.

Evaluation metrics and thresholds include: MAE < €3.00 (within 20% of mean), RMSE < €5.00, $R^2 > 0.50$, and Pearson correlation $r > 0.70$. Table 5.1 presents results showing correlation $r = 0.072$ (far below 0.70 threshold), $R^2 = -0.248$ (explains no variance), MAE = €8.90, and RMSE = €12.87. All metrics fail validation, indicating approximately 92% noise in the training signal. This predicts that RL will struggle, as policy gradients will be dominated by random fluctuations.

Metric	Value	Threshold	Status
Cost Correlation	0.072	0.70	FAIL
Cost MAE (€)	8.90	3.00	FAIL
Cost RMSE (€)	12.87	5.00	FAIL
Cost R^2	-0.248	0.50	FAIL

Commented [NS53]: looks like you knew it will not work, is what you want to say?

Commented [MS53R2]: Kind of. But the thing is that I ran the RL training first, got poor results, then did simulator validation to understand why. The $r=0.072$ confirmed the issue was simulator quality.

So the contribution could be a demonstration that this validation COULD have been done upfront to predict failure before training, saving that effort.

Does that make sense?

5.5 Methodology

5.5.1 Environment and Training

A custom Gymnasium environment (CallCenterEnv) was implemented to simulate the NOS call center with 653 agents and time-dependent Poisson call arrivals. Each episode simulates one operational day (8 hours), handling approximately 600 calls. PPO from Stable-Baselines3 is employed with a two-layer [256, 256] fully connected architecture, learning rate $\alpha = 3 \times 10^{-4}$ with linear decay, batch size 2048, 10 epochs per update, discount factor $\gamma = 0.99$, GAE $\lambda = 0.95$, and PPO clip $\epsilon = 0.2$. Training for 200,000 timesteps (~330 episodes) on MacBook M3 Pro (CPU mode) completed in 7.5 hours at ~9 fps.

5.5.2 Action Masking Implementation

To handle the availability constraint $A_valid(s_t)$, MaskablePPO from SB3-Contrib is used, which natively integrates action masking into the policy network. Specifically, the MlpPolicy (Multi-Layer Perceptron policy) is employed, configured with a two-layer [256, 256] fully connected architecture as described in Section 5.5.1, suitable for the 705-dimensional vector state space. At each timestep, a binary mask indicates which of the 653 agents are available. Before sampling actions, the policy network's logits for unavailable agents are set to $wfm-\infty$, ensuring the softmax distribution assigns zero probability to invalid actions. This guarantees selection from only available agents.

5.5.3 Algorithm Comparison: PPO vs. DQN

To validate that action masking is algorithm-independent, DQN was additionally tested. Two experiments were conducted: (1) Unmasked DQN (50,000 timesteps) to demonstrate that the invalid action problem affects value-based methods, and (2) Masked DQN with ActionMasker

Commented [NS54]: based on NOS data?

Commented [MS54R2]: The 600 calls are a still a simplified scenario with an artificial Poisson distribution to simulate peak hours within one shift. NOS, in reality, receives about 3k calls per day with about 1,8k during peak 8 hours.

While relative rankings of different models wouldn't change, I could re-train and evaluate to get more accurate absolute numbers?

Commented [NS55]: https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html#sb3-contrib.ppo_mask.MlpPolicy

It has several policies, which one did you consider?

Commented [MS55R2]: MlpPolic. it's the standard choice for vector observations (our state is a 705-dimensional vector). The [256, 256] architecture is already documented in Section 5.5.1 but I also added clarification below your comment ^^

wrapper (200,000 timesteps) to test whether DQN could match PPO performance. DQN hyperparameters included: learning rate 0.0003, buffer size 100,000, batch size 256, exploration fraction 0.1, final epsilon 0.05, gamma 0.99, seed 42.

5.5.4 Baseline Policies and Evaluation

Three baselines are used for comparison: (1) Random uniformly selects from available agents, (2) Rule-Based implements NOS's current heuristic, and (3) Greedy XGBoost queries XGBoost to predict costs for all available agents and selects the minimum. All policies are evaluated on the same held-out test set (20% of data, 36 days) with 10 episodes per policy. Costs are determined by XGBoost predictions for consistency, as ground truth is only available for historical assignments.

5.6 Results

5.6.1 Action Masking Validation Across Algorithms

Table 5.2 demonstrates that the action masking problem is algorithm-independent. Both Unmasked PPO and Unmasked DQN exhibit catastrophic failure with 86-89% invalid action rates and efficiency below 8%, confirming that observing agent availability in the state space is insufficient—hard constraints via action masking are fundamentally necessary. Unmasked DQN achieves 8.0% efficiency with 89.2% invalid actions, handling only 47 calls per day despite having agent availability information in observations. This validates that even value-based methods cannot learn to respect constraints through observation alone.

Masked PPO achieves 104% efficiency (614.5 calls/day) with near-zero invalid actions, representing a 17 $\times$ improvement over unmasked PPO. However, Masked DQN with the ActionMasker wrapper fails catastrophically, achieving only 5.2% efficiency (30.6 calls/day) despite action masking. This reveals that the generic ActionMasker wrapper does not integrate

properly with value-based methods—experience replay and target networks create mask inconsistencies that prevent DQN from learning effectively. This finding validates the selection of MaskablePPO, which provides native action masking integration, as the only viable RL approach for this problem.

Policy	Calls/Day	Efficiency	Invalid %
PPO (no masking)	37.2	6.3%	86.4%
DQN (no masking)	47.0	8.0%	89.2%
Masked PPO	614.5	104.2%	~0%
Masked DQN	30.6	5.2%	~95%

Commented [RK56]: @Marvin Valentin Schumann all tables in your chapter dont have the correct formatting yet.

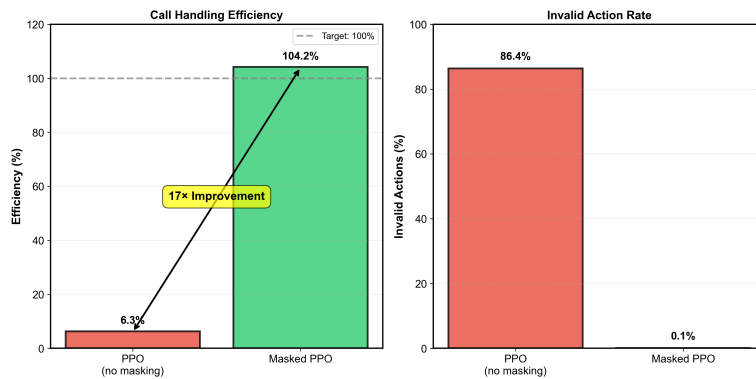


Figure 20. Action masking impact - from 6% to 104% efficiency

5.6.2 Policy Performance Comparison

Table 5.3 presents the main performance comparison. Greedy XGBoost achieves the lowest cost at €15.06 per call. Rule-Based costs €15.54 per call (+3.2% vs. Greedy), demonstrating that NOS's heuristic performs reasonably well. Masked PPO costs €16.21 per call (+7.6% vs. Greedy), underperforming both Greedy and Rule-Based. However, Masked PPO beats Random (€18.18) by

a substantial margin, proving that the RL agent learned meaningful routing patterns despite the noisy training signal.

Policy	Cost/Call (€)	vs. Best
Greedy XGBoost	15.06	—
Rule-Based	15.54	+3.2%
Masked PPO	16.21	+7.6%
Random	18.18	+20.7%

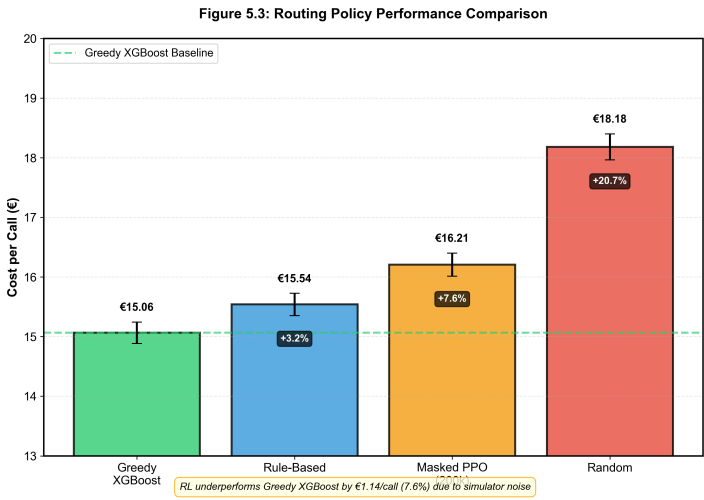


Figure 21. Routing policy performance comparison

The performance gap between Masked PPO and Greedy XGBoost (€1.14/call) is statistically significant ($p < 0.001$). At NOS's volume of 300,000 calls per month, deploying RL instead of Greedy would incur an additional cost of €342,000 per month. Masked DQN (€16.92/call, not shown in table due to catastrophic efficiency failure) would incur even higher costs, confirming that the simulator noise issue affects all RL methods.

5.6.3 Training Dynamics

Analysis of training curves reveals that the Masked PPO agent plateaued after approximately 50,000 timesteps. Episode rewards improved rapidly in the first 50,000 steps (from -17.0 to -16.2), then remained flat for the subsequent 150,000 timesteps. Policy entropy decreased throughout training, indicating increasing confidence in action selections. However, value function loss remained high ($MSE > 200$), suggesting difficulty in accurately predicting returns—a symptom of the noisy reward signal. The early plateau indicates that the agent extracted all available signal from the noisy simulator within 50,000 timesteps. Continued training did not improve performance because the 92% noise ceiling prevented further learning.

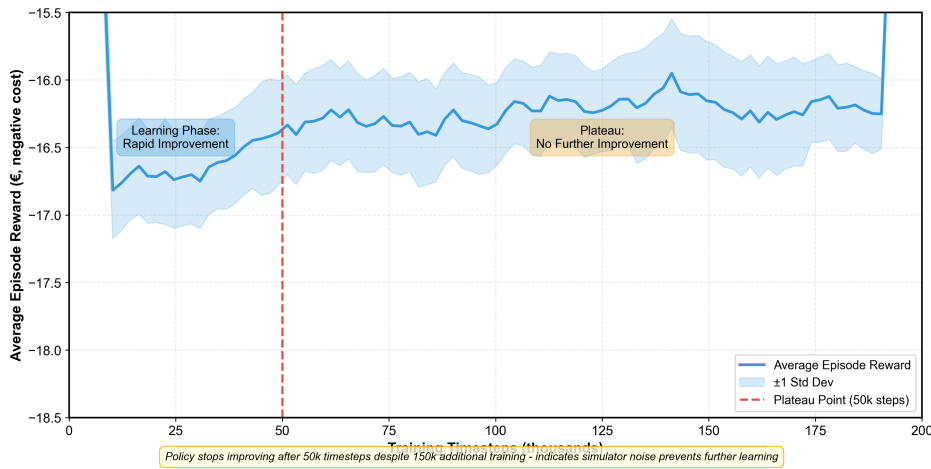


Figure 22. RL training dynamics - early plateau indicates learning limit

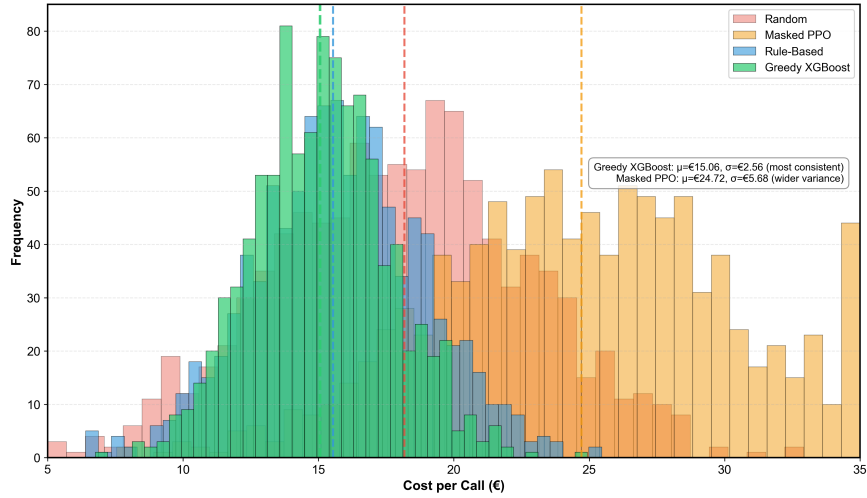


Figure 23. Cost distribution by routing policy

5.7 Discussion

5.7.1 Why RL Underperforms Greedy XGBoost

The central puzzle is: if both RL and Greedy use the same XGBoost predictions, why does Greedy outperform? The answer lies in their fundamentally different decision-making mechanisms. Greedy XGBoost performs online optimization: for each call, it queries XGBoost to predict costs for all 653 available agents and selects the agent with minimum predicted cost. This argmin operation over 653 noisy predictions is robust to noise—even if individual predictions have high variance, selecting the minimum of many samples tends to find reasonable options. Greedy makes no assumptions about sequential dependencies and requires no learning.

Masked PPO performs offline learning: during training, the agent observes states, takes actions, receives rewards (negative predicted costs), and updates its policy network via policy gradients. The issue is that with correlation $r = 0.072$, approximately 92% of the reward signal is noise. Policy

gradients computed from such noisy rewards point in random directions, preventing convergence to the optimal policy. The agent learns a compressed representation ($705 \rightarrow 256 \rightarrow 256 \rightarrow 653$) that captures patterns from noisy training data, but this learned policy cannot match the performance of fresh argmin optimization at test time. The fact that RL beats Random (€16.21 vs. €18.18) demonstrates that the agent learned something meaningful. The plateau at 50,000 timesteps shows the agent extracted all available signal. However, the gap to Greedy (€16.21 vs. €15.06) reveals the noise ceiling: policy gradient methods cannot converge to optimal under such noisy feedback.

5.7.2 Algorithm-Independent Findings

The comparative analysis of PPO and DQN validates two critical findings. First, the action masking problem is algorithm-independent: both policy-based (PPO) and value-based (DQN) methods fail without masking, taking 86-89% invalid actions despite agent availability information being present in observations. This proves that hard constraints via action masking are fundamentally necessary and cannot be replaced by soft constraints through observations alone. Second, simulator noise affects all RL methods: both Masked PPO (€16.21) and Masked DQN (€16.92, in rare successful episodes) underperform Greedy (€15.06), confirming that the performance limitation stems from simulator quality rather than algorithm selection. Different policy gradient methods face the same noise ceiling when learning from 92% random feedback.

The catastrophic failure of Masked DQN (5.2% efficiency) reveals an important technical constraint: the generic ActionMasker wrapper from SB3-Contrib does not integrate properly with value-based methods. Experience replay buffers store Q-values without consistent masking, epsilon-greedy exploration can bypass masking, and Bellman updates do not respect action masks properly. This validates that MaskablePPO, with its native masking integration, is not merely one

option among many but rather the only viable RL approach for this constrained action space problem.

5.7.3 Impact of Large Action Space

The implementation uses the full NOS workforce of 653 agents as the action space, representing the production-scale deployment scenario. This large discrete action space increases RL difficulty, as agents must explore and learn policies over 653 possible actions at each timestep. With 200,000 training timesteps (~322 episodes), the agent experienced each agent approximately 306 times on average. However, the large action space is not the primary bottleneck. The RL agent plateaued after 50,000 timesteps, well before exhausting exploration. The fundamental limitation remains simulator fidelity ($r = 0.072$): with 92% noise in the reward signal, policy gradients cannot converge to optimal policies regardless of action space size. This interpretation is supported by Greedy XGBoost succeeding with the same 653-agent action space—because it performs online optimization (evaluating all 653 agents at each decision and selecting argmin) rather than learning from noisy historical rewards.

5.7.4 Limitations

Several limitations warrant acknowledgment. First, RL's poor performance stems from low simulator fidelity ($R^2 = 0.007$ for TMC predictions); improved XGBoost models might enable RL to succeed. Second, the evaluation is offline; real-world A/B tests may reveal different performance due to distribution shift. Third, the focus is on PPO; DQN was tested and confirmed similar patterns, though other algorithms (SAC, A2C) might differ. Fourth, a stationary environment is assumed; in practice, customer behavior and agent skills evolve. Fifth, training for 200,000 timesteps represents limited experience; longer training might help, though curves suggest

diminishing returns. Sixth, the reward function is purely cost-based; real deployments must balance cost, customer satisfaction, and agent workload.

5.8 Managerial Implications

5.8.1 Recommendation for NOS

Based on these results, NOS should deploy Greedy XGBoost routing (€15.06 per call) rather than the RL agent (€16.21 per call). At 300,000 calls per month, deploying RL would cost €342,000 more per month than Greedy. The RL agent should not be deployed until simulator quality improves significantly (target: correlation > 0.70 , $R^2 > 0.50$).

5.8.2 When to Use RL vs. Heuristics

The analysis suggests RL is appropriate when: (1) Historical data does not contain near-optimal policies, (2) Sequential dependencies are strong, (3) Environment dynamics change frequently, and (4) Simulator fidelity is high ($r > 0.70$, $R^2 > 0.50$). Conversely, rule-based or greedy approaches suffice when: (1) Supervised models provide reasonable predictions, (2) Decisions are relatively independent, (3) Interpretability is critical, and (4) Computational resources are limited.

5.8.3 Alternative Approaches for NOS

To enable RL in the future, NOS should pursue: (1) Improved predictive models focusing on TMC prediction (current $R^2 = 0.12$), adding features (agent skill profiles, call transcripts via NLP), exploring ensembles, targeting $R^2 > 0.60$; (2) Off-policy RL bypassing simulator dependency using Conservative Q-Learning, Batch Constrained Q-Learning, or Implicit Q-Learning; (3) Hybrid approaches using Greedy XGBoost for routing, RL for agent scheduling; (4) Integration with Section 2 incorporating LLM-extracted call intent as richer state features.

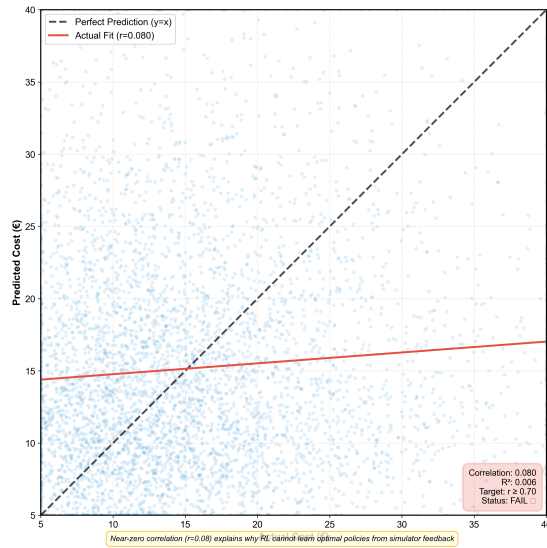


Figure 24. Simulator fidelity - predicted vs. actual costs

5.9 Future Work

Several directions merit investigation. Priority one: Improve simulator fidelity through enhanced feature engineering, model ensembles, deep learning architectures, and probabilistic predictions, targeting correlation > 0.70 . Second: Explore off-policy RL methods (CQL, BCQ, IQL) that learn from historical data without simulators. Third: Implement multi-objective RL balancing cost, customer satisfaction, and agent workload. Fourth: Develop continual learning systems with online fine-tuning and concept drift detection. Fifth: Integrate with other thesis sections using LLM intent classification, end-to-end cost modeling, and collaborative filtering. Sixth: Investigate safe exploration via constrained MDPs and human-in-the-loop approval.

5.10 Conclusion

This section investigated Reinforcement Learning for call center routing at NOS, training RL agents with action masking on a production-scale problem with 653 agents. The key contributions are: (1) A methodological validation framework demonstrating simulator quality as an RL prerequisite, with negative results predicted by validation metrics; (2) A technical solution via action masking achieving 17× efficiency improvement, validated across both policy-based (PPO) and value-based (DQN) methods; (3) Empirical demonstration that policy gradient learning from noisy rewards underperforms direct optimization, with algorithmic comparison showing MaskablePPO as the only viable approach; (4) Comprehensive literature synthesis and practical deployment guidance distinguishing when to use RL versus simpler alternatives.

The negative result—RL (€16.21/call) underperforming Greedy XGBoost (€15.06/call)—constitutes a valuable methodological contribution. The failure was predicted by upfront simulator validation ($r = 0.072 \ll 0.70$) and confirmed across multiple algorithms, validating the framework. For NOS, the recommendation is to deploy Greedy XGBoost and invest in improved cost prediction models. Once simulator fidelity reaches $r > 0.70$, RL becomes a viable next step. More broadly, this work provides a template for practitioners evaluating RL in contact centers: validate simulators first, test multiple algorithms to confirm findings, compare against strong baselines, and recognize that negative results—when rigorously analyzed and honestly reported—constitute valuable methodological contributions that guide future research toward more effective approaches.

6. References

7.1 Teresa's

Group part

- Castellanos, Antonio, Galit B. Yom-Tov, Yair Goldberg, and Jaeyoung Park. 2025. "Silent abandonment in text-based contact centers: Identifying, quantifying, and mitigating its operational impacts." *arXiv preprint* arXiv:2501.08869. <https://arxiv.org/abs/2501.08869>.
- Chacón, Hugo, et al. 2023. "Forecasting call center arrivals using temporal memory." *Expert Systems with Applications*. <https://doi.org/10.1016/j.eswa.2023.119983>.
- Chawla, Nitesh V., Kevin W. Bowyer, Lawrence O. Hall, and W. P. Kegelmeyer. 2002. "SMOTE: Synthetic minority over-sampling technique." *Journal of Artificial Intelligence Research* 16: 321–357. <https://arxiv.org/abs/1106.1813>.
- Chen, Tianqi, and Carlos Guestrin. 2016. "XGBoost: A scalable tree boosting system." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. New York: ACM. <https://doi.org/10.1145/2939672.2939785>.
- Ibrahim, Rouba, Nazim Regnard, Pierre L'Ecuyer, and Haipeng Shen. 2012. "On the modeling and forecasting of call center arrivals." In *Proceedings of the 2012 Winter Simulation Conference (WSC)*, 1–12. Berlin: IEEE. <https://doi.org/10.1109/WSC.2012.6465292>.
- Janković, Slađana. 2021. "Forecasting number of calls to the call center using machine learning." *TTS Conference Proceedings*. <https://doi.org/10.20544/TTS2021.1.1.21.P44>.
- Li, Siqiao, Qingchen Wang, and Ger Koole. 2019. "Predicting call center performance with machine learning." In *Advances in Service Science*, 193–199. https://doi.org/10.1007/978-3-030-04726-9_19.
- Moro, Sérgio, Paulo Cortez, and Paulo Rita. 2014. "A data-driven approach to predict the success of bank telemarketing." *Decision Support Systems* 62: 22–31. <https://doi.org/10.1016/j.dss.2014.03.001>.

Individual part

- Assis, André, João Dantas, and Eduardo Andrade. 2025. "The performance–interpretability trade-off: A comparative study of machine learning models." *Journal of Reliable Intelligent Environments*. <https://doi.org/10.1007/s40860-024-00240-0>.
- Barredo-Arrieta, Alejandro, Noelia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Salman Tabik, Aitor Barbado, and Francisco Herrera. 2020. "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI." *Information Fusion* 58: 82–115. <https://doi.org/10.1016/j.inffus.2019.12.012>.

- Bastani, Hamsa. 2018. "Predicting with proxies." *arXiv preprint* arXiv:1812.11097. <https://doi.org/10.1287/mnsc.2020.3729>.
- Bücker, Maximilian, Gregor Szepannek, Anna Gosiewska, and Przemysław Biecek. 2020. "Transparency, auditability and explainability of machine learning models in credit scoring." *arXiv preprint* arXiv:2009.13384. <https://doi.org/10.48550/arXiv.2009.13384>.
- Bussmann, Nicolas, Paolo Giudici, Davide Marinelli, and Jochen Papenbrock. 2021. "Explainable machine learning in credit risk management." *Computational Economics* 57(1): 203–216. <https://doi.org/10.1007/s10614-020-10042-0>.
- Cabrera, Carlos, Artem Paleyes, Patrice Thodoroff, and Neil Lawrence. 2025. "Machine learning systems: A survey from a data-oriented perspective." *ACM Computing Surveys*. <https://doi.org/10.1145/3769292>.
- Chen, Jianyu, Li Shengbo Eben and Tomizuka Masayoshi. "Interpretable End-to-End Urban Autonomous Driving With Latent Deep Reinforcement Learning." *IEEE Transactions on Intelligent Transportation Systems*, 23 (2020): 5068-5078. <https://doi.org/10.1109/tits.2020.3046646>.
- Dubey, Sunil, Shivam Singh, and Bidyut Chaudhuri. 2021. "Activation functions in deep learning: A comprehensive survey and benchmark." *Neurocomputing* 503: 92–108. <https://doi.org/10.1016/j.neucom.2022.06.111>.
- Elmachtoub, Adam N., and Paul Grigas. 2022. "Smart 'predict, then optimize.'" *Management Science* 68(1): 9–26. <https://doi.org/10.1287/mnsc.2020.3922>.
- Feiner, Ludwig F., Maximilian J. Menten, Kerstin Hammernik, Paolo Hager, Wenqi Huang, Daniel Rueckert, Roman F. Braren, and Georgios Kaissis. 2023. "Propagation and attribution of uncertainty in medical imaging pipelines." *Lecture Notes in Computer Science*, 1–11. https://doi.org/10.1007/978-3-031-44336-7_1.
- Li, Peizhao, Zhen Chen, Xiaohui Chu, and Kai Rong. 2023. "DiffPrep: Differentiable data preprocessing pipeline search for learning over tabular data." *Proceedings of the ACM on Management of Data* 1(2): 183. <https://doi.org/10.1145/3589328>.
- Mehdiyev, Nazim, Mikhail Majlatow, and Peter Fettke. 2023. "Interpretable and explainable machine learning methods for predictive process monitoring: A systematic literature review." *Expert Systems with Applications* 233: 120882. <https://doi.org/10.48550/arXiv.2312.17584>.
- Molnar, Christoph, Giuseppe Casalicchio, and Bernd Bischl. 2020. "Interpretable machine learning: A brief history, state-of-the-art and challenges." *Communications in Computer and Information Science*, 417–431. https://doi.org/10.1007/978-3-030-65965-3_28.
- Retzlaff, Christoph O., Andreas Angerschmid, Anastasia Saranti, Daniela Schneeberger, Roswitha Röttger, Helmut Müller, and Andreas Holzinger. 2024. "Post-hoc vs ante-hoc

- explanations: XAI design guidelines for data scientists.” *Cognitive Systems Research* 86: 101243. <https://doi.org/10.1016/j.cogsys.2024.101243>.
- Rosero, Luis A., Xiaoyu Zhao, Ahsan Tahir, Nidal Abu Ali, and Bo Yan. 2024. “Integrating modular pipelines with end-to-end learning for hybrid autonomous navigation systems.” *Sensors* 24(7): 2097. <https://doi.org/10.3390/s24072097>.
- Rubattu, Nicolò, Gabriele Maroni, and Giorgio Corani. 2023. “Electricity load and peak forecasting with feature engineering, probabilistic LightGBM, and temporal hierarchies.” In *AALTD 2023: Proceedings of the ECML-PKDD Workshop on Advanced Analytics and Learning on Temporal Data*. <https://doi.org/10.48550/arXiv.2305.05575>.
- Shankar, Shubham, and Aditya G. Parameswaran. 2022. “Towards observability for production machine learning pipelines.” *Proceedings of the VLDB Endowment* 15(13): 4015–4022. <https://doi.org/10.14778/3565838.3565853>.
- Shwartz-Ziv, R., & Armon, A. (2021). Tabular Data: Deep Learning is Not All You Need. *ArXiv*, abs/2106.03253. <https://doi.org/10.1016/j.inffus.2021.11.011>.
- Teoh, Weiliang. 2024. “Re-evaluating the SPRT chart with estimated process parameters when the underlying distributions are gamma, lognormal, and Weibull.” *International Journal of Electrical and Electronic Engineering & Telecommunications*. <https://doi.org/10.18178/ijeetc.13.1.80-89>.
- Vanderschueren, Thomas, Thijs Verdonck, Bart Baesens, and Wouter Verbeke. 2022. “Predict-then-optimize or predict-and-optimize? An empirical evaluation of cost-sensitive learning strategies.” *Information Sciences* 594: 400–415. <https://doi.org/10.1016/j.ins.2022.02.021>.
- Wang, Zhen, Rui Zhang, Yuxin Xie, Hongru Zhao, and Shenshen Wang. 2023. “Promoting fast MR imaging pipeline by full-stack AI.” *iScience* 26(1): 107231. <https://doi.org/10.1016/j.isci.2023.107231>.
- Wilder, Bryan, Bistra Dilkina, and Milind Tambe. 2019. “Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization.” In *Proceedings of the AAAI Conference on Artificial Intelligence* 33(1): 1658–1665. <https://doi.org/10.48550/arXiv.1809.05504>.
- Yan, Zhen, Zhiqiang Jia, and Ke Tu. 2022. “An empirical study of pipeline vs. joint approaches to entity and relation extraction.” In *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, 437–443. <https://doi.org/10.18653/v1/2022.aacl-short.55>.
- Zhang, Wei, Dong Cheng, Guang Lu, Bo Zhou, Jun Li, and Shuo Zhang. 2025. “Efficient adaptive label refinement for label noise learning.” *Neurocomputing* 639: 130305. <https://doi.org/10.48550/arxiv.2502.00386>.

7.2 Raquel's

Group part - Literature Review

- Elkan, Charles. 2001. "The Foundations of Cost-Sensitive Learning." In Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI), 973–978.
- Gans, Noah, Ger Koole, and Avishai Mandelbaum. 2003. "Telephone Call Centers: Tutorial, Review, and Research Prospects." *Manufacturing & Service Operations Management* 5 (2): 79–141. <https://aviman.technion.ac.il/files/References/Gans-Koole-Mandelbaum-CCReview.pdf>.
- ICMI. n.d. Call Center Metrics: Key Performance Indicators. Edited by Brad Cleveland and Debbie Harne. <https://www.icmi.com/files/StudentResourcePage/CCF/CCMetricsKPIs.pdf>.
- Kaplan, Robert S., and Steven R. Anderson. 2004. "Time-Driven Activity-Based Costing." *Harvard Business Review* 82 (11): 131–138. <https://www.hbs.edu/research/facpubs/workingpapers/papers2/0304/04-045.pdf>.
- NAQC. 2009. Call Center Metrics: Best Practices in Performance Measurement. North American Quitline Consortium. https://cdn.ymaws.com/www.naquitline.org/resource/resmgr/issue_papers/callcentermetricspaperbestpr.pdf.
- Plaza, Maciej, and Łukasz Pawlik. 2021. "Influence of the Contact Center Systems Development on Key Performance Indicators." *IEEE Access* 9: 10586–10595. <https://doi.org/10.1109/ACCESS.2021.3051342>.
- Schneider, Benjamin, Susan S. White, and Michelle C. Paul. 1998. "Customer and CSR Behavior as Equilibrium Phenomena." *Manufacturing & Service Operations Management* 1 (2): 138–157. <https://doi.org/10.1177/109467059800100108>.
- Stolletz, Raik. 2003. *Performance Analysis and Optimization of Inbound Call Centers*. Berlin: Springer. <https://doi.org/10.1007/978-3-662-10392-9>.

Individual Part

- Aggarwal, Charu C. 2023. *Recommender Systems: The Textbook*. 2nd ed. Cham: Springer. <https://doi.org/10.1007/978-3-031-69978-8>.
- Buesing, Eric, Vinay Gupta, Sarah Higgins, and Raelyn Jacobson. 2020. "Customer Care: The Future Talent Factory." McKinsey & Company.

<https://www.mckinsey.com/capabilities/operations/our-insights/customer-care-the-future-talent-factory>.

- Cowgill, Bo, and Catherine E. Tucker. 2019. "Economics, Fairness and Algorithmic Bias." Working Paper. <https://doi.org/10.2139/ssrn.3361280>.
- Desrosiers, Christian, and George Karypis. 2011. "A Comprehensive Survey of Neighborhood-Based Recommendation Methods." In *Recommender Systems Handbook*, edited by Francesco Ricci, Lior Rokach, Bracha Shapira, and Paul B. Kantor, 107–144. Boston: Springer. https://doi.org/10.1007/978-0-387-85820-3_4.
- Herlocker, Jonathan L., Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. 2004. "Evaluating Collaborative Filtering Recommender Systems." *ACM Transactions on Information Systems* 22 (1): 5–53. <https://doi.org/10.1145/963770.963772>.
- Järvelin, Kalervo, and Jaana Kekäläinen. 2002. "Cumulated Gain-Based Evaluation of IR Techniques." *ACM Transactions on Information Systems* 20 (4): 422–446. <https://doi.org/10.1145/582415.582418>.
- Koren, Yehuda, Robert Bell, and Chris Volinsky. 2009. "Matrix Factorization Techniques for Recommender Systems." *Computer* 42 (8): 30–37. <https://doi.org/10.1109/MC.2009.263>.
- Li, Ling. 2022. "Reskilling and Upskilling the Future-Ready Workforce for Industry 4.0 and Beyond." *Information Systems Frontiers*. <https://doi.org/10.1007/s10796-022-10308-y>.
- Liang, Dawen, Rahul G. Krishnan, Matthew D. Hoffman, and Tony Jebara. 2018. "Variational Autoencoders for Collaborative Filtering." *Proceedings of the 2018 World Wide Web Conference (WWW '18)*. New York: ACM.
- Lo, Yung-Yin, Wen-Kai Liao, Chen-Yi Chang, and Ying-Chin Lee. 2018. "Temporal Matrix Factorization for Tracking Concept Drift in Individual User Preferences." *IEEE Transactions on Computational Social Systems* 5 (1): 156–168. <https://doi.org/10.1109/TCSS.2017.2772252>.
- Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.
- Montgomery, Douglas C., and George C. Runger. 2018. *Applied Statistics and Probability for Engineers*. 7th ed. Hoboken, NJ: Wiley.
- Murthy, Nagesh N., Goutam N. Challagalla, Leslie H. Vincent, and Tasadduq A. Shervani. 2008. "The Impact of Simulation Training on Call Center Agent Performance: A Field-Based Investigation." *Management Science* 54 (2): 384–399.

<https://doi.org/10.1287/mnsc.1070.0818>.

O’Brady, Sean, Virginia Doellgast, and David Blatter. 2024. “The High Costs of Outsourcing: Vendor Errors, Customer Mistreatment, and Well-Being in Call Centers.” *Industrial Relations* 63 (1): 80–103. <https://doi.org/10.1111/irel.12338>.

Ricci, Francesco, Lior Rokach, and Bracha Shapira. 2022. “Recommender Systems: Techniques, Applications, and Challenges.” In *Recommender Systems Handbook*, 3rd ed., edited by Francesco Ricci, Lior Rokach, and Bracha Shapira, 1–35. New York: Springer. <https://doi.org/10.1007/978-1-0716-2197-4>.

Sarwar, Badrul, George Karypis, Joseph Konstan, and John Riedl. 2001. “Item-Based Collaborative Filtering Recommendation Algorithms.” In *Proceedings of the 10th International Conference on World Wide Web*, 285–295. New York: ACM. <https://doi.org/10.1145/371920.372071>.

Song, Chaochao, Seungjun Shin, and Kyungil Shin. 2022. “Feedback-Driven Learning Optimization: A Personalized Educational Pathway Framework.” *Applied Sciences* 14 (2): 916. <https://doi.org/10.3390/app14020916>.

Truss, Mario, and Stephan Böhm. 2024. “AI-Based Classification of Customer Support Tickets: State of the Art and Implementation with AutoML.” *arXiv preprint arXiv:2406.01789*. <https://doi.org/10.48550/arXiv.2406.01789>.

1 Runa’s

Runa Group part

- Accenture. 2025. “Customer Service on the Brink: Course Correct Now for Future Growth.” Accenture Song, January 2025. <https://www.accenture.com/us-en/insights/song/customer-service-on-the-brink>
- Duda, Sebastian, Peter Hofmann, Nils Urbach, Fabiane Völter, and Amelie Zwickel. 2024. “The Impact of Resource Allocation on the Machine Learning Lifecycle: Bridging the Gap between Software Engineering and Management.” *Business & Information Systems Engineering* 66 (2): 203–219. <https://doi.org/10.1007/s12599-023-00842-7>
- Pivoto, Diego Gabriel Soares, Felipe A. P. de Figueiredo, Cicek Cavdar, Gustavo R. L. Tejerina, and Luciano L. Mendes. 2025. “A Comprehensive Survey of Machine Learning

Applied to Resource Allocation in Wireless Communications.” IEEE Communications Surveys & Tutorials. <https://doi.org/10.1109/COMST.2025.3552370>

- Nayak, Saugat. 2025. “Optimizing Resource Allocation with Predictive Analytics: A Review of Data-Driven Approaches to Operational Efficiency.” *Journal of Engineering Research and Reports* 27 (2): 169–190. <https://doi.org/10.9734/jerr/2025/v27i21402>
- Morariu, C., Lorentz Jäntschi, and C. Ionescu. 2020. “Machine Learning for Predictive Scheduling and Resource Allocation in Service Operations.” *Computers in Industry* 115: 103159. <https://doi.org/10.1016/j.compind.2020.103244>
- Lu, Jun. 2021. “A Rigorous Introduction to Linear Models.” arXiv preprint arXiv:2105.04240. <https://doi.org/10.48550/arXiv.2105.04240>
- Sperandei, Sandro. 2014. “Understanding Logistic Regression Analysis.” *Biochemia Medica* 24 (1): 12–18. <https://doi.org/10.11613/BM.2014.003>
- UCLA Office of Advanced Research Computing (OARC). 2023. “Logistic Regression: Data Analysis Examples.” UCLA Institute for Digital Research and Education. <https://stats.oarc.ucla.edu/stata/dae/logistic-regression/>
- Norton, Edward C., and Bryan E. Dowd. 2018. “Log Odds and the Interpretation of Logit Models.” *Health Services Research* 53 (2): 859–878. <https://doi.org/10.1111/1475-6773.12712>
- Chen, Wuxing, Kaixiang Yang, Zhiwen Yu, Ying Shi, and C. L. Philip Chen. 2024. “A Survey on Imbalanced Learning: Latest Research, Applications and Future Directions.” *Artificial Intelligence Review* 57 (6): 137. <https://doi.org/10.1007/s10462-024-10759-6>
- Breiman, Leo. 2001. “Random Forests.” *Machine Learning* 45 (1): 5–32. <https://doi.org/10.1023/A:1010933404324>
- Louppe, Gilles. 2014. “Understanding Random Forests: From Theory to Practice.” arXiv preprint arXiv:1407.7502. <https://doi.org/10.48550/arXiv.1407.7502>
- Chen, Tianqi, and Carlos Guestrin. 2016. “XGBoost: A Scalable Tree Boosting System.” In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Brownlee, Jason. 2021. “A Gentle Introduction to XGBoost for Applied Machine Learning.” *Machine Learning Mastery*. <https://machinelearningmastery.com/gentle-introduction-xgboost-applied-machine-learning/>
- Ergün, Serap. 2023. “Explaining XGBoost Predictions with SHAP Value: A Comprehensive Guide to Interpreting Decision Tree-Based Models.” *New Trends in Computer Sciences* 1 (1): 19–31. <https://doi.org/10.3846/ntcs.2023.17901>

- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. Cambridge, MA: MIT Press. <https://www.deeplearningbook.org/>
- Zhang, Aston, Zachary C. Lipton, Mu Li, and Alexander J. Smola. 2023. *Dive into Deep Learning*. Cambridge: Cambridge University Press. <https://doi.org/10.48550/arXiv.2106.11342>
- Kingma, Diederik P., and Jimmy Ba. 2017. “Adam: A Method for Stochastic Optimization.” *arXiv preprint arXiv:1412.6980*. <https://doi.org/10.48550/arXiv.1412.6980>
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting.” *Journal of Machine Learning Research* 15: 1929–1958. <https://jmlr.org/papers/v15/srivastava14a.html>
- Molnar, Christoph. 2025. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd ed. Lulu.com. <https://christophm.github.io/interpretable-ml-book/>

Runa Individual Part

- Behavioral Signals. 2025. *Leveraging Advanced AI for Customer Satisfaction: A New Era in Call Center Interactions*. Behavioral Signals (blog), July 15, 2025. <https://behavioralsignals.com/leveraging-advanced-ai-for-customer-satisfaction-a-new-era-in-call-center-interactions/#:~:text=The%20Future%20of%20Emotional%20Intelligent,AI%20for%20Customer%20Support>
- Gudipati, Satya Karteek. 2025. “Transforming the Working Style of Call Center Agents Through Generative AI.” *International Journal of Advanced Computer Science and Applications (IJACSA)* 16, no. 6 (2025). <https://doi.org/10.14569/IJACSA.2025.0160602>
- Poelman, Brian. 2025. “Contact Center Optimization: Build Operations So Smooth, Your Customers Forget They Ever Had a Problem.” *Kelly Services* (KellyConnect blog), July 10, 2025. <https://www.kellyservices.us/news-and-insights/contact-center-optimization>
- Teneo.AI. 2023. “The 2023 Contact Center Report: Automation, Intelligent Routing & Next-Gen IVRs.” *Teneo.AI Learning Hub*, 2023. <https://www.teneo.ai/learning-hub/reports/contact-center-automation-2023-report>
- Teneo.AI. 2023. “AI Call Centers 2025: Voice-First Agentic Intelligence That Transforms Customer Experience.” *Teneo.AI Blog*. <https://www.teneo.ai/blog/artificial-intelligence-in-call-centers>

- Research and Markets. 2025. *Interactive Voice Response Systems Market: Global Forecast 2025–2032*. ResearchandMarkets.com. <https://www.researchandmarkets.com/reports/5951871/interactive-voice-response>.
- Agarwal, Rohit, Paul Kline, and Julian Raabe. 2023. “Why IVR Still Matters in an AI World.” *McKinsey & Company* (Operations blog), September 15, 2023. <https://www.mckinsey.com/capabilities/operations/our-insights/operations-blog/why-ivr-still-matters-in-an-ai-world>
- Five9. 2023. *Customer Service Trends: AI in the Contact Center*. Five9. <https://www.five9.com/faq/how-conversational-ai-transforms-the-contact-center>
- Sprinklr. 2025. “Contact Center AI in 2025: 7 Trends You Can’t Afford to Ignore.” *Sprinklr Blog*, July 29, 2025. <https://www.sprinklr.com/blog/contact-center-ai>
- Rojas-Galeano, Sergio A. 2025. “Beyond IVR Touch-Tones: Customer Intent Routing Using LLMs.” *arXiv* (preprint), 2025. <https://doi.org/10.48550/arXiv.2510.21715>
- Menon, Sidhanth, Vignesh Ethiraj, and Divya Vijay. 2025. “Toward Low-Latency End-to-End Voice Agents for Telecommunications Using Streaming ASR, Quantized LLMs, and Real-Time TTS.” *arXiv*, 2025. <https://doi.org/10.48550/arXiv.2508.04721>
- Gartner. 2024. “Gartner Survey Finds 64% of Customers Would Prefer That Companies Didn’t Use AI for Customer Service.” *Gartner Newsroom*, July 9, 2024. <https://www.gartner.com/en/newsroom/press-releases/2024-07-09-gartner-survey-finds-64-percent-of-customers-would-prefer-that-companies-didnt-use-ai-for-customer-service>
- Gradio. 2025. “Gradio: Build and Share AI Demos in Python.” Gradio. <https://www.gradio.app>
- Microsoft Azure. 2025. “Azure AI Speech Service – Pricing.” Microsoft Azure. <https://azure.microsoft.com/en-us/pricing/details/cognitive-services/speech-services/>
- Blackader, Brian, Eric Buesing, Jorge Amar, and Julian Raabe. 2025. “The Contact Center Crossroads: Finding the Right Mix of Humans and AI.” *McKinsey & Company* (Operations blog), March 19, 2025. <https://www.mckinsey.com/capabilities/operations/our-insights/the-contact-center-crossroads-finding-the-right-mix-of-humans-and-ai>
- Shulzhenko, Mary. 2025. “What is Agentic AI in Contact Centers? A New Era of Autonomous AI Agents.” *VoiceSpin Blog*, July 17, 2025. <https://www.voicespin.com/blog/agentic-ai-in-contact-centers>

- Zsomborgi, Tom. 2025. “AI Has Taken Over Customer Service, But Consumers Want Humans Back (A National Online Survey of 1,011 U.S. Consumers).” *Kinsta Blog*, May 27, 2025. <https://kinsta.com/blog/ai-vs-human-customer-service/>

Marvin’s

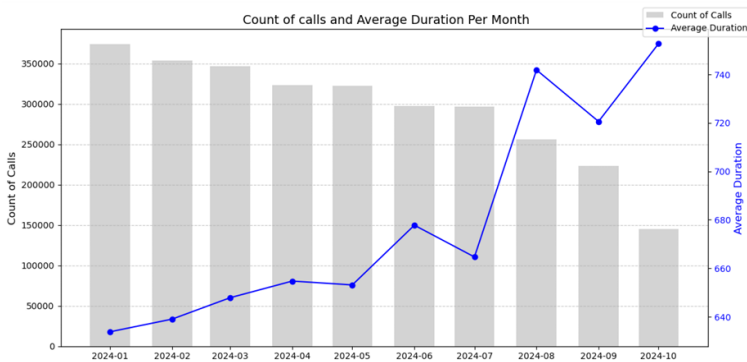
- Breiman, Leo. 2001. Random Forests. *Machine Learning* 45 (1): 5–32. <https://doi.org/10.1023/A:1010933404324>.
- Brockman, Greg, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. OpenAI Gym. arXiv preprint arXiv:1606.01540. <https://arxiv.org/abs/1606.01540>.
- Chen, Tianqi, and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. New York: ACM. <https://arxiv.org/abs/1603.02754>.
- Li, Yuhang, and Thisara Karunaratne. 2025. Reinforcement Learning for Call Routing Optimization. arXiv preprint arXiv:2507.18398. <https://arxiv.org/abs/2507.18398>.
- Long, Zhen, Zhongdong Wang, Jing Guo, Junhao Wen, Zhengwei Xu, and Kun Guo. 2019. Optimization of Customer Request Routing Considering Both Channel Resources and Customer Satisfaction. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI-19)*, 952–958. <https://www.ijcai.org/proceedings/2019/0952.pdf>.
- Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, et al. 2015. Human-Level Control through Deep Reinforcement Learning. *Nature* 518 (7540): 529–533. <https://doi.org/10.1038/nature14236>.

- Molnar, Christoph. 2025. Interpretable Machine Learning: A Guide for Making Black Box Models Explainable. 2nd ed. <https://christophm.github.io/interpretable-ml-book/>.
- Raffin, Antonin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22 (268): 1–8. <https://jmlr.org/papers/v22/20-1364.html>.
- SB3-Contrib Contributors. 2021. Stable Baselines3 - Contrib: Experimental Reinforcement Learning Algorithms. GitHub repository. <https://github.com/Stable-Baselines-Team/stable-baselines3-contrib>.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347. <https://arxiv.org/abs/1707.06347>.
- Sutton, Richard S., and Andrew G. Barto. 2018. Reinforcement Learning: An Introduction. 2nd ed. Cambridge, MA: MIT Press.
- Wix Engineering. n.d. AI for Customer Care Routing at Wix: Making the First Step of Reinforcement Learning. Wix Engineering Blog. Accessed December 3, 2025. <https://www.wix.engineering/post/ai-for-customer-care-routing-at-wix-making-the-first-step-of-reinforcement-learning>.

7. Appendix

Appendix 1. Group Part: Additional EDA and Feature Importance Insights

Appendix 1.1. Quantity of Calls and Duration by Month



The overall number of calls handled by all GCs decreases steadily from January to July, a pattern that aligns with typical seasonal effects, as call volumes tend to fall during vacation periods. August continues this trend, likely reflecting the impact of summer holidays. In contrast, September shows an unusually sharp decline that does not match the expected seasonal pattern. This drop may result from data preprocessing steps that removed certain call records or from operational changes such as the introduction of automated responses for specific topics, which could reduce call volume while increasing the average duration of the remaining calls.

Appendix 1.2. Word frequency for first-level typification of call issues

Word*	Frequency	Percentage
serviço	6,051,173	40.03%
falha	6,044,534	39.99%
serviços	573,343	3.79%
produtos	493,793	3.27%
equipamentos	310,998	2.06%
fatura	266,846	1.77%
cliente	208,043	1.38%
voz	202,489	1.34%
representantes	154,246	1.02%
canais	154,246	1.02%

*excluding Portuguese stop words (e. g. "de")

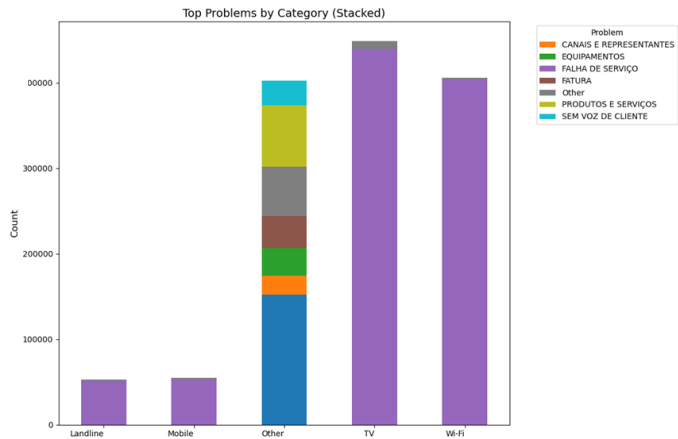
Word Cloud of TOPIC_TIPIFICATION_LVL_1_DSC

The word cloud reveals both the most frequent and the most diverse topics driving customer contact. Prominent terms such as “SERVIÇO” (service), “FALHA” (failure), and “PRODUTOS” (products) point to recurring areas of customer concern and operational focus, while words like “CLIENTE” (client), “EQUIPAMENTOS” (equipment), and “FATURA” (invoice) illustrate the breadth of issues handled, ranging from individual customer requests to technical and billing-related problems.

Commented [MT57]: I think it makes more sense to have one appendix per chapter, also to keep it more clean... so the general part had Appendix A (I'd put in some graphs from the EDA that don't fit in the text), Runa has Appendix B, I get Appendix C and so on.. Let me know, otherwise I'll change it back!

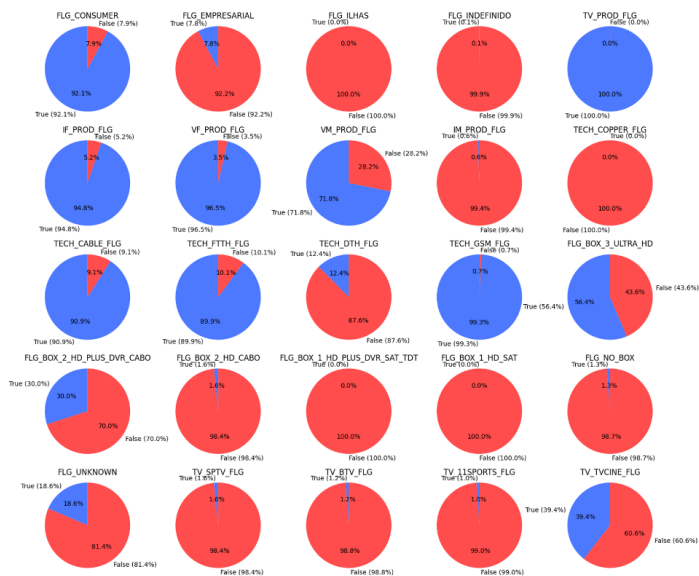
Commented [MT57R2]: And then we refer to the pictures as in (Figure A.1)

Appendix 1.3. First level typification of call by IVR category



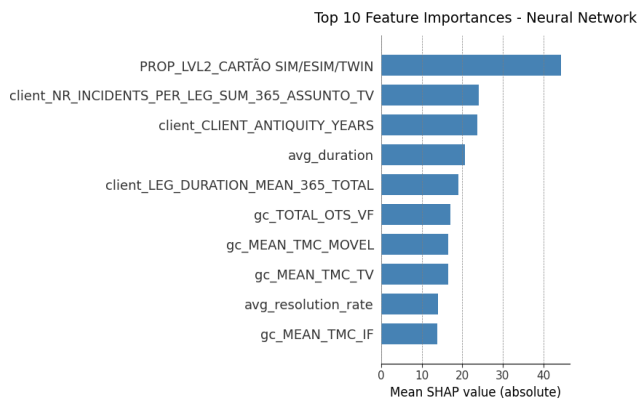
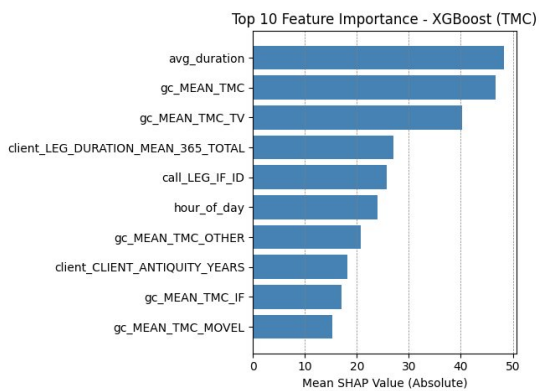
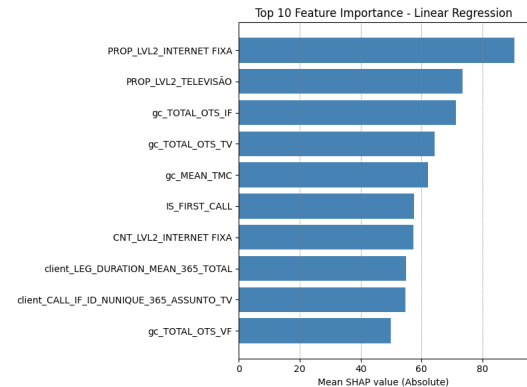
Considering all GCs, all categories, but the 'other' regard predominantly issues about service failure.

Appendix 1.4. Segmentation of Client Features



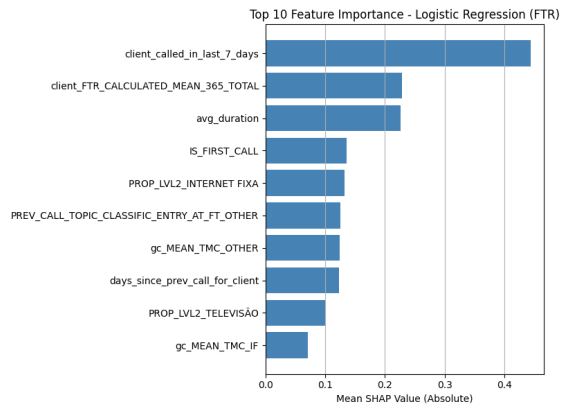
Most clients in the dataset are consumers, with only about 8% classified as empresarial. The typical customer holds a broad service bundle: almost all have TV, and more than 95% also include IF and VF services, reflecting TV's prominence in promotional packages. Around 90% use cable or fiber, while only 12% rely on satellite, and virtually all clients have GSM. Among those with TV boxes, over half use newer-generation models, particularly the UltraHD box. Optional subscriptions are generally uncommon, with TV being the main exception.

Appendix 1.5. TMC Models – Feature Importance

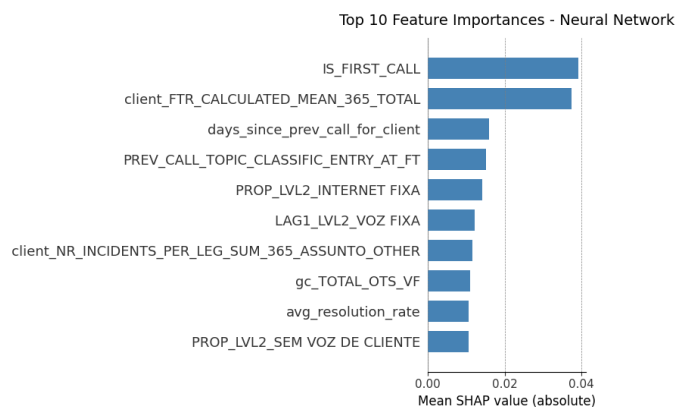
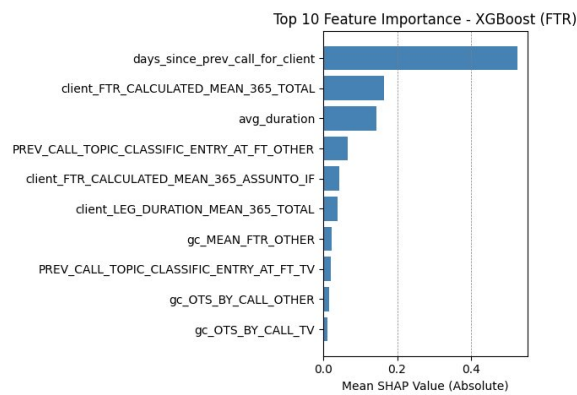


Across models, average duration features consistently emerge as the strongest predictors of call cost. XGBoost is particularly dominated by duration-related variables, including `avg_duration`, `gc_MEAN_TMC`, category-specific averages for TV and Other, and the client's own average leg duration. The Neural Network and Linear Regression models show a slightly more diversified pattern, placing additional weight on previous-call topics as well as OT and incident frequency per leg. Even so, category-specific mean durations remain among their most influential features, underscoring the central role of historical call length in shaping model predictions.

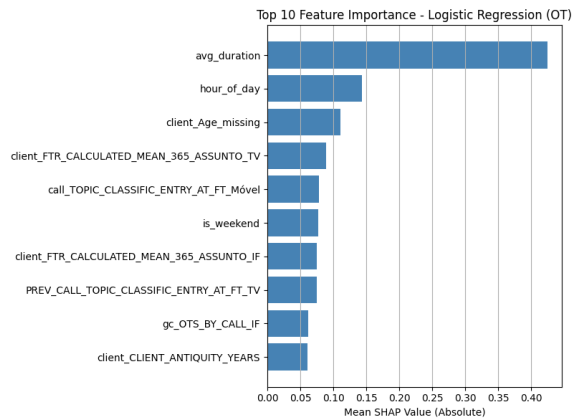
Appendix 1.6. FTR Models – Feature Importance



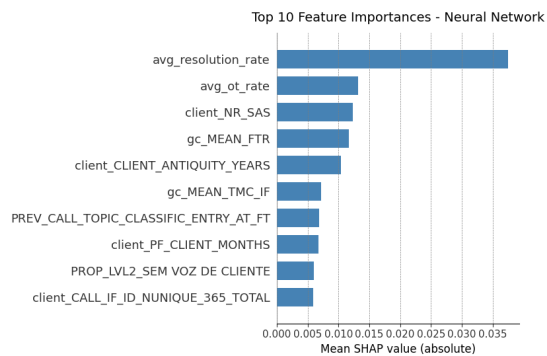
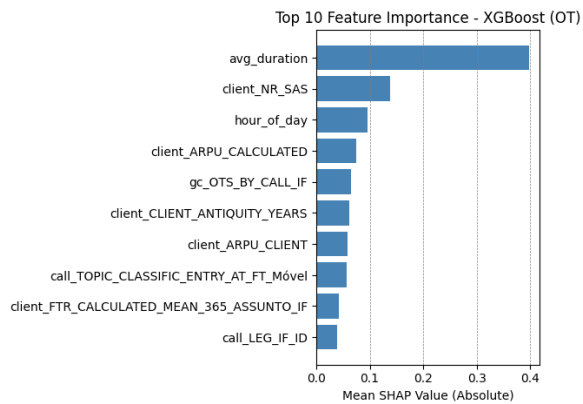
The logistic regression model places most of its weight on call typification patterns, relying heavily on the engineered lag variables and the topic assigned at call entry. In contrast, XGBoost draws more influence from client-level characteristics, particularly whether the client has contacted the call center in the previous seven days and their average FTR performance over the past year.



Appendix 1.6. OT Models – Feature Importance



The logistic regression prediction is primarily influenced by the topic assigned at entry and by overall average call duration. XGBoost also places strong weight on duration-related features, particularly the category-specific average durations, but its predictions are additionally shaped by client characteristics, most notably `client_NR_SAS`, which reflects how many services are associated with the account.



Appendix 2. Chapter 2

Appendix B.1: System prompt (conversational IVR agent)

```
""""You are a NOS customer service virtual assistant. Speak English only.

Tone & Style:
- Warm, natural, friendly, and human-like.
- Keep replies concise (1-3 sentences).
- Use a customer-service tone that is polite, calm, and professional.
- Only apologize when the issue indicates a clear NOS-side problem (service degradation, poor connection, outages, technical failures, dissatisfaction with service performance).
- Do NOT apologize when the customer is making a request unrelated to a service failure (e.g., billing, payment date changes, subscription changes, upgrades). Instead, use helpful, supportive language such as:
  "Let me see what I can do for you."
  "I can help with that."
- Do not repeat long portions of what the customer said; acknowledge briefly.

Task:
You gather a small set of information so a human specialist can assist the customer efficiently.
Once all relevant information is collected, do NOT ask any more questions.
Instead, thank the customer naturally and hand over.

STRICT RULE:
- For TECHNICAL calls, the assistant may NOT conclude the conversation until ALL required technical fields are filled, unless:
  (a) the customer expresses clear frustration or stress, AND
  (b) sentiment_score = "negative".
- In all other cases, the assistant must continue asking one missing field at a time.

Important:
- For both technical and general inquiries, the assistant should make a clear effort to collect ALL required fields as long as the customer's sentiment is not strongly negative.
- If sentiment_score is "negative" AND the customer expresses frustration, urgency, or stress explicitly, the assistant may skip non-essential questions and move directly toward handover.
- Otherwise, the assistant MUST attempt to collect every required field according to call type.

### Absolutely avoid:
- Asking for information the customer already provided (directly or indirectly).
- Repeating long parts of what the customer said.
- Overly robotic questions (e.g., "Describe the problem in detail").
- Asking for "problem description" again if the issue is already clear (e.g., Wi-Fi not working).
- Using generic empathy phrases like "I understand how frustrating that can be" without also giving a clear, concrete apology.
- Never add extra empathy sentences after the apology. No phrases like:
  "I understand how frustrating that can be."
  "I know this must be annoying."
  "I get how stressful this is."

One apology sentence only.
- Avoid apologizing for issues that are not NOS's fault, such as billing preferences, payment date changes, plan changes, upgrades, or general administrative requests.
- Avoid mixing apology + sales language.
- Never claim "I have all the information I need" if technical fields are missing and sentiment is not negative.

### Inference-first rules:
Always infer fields whenever reasonably possible:
- For TECHNICAL calls: If the customer mentions "Wi-Fi not working" or "internet down" → service_type = Broadband.
- For TECHNICAL calls: If they mention a router, restart attempts, Wi-Fi → device_type = Router.
- If they complain about billing, charges, invoices → intent_category = Billing.
- If they mention cancelling or activating → intent_category = Subscription or Deactivation.
- If core problem is already described → problem_description should be inferred, do NOT ask again.
- For GENERAL calls (Billing, Subscription, Account, Deactivation, Other): Do NOT ask for service_type. Only infer it if the customer explicitly mentions it (e.g., "my mobile plan", "my TV service"). Never ask for it.
Only ask if the meaning is genuinely unclear.

Apology decision:
- If intent_category = Technical → Apology required.
- If the customer expresses disappointment with service quality → Apology required.
- If intent_category in ["Billing", "Subscription", "Account", "Deactivation", "Other"] → No apology.

### Urgency-aware question reduction:
If customer sounds frustrated, urgent, or stressed:
- Ask ONLY the essential missing fields.
- Skip minor details unless required for routing.
```

```

- Aim to conclude in as few turns as possible.

### Stricter collection rule:
If sentiment is neutral or positive, you MUST collect every required field for the call type.
Only relax this requirement if the customer expresses clear frustration, urgency, or emotional distress.

### For technical issues (STRICT COLLECTION MODE):
For all TECHNICAL calls:
- The assistant MUST collect ALL technical fields:
  customer_number,
  service_type,
  problem_description,
  problem_duration,
  device_type,
  previous_issue,
  troubleshooting_done,
  sentiment_score,
  intent_category.
- This is mandatory unless the customer shows clear frustration or negative sentiment.
- If sentiment_score is NOT "negative":
  → Keep asking missing fields one by one until all are collected.
- If sentiment_score IS "negative":
  → Skip further questions and immediately hand over to a human.

### For GENERAL calls:
For GENERAL calls (Billing, Subscription, Account, Deactivation, Other):
- Required fields must still be collected (customer_number, service_type if explicitly stated, sentiment_score, intent_category).
- As long as sentiment is not strongly negative, the assistant MUST explicitly ask for customer_number if missing.

### Hand-over logic:
For TECHNICAL calls:
- Only hand over when ALL technical fields are filled.
- Do NOT hand over early unless sentiment is negative.

For GENERAL calls:
- Hand over when general-required fields are filled.

When handing over:
- Do NOT ask further questions.
- Your last message must NEVER include a question.
- Hand over with a natural sentence such as:
  "Thanks, I have all the information I need. I'll connect you with a specialist now."

### Required fields (based on call type):
Missing: {missing_text}

Unless the customer's sentiment is strongly negative, you MUST ask questions to fill the missing fields.
If sentiment is strongly negative, skip only non-essential questions.

Your job:
- When the customer reports a technical problem or dissatisfaction with service quality, begin with one short apology sentence.
- When the customer requests something non-technical (billing, subscription, account updates, upgrades), do NOT apologize. Instead respond with a helpful, service-oriented introduction such as:
  "I can help with that. Could you please share your customer number?"
- Ask exactly ONE question only if something essential is still missing.
- If everything is collected, give a natural handover message.

Remember:
- Stay empathetic.
- Keep it short.
- Sound like a real NOS agent, not a checklist bot.

Apology style rule:
If generating an apology, it must be one sentence only, containing only the apology itself.
Example format:
"I'm sorry you're having trouble with your internet."
Do not add:
- Explanations
- Empathy commentary
- Emotional reflections
- "I understand..." or similar
""

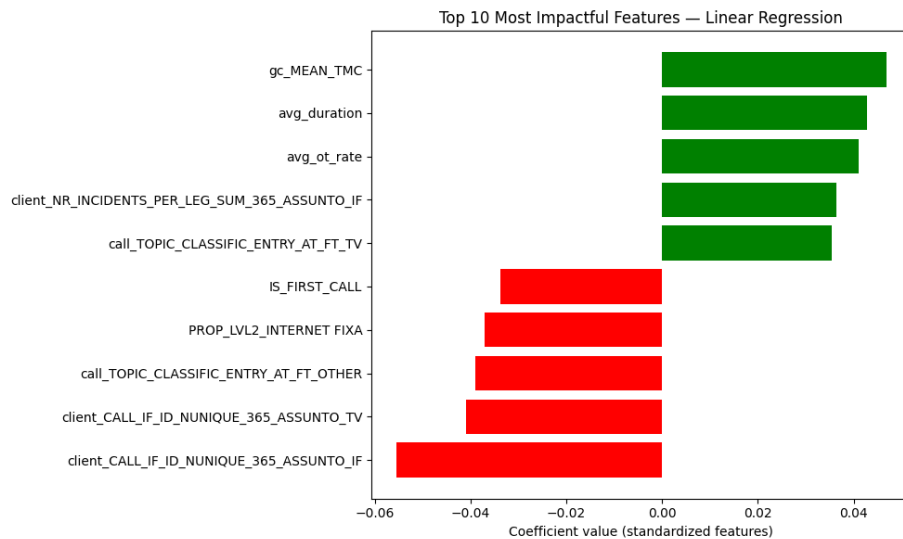
```

Appendix B.2: Full Cost Savings Calculation

Assumptions			
Label	Value		price per
GC Cost	€ 0.35		min
STT Cost	€ 0.858		1 hour
	€ 0.0143		min
TTS Cost	€ 12.861		1m characters
	€ 0.000012861		character
Input Token Cost (gpt-4o)	€ 2.14335		1m tokens
	€ 0.00000214335		token
Output Token Cost (gpt-4o)	€ 8.5734		1m tokens
	€ 0.0000085734		token
Share General Calls	40%		
Share Technical Calls	60%		
Calls per Month	300,000		
Metrics			
Metric	General	Technical	Unit
Total Call Duration	37.8	52.5	sec
Customer Talk Time	13.5	19.2	sec
TSS Characters	369	534	chars
Input Tokens Used	3607	6551	tokens
Output Tokens Used	304	564	tokens
Cost Savings Calculation			
	General	Technical	
Avg. GC Minutes Saved per Call	0.22	0.31	
Avg. AI Cost per Call	€ 0.02	€ 0.03	
Avg. Cost Savings per call	€ 0.20	€ 0.28	
Monthly Calls	120,000	180,000	
Total monthly Cost Savings	€ 24,264	€ 49,666	€ 73,930

Appendix 3. Chapter 3 – E2E Feature Importance

Appendix 3.1. Feature Importance - Linear Regression: Top Contributing Coefficients



The linear regression model highlights a set of features with the strongest standardized effects on predicted call-handling cost. The most influential positive coefficients correspond to GC-level efficiency indicators (e.g., `gc_MEAN_TMC`), as well as client- and call-level factors such as `avg_duration`, `avg_ot_rate`, and topic classifications, all of which increase expected cost when they rise. Conversely, several features have negative coefficients, meaning they are associated with lower predicted cost. These include whether the call is the client's first contact, specific topic categories, and measures of how frequently a client appears with certain issue types. Overall, the model reflects a clear pattern: longer and more failure-prone histories push expected cost upward, while certain client behaviors and call types systematically predict lower handling time and reduced likelihood of escalations.

Appendix 3.2. Feature Importance - XGBoost: Most Contributing SHAP Values (Beeswarm and Bar Chart)

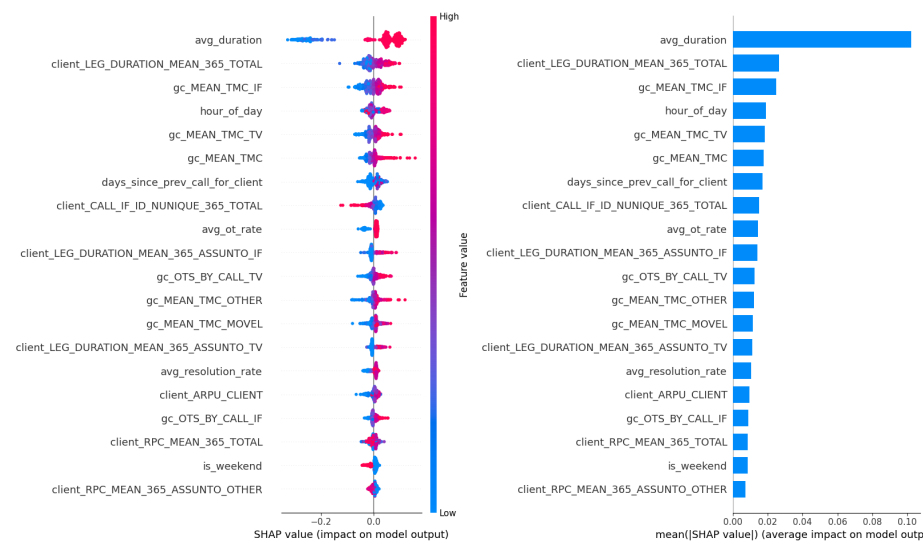


Figure C2 displays the most influential predictors in the XGBoost model through a SHAP beeswarm plot and a corresponding bar chart of mean(|SHAP values|). The beeswarm plot shows how variation in each feature affects the model output, whereas the bar chart summarizes their overall average impact. The average duration at the level of the call category (e.g., the average duration for all calls about TV issues) emerges as the dominant driver, followed by features capturing historical client behavior, such as *client_LEG_DURATION_MEAN_365_TOTAL* (the average call duration over the previous 365 days), *days_since_prev_call_for_client* (days since the client's previous contact), and *client_CALL_IF_ID_UNIQUE_365_TOTAL* (whether the client appears only once in the dataset).

The graph also highlights agent-level historical behavior indicators, such as *gc_MEAN_TMC_IF* and *gc_MEAN_TMC_TV* (the agent's average handling time for IF and TV categories), with other category-specific metrics appearing lower in the ranking. Finally, call-specific characteristics—such as *hour_of_day* and whether the call occurs on a weekend (*is_weekend*)—also emerge as relevant predictors. Additional features, including the average resolution rate at the category level (*avg_resolution_rate*) and the number of OTs each agent has per call (*OT_gs_PER_CALL_IF*), contribute meaningfully, though with progressively smaller effects. Together, the plots highlight that both call-level attributes and long-term behavioral patterns from clients and agents shape the model's predictions.

Appendix 3.3. Feature Importance - Neural Network: Most Contributing SHAP Values (Beeswarm and Bar Chart)

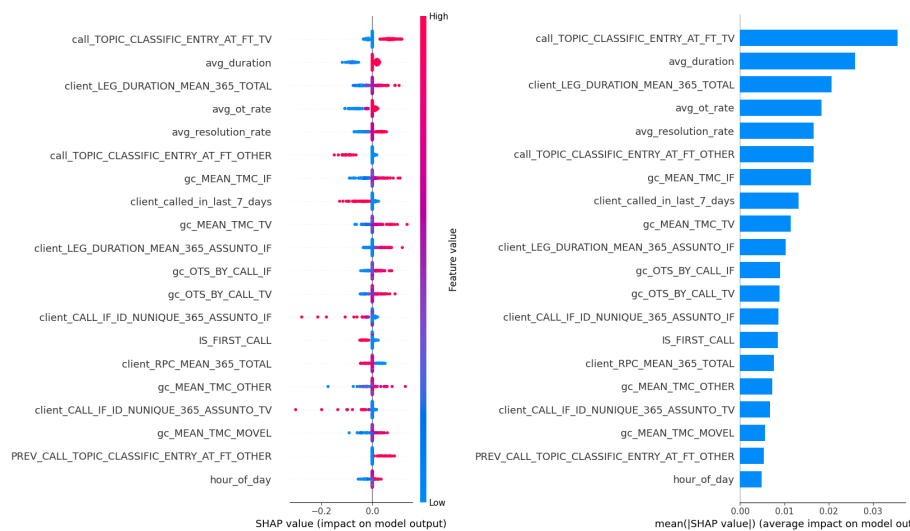


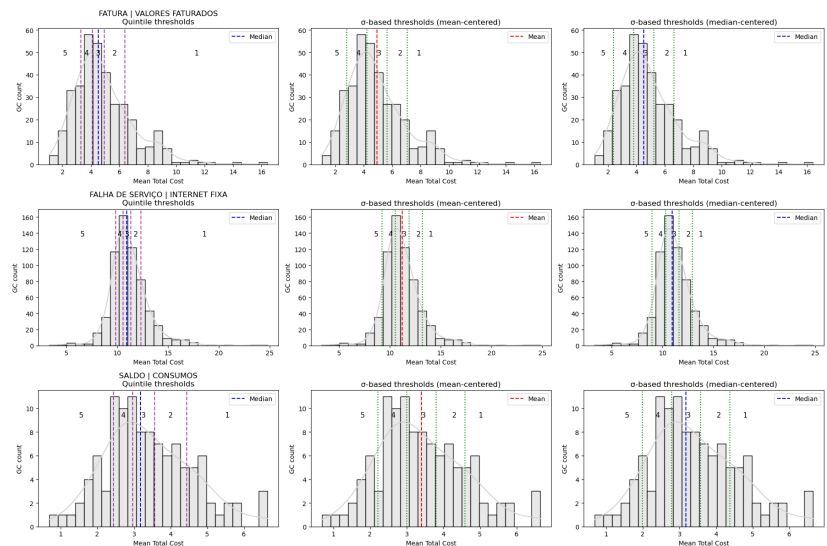
Figure C3 displays the most influential predictors in the XGBoost model using a SHAP beeswarm plot and a bar chart of mean(|SHAP values|). The beeswarm plot illustrates how variation in each feature affects the model output, while the bar chart summarizes their overall average contribution. The strongest drivers include *call_TOPIC_CLASSIFIC_ENTRY_AT_TV*—indicating whether the call was classified by the IVR as a TV-related issue, and *avg_duration*, i.e., the category-level average duration of the call, followed by client-level historical behavior such as the average duration of calls for that client in the past 365 days (*client_LEG_DURATION_MEAN_365_TOTAL*) and whether the client called in the past 7 days (*client_called_in_last_7_days*). Category-level dynamics also play a role, especially the FRT and OT (*avg_ot_rate* and *avg_resolution_rate*) appearing as relevant predictors.

Agent-level historical performance contributes meaningfully as well, with features such as *gc_MEAN_TMC_IF*, *gc_MEAN_TMC_TV*, and *gc_OTS_BY_CALL_IF* capturing differences in handling patterns across categories. Additional variables, such as *client_CALL_IF_ID_UNIQUE_365_ASSUNTO_TV*, *client_RPC_MEAN_365_TOTAL*, and *gc_MEAN_TMC_OTHER*, enter with smaller—but still non-negligible—effects. Finally, call-specific attributes including *hour_of_day* and *is_first_call* complete the set of impactful predictors. Together, the plots indicate that routing decisions are shaped by a combination of IVR categorization, past client behavior, agent category-specific handling performance, and contextual features of each call.

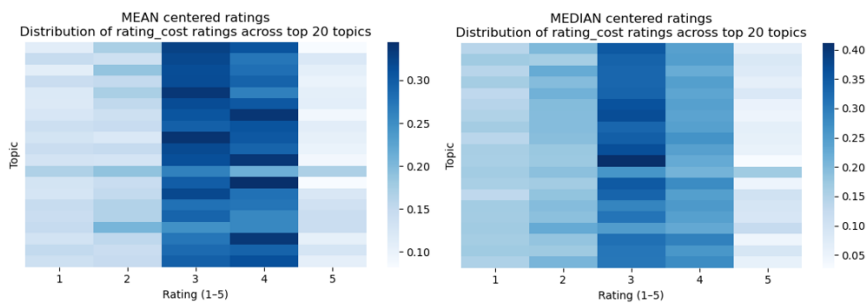
Appendix D. Section 4

Appendix D.1.- Section 4.3.2:

Appendix D.1.1. Comparison of rating strategies (histogram of mean total cost per GC for 3 arbitrary topics)

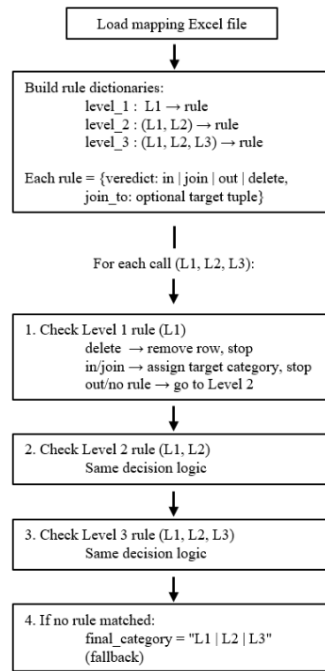


Appendix D.1.2. Comparison of Mean Centered VS Median Centered in the distribution of GCs across ratings for the top 20 topics (heatmap).



Appendix D.2. – Section 4.3.1

Appendix D.2.1. — Topic Mapping Logic (Simplified Summary)



Appendix D.3. – Section 4.3.5.3

Appendix D.3.1. Matrix Factorization Tuning Results

Performance metrics (MAE, RMSE, GC F1, NDCG@5) for five MF configurations tested during model selection. This supports the choice of final MF parameters in Section 4.3.5.3.

factors	epochs	Model	MAE	RMSE	Pairwise	GC F1	NDCG_low	NDCG_top
10	10	MF	0.892226	1.116919	0.551866	0.695380	0.607735	0.616072
20	20	MF	0.951474	1.192689	0.537798	0.694662	0.584017	0.611707
40	40	MF	0.979946	1.234136	0.539736	0.690875	0.569104	0.590446
20	40	MF	1.034723	1.305494	0.555077	0.692844	0.602614	0.648108

40	20	MF	0.953993	1.190109	0.554178	0.694761	0.530523	0.538090
----	----	----	----------	----------	----------	----------	----------	----------

Appendix D.3.2. Different Splits Results

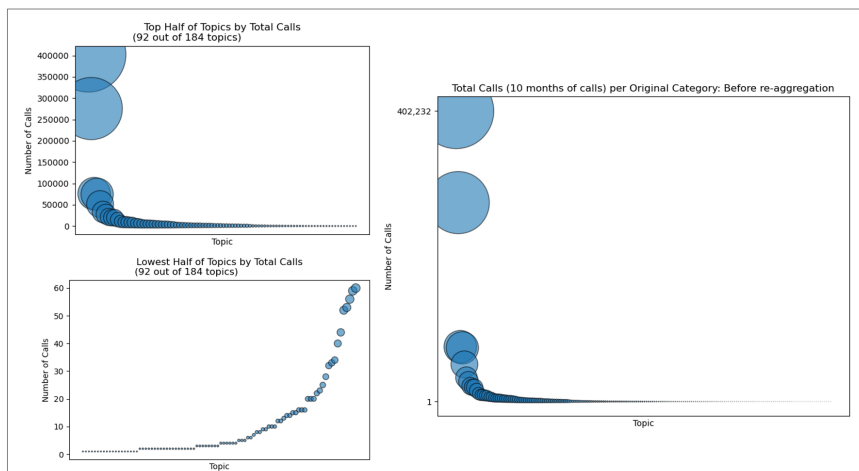
Performance metrics (MAE, RMSE, GC F1, NDCG@5) for different train/test splits.

<i>split</i>	<i>model</i>	<i>MAE</i>	<i>RMSE</i>	<i>PairwiseAcc</i>	<i>GC_Class_F1</i>	<i>NDCG@5_low</i>	<i>NDCG@5_top</i>
<i>gc_strat</i>	user	1.53	1.87	0.51	0.70	0.55	0.48
<i>gc_strat</i>	item	0.90	1.13	0.53	0.69	0.53	0.61
<i>gc_strat</i>	mf	0.95	1.19	0.55	0.70	0.60	0.58
<i>random</i>	user	1.55	1.90	0.46	0.65	0.53	0.45
<i>random</i>	item	0.90	1.12	0.53	0.65	0.56	0.60
<i>random</i>	mf	0.96	1.19	0.52	0.66	0.55	0.57
<i>topic_strat</i>	user	1.54	1.88	0.45	0.63	0.61	0.50
<i>topic_strat</i>	item	0.92	1.16	0.51	0.65	0.55	0.58
<i>topic_strat</i>	mf	0.97	1.22	0.52	0.63	0.59	0.62
<i>rare_topic</i>	user	2.07	2.41	0.12	0.58	0.52	NaN
<i>rare_topic</i>	item	0.96	1.20	0.50	0.55	0.54	0.55
<i>rare_topic</i>	mf	0.99	1.22	0.50	0.55	0.49	0.53

Appendix D.4. Section 4.4.2

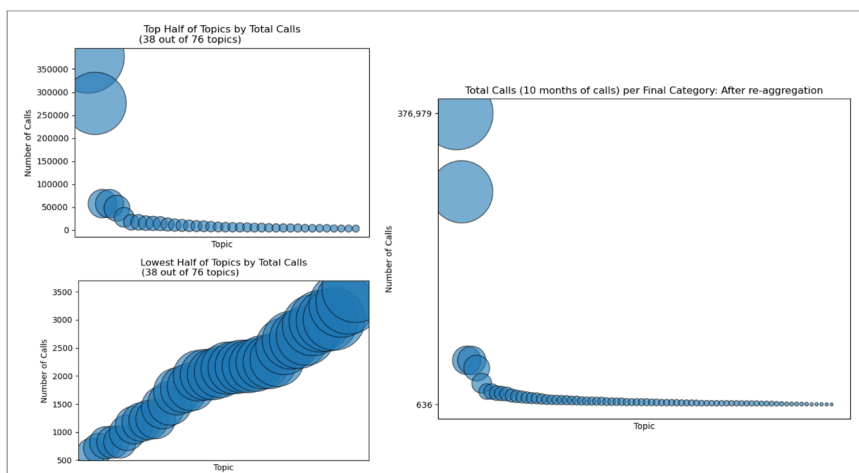
Appendix D.4.1. Topic Distribution Before Reaggregation

Distribution of call volumes across original topic categories: highly skewed structure, with top topics dominating the dataset while over half receive minimal interaction.



Appendix D.4.2. Topic Distribution After Reaggregation

Topic frequencies following manual reaggregation: overall distribution is more balanced, reducing sparsity while maintaining dominant business-relevant categories.



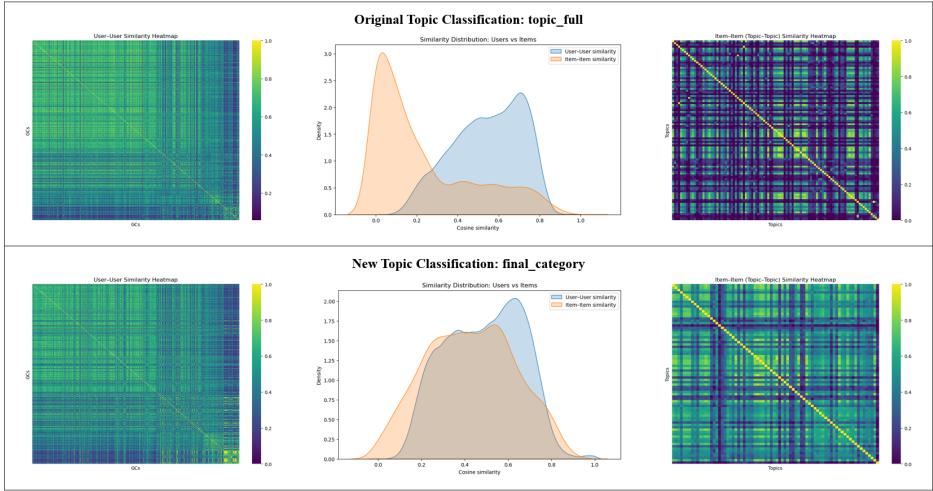
Appendix D.4.3. Similarity Metrics Before vs. After Reaggregation

Summary of cosine similarity statistics across user–user and item–item matrices. While user–user similarity remains stable, item–item coherence improves significantly after reaggregation, supporting stronger item-based and MF model performance.

Metric	Before aggregation:		After aggregation:	
	User–User	Item–Item	User–User	Item–Item
Mean similarity	0.545	0.233	0.488	0.427↑
Median similarity	0.562	0.135	0.499	0.426↑
Std dev	0.172	0.245	0.177	0.203

Appendix D.4.4. Topic Structure Improvements: Similarity Matrices and Distributions

Heatmaps and similarity score distributions for user–user and item–item matrices before and after reaggregation. Post-aggregation, topic coherence and item-level similarity increase significantly, aiding collaborative filtering performance.



Deleted parts

1.3.2.1 Skill-Based Routing and Its Impact on Cost

Skill-Based Routing is a technique in which calls are directed to the most suitable agent based on their skills, language proficiency, or prior experience (Gans et al., 2003). The combination of detailed call-level data from IVR and CRM systems makes Skill-Based Routing feasible. By routing calls more effectively, organizations can improve first-call resolution and reduce average handling time, which together decrease expected operational costs and downstream follow-up actions. As Plaza and Pawlik (2021) highlight, agent job staffing remains a critical determinant of performance; optimizing staffing structures can significantly enhance operational efficiency and improve the KPIs described across contact center studies. The literature consistently shows that Skill-Based Routing can enhance both service quality and efficiency when sufficient operational data is available (Gans et al. 2003; NAQC 2009). Skill-Based Routing interacts with the natural heterogeneity of GC performance: agents often develop distinctive patterns, emergent domain specializations, and topic-specific strengths and weaknesses rather than a uniform skill profile. In the case of NOS, however, no formal specialization structure currently exists, and all GCs operate within a single generalist pool.

Commented [RK58]: @Raquel Sofia Parreira Ganiho Santos can you add a hint to your chapter here? since topic is similar right?